



UE23AM342BA3
Explainable AI

Neural Network Interpretation

Unit: 3

Prof. Ayisha Noori VK

Dept. of CSE(AI&ML), PES University, Bengaluru



Understanding How Neural Networks See, Think & Decide

Contents :

Learned Features

Pixel Attribution

Detecting Concepts

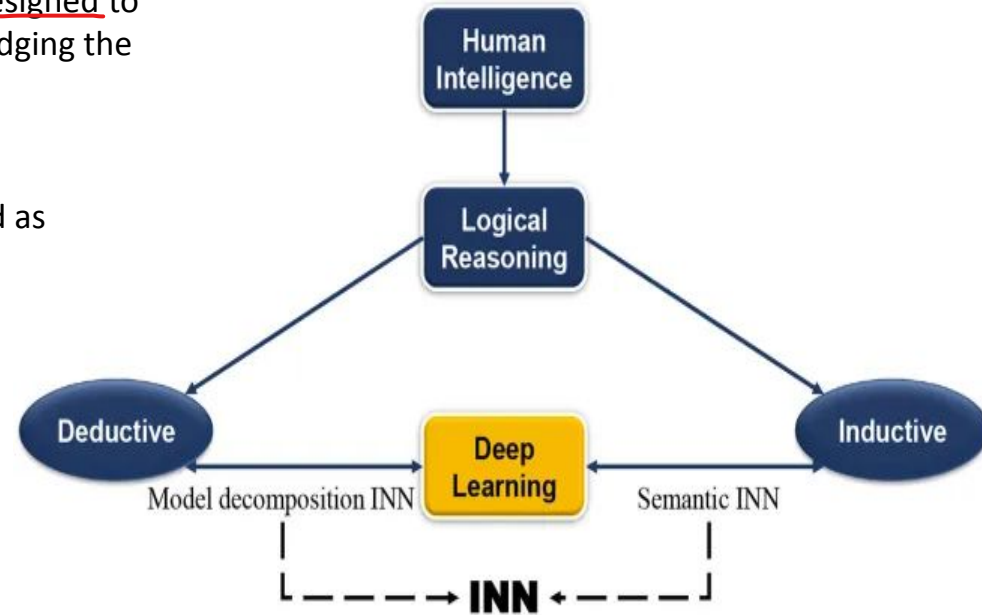
Adversarial Examples

Influential Instances

Neural Network Interpretation

INN opposite black box model

- Interpretable Neural Networks (INNs) are AI models designed to make their decision-making processes transparent, bridging the gap between high-accuracy black-box models and human-understandable logic
- Human intelligence's logical reasoning can be classified as either deductive or inductive reasoning



The Black Box Problem

Modern deep neural networks achieve remarkable accuracy — but remain largely opaque. Explainable AI (XAI) bridges the gap between performance and understanding, enabling trust, debugging, and regulatory compliance.

Trust & Safety

Clinicians, judges, and engineers must understand why a model made a decision before acting on it.

Debugging & Bias

XAI reveals spurious correlations — e.g., a model predicting 'wolf' based on snow backgrounds.

Regulation

GDPR Article 22 requires a 'right to explanation' for automated decisions affecting individuals.

Scope: Neural Network Interpretability methods for image and tabular data

Dimension 1 — Passive vs. Active Approaches

Passive	Post hoc explain trained neural networks
Active	Actively change the network architecture or training process for better interpretability

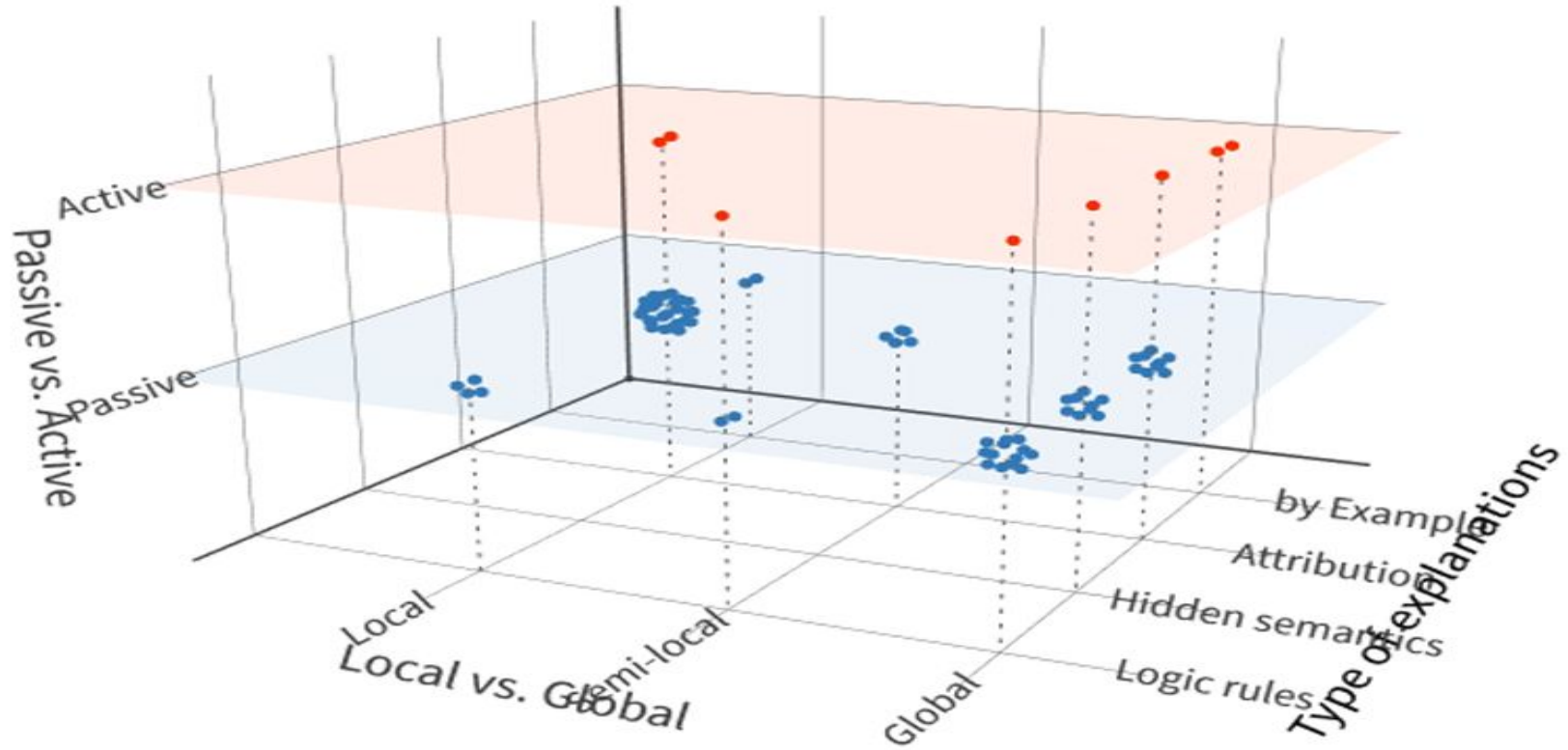
Dimension 2 — Type of Explanations (in the order of increasing explanatory power)

To explain a prediction/class by

Examples	Provide example(s) which may be considered similar or as prototype(s)
Attribution	Assign credit (or blame) to the input features (e.g. feature importance, saliency masks)
<u>Hidden semantics</u>	<u>Make sense of certain hidden neurons/layers</u>
Rules	Extract logic rules (e.g. decision trees, rule sets and other rule formats)

Dimension 3 — Local vs. Global Interpretability (in terms of the input space)

Local	Explain network's <i>predictions on individual samples</i> (e.g. <u>a saliency mask for an input image</u>)
Semi-local	In between, for example, explain a group of similar inputs together
Global	Explain the network <i>as a whole</i> (e.g. a set of rules/a decision tree)



01

Learned Features

What do neural networks actually learn to detect?

Feature Visualization

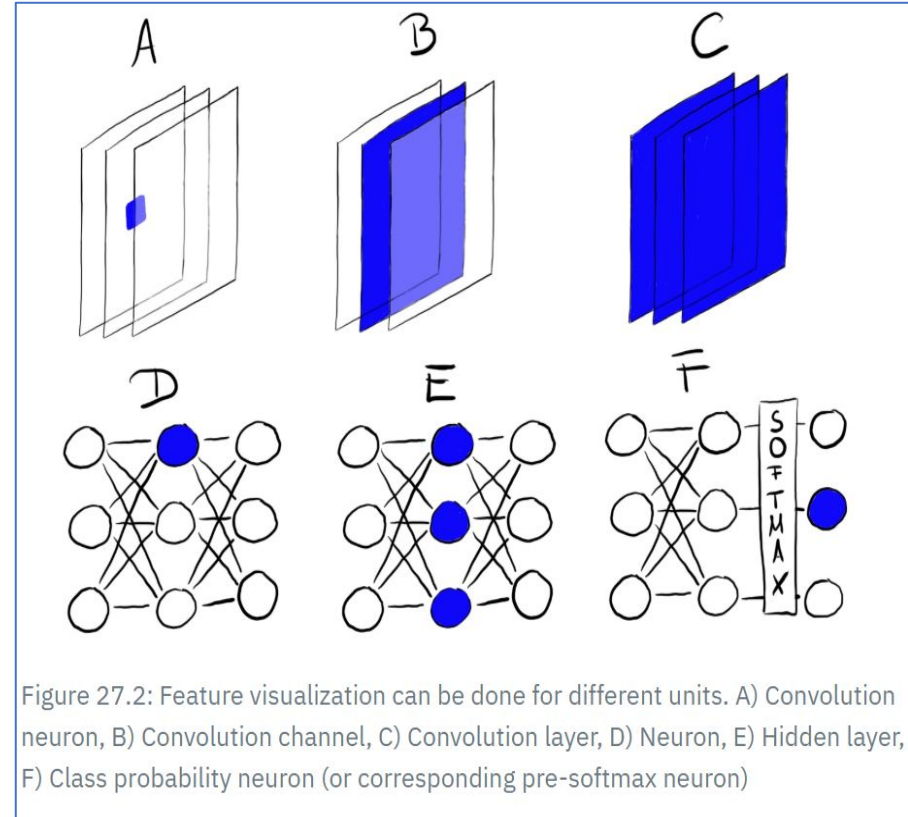
- The approach of making the learned features explicit is called **Feature Visualization**.
- Feature visualization for a unit of a neural network is done by finding the input that maximizes the activation of that unit.
- “Unit” refers either to individual neurons, channels (also called feature maps), entire layers, or the final class probability in classification (or the corresponding pre-softmax neuron, which is recommended)

1. **Individual neurons** are atomic units of the network, so we would get the most information by creating feature visualizations for each neuron. But there is a problem: Neural networks often contain millions of neurons. Looking at each neuron’s feature visualization would take too long.

2. **The channels (sometimes called activation maps)** as units are a good choice for feature visualization.

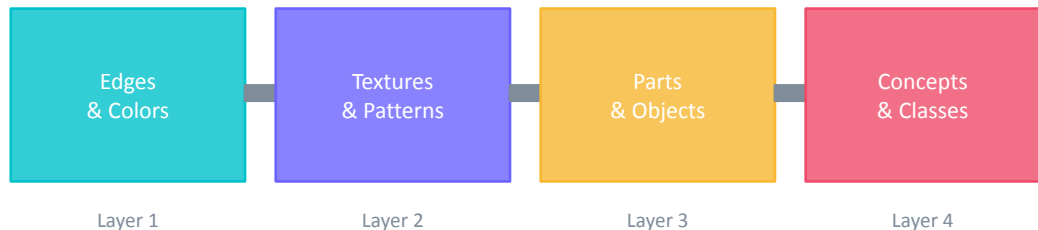
3. **visualize an entire convolutional layer.** Layers as a unit are used for **Google’s DeepDream**, which repeatedly adds the visualized features of a layer to the original image, resulting in a dream-like version of the input

pen → features



Feature Visualization

Feature visualization synthesizes inputs that maximally activate a neuron or channel using gradient ascent. Pioneered by Erhan et al. (2009) and extended by Olah et al. (2017) at Google Brain, this technique reveals that early layers detect edges and colors, mid layers detect textures and patterns, and deep layers detect object parts and semantic concepts.



Features learned by a convolutional neural network (Inception V1) trained on the ImageNet data

Key Techniques

Activation Maximization

Optimize input pixels via gradient ascent to maximally excite a neuron — reveals what the network 'looks for'.

Network Dissection

Align hidden units with human-interpretable concepts (Bau et al., 2017). Detectors for 'trees', 'faces' emerge spontaneously.

Multifaceted Feature Viz

Each neuron responds to multiple semantic concepts — diversity in activation space (Olah et al., 2020).

01 LEARNED FEATURES : Feature Visualization through Optimization

In mathematical terms, feature visualization is an optimization problem. We assume that the weights of the neural network are fixed, which means that the network is trained. We are looking for a new image that maximizes the (mean) activation of a unit, here a single neuron:

$$\mathbf{x}^* = \arg \max_{\mathbf{x}} h_{n,u,v,z}(\mathbf{x}) \rightarrow \text{per neuron}$$

The function h one particular the activation of a neuron, $\mathbf{x}$ the input of the network (an image), u and v describe the spatial position of the neuron, n specifies the layer, and z is the channel index. For the mean activation of an entire channel z in layer n , we maximize:

$n \rightarrow$ layers
 $u, v \rightarrow$ spatial position
 $z \rightarrow$ channel index

$$\mathbf{x}^* = \arg \max_{\mathbf{x}} \sum_{u,v} h_{n,u,v,z}(\mathbf{x})$$

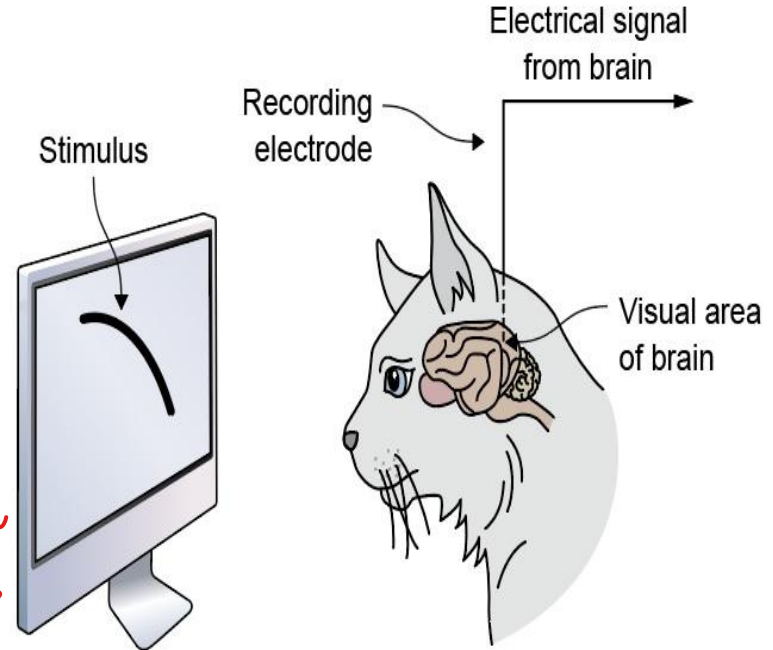
In this formula, all neurons in channel z are equally weighted.

Alternatively, you can also maximize random directions, which means that the neurons would be multiplied by different parameters, including negative directions. In this way, we study how the neurons interact within the channel. Instead of maximizing the activation, you can also minimize it (which corresponds to maximizing the negative direction).

01 LEARNED FEATURES : Feature Visualization through Activation Maximization

- Activation maximization is a **visualization technique** used to understand what features a neural network has learned, by creating an image that causes a specific neuron or class to fire strongly.
- Offers a direct means to probe what a neuron or channel has learned by synthesizing inputs that maximize its activation
- It works by starting with a random input image and using gradient ascent to modify pixel values to maximize the target neuron's activation
- These optimized inputs, often referred to as Activation Maximization Signals (AMS), aim to reveal the underlying features to which neurons are most responsive.
- A simple strategy for estimating neuron preferences involves identifying real input samples from the dataset that produce high activations. However, this approach is limited by data coverage and scalability.
- Figure shows In the classic neuroscience experiment, Hubel, Wiesel et al. (1959) discovered a cat's visual cortex neuron that fires strongly and selectively for a light bar when it is in certain positions and orientations.

How to achieve this



How Activation Maximization Works

Algorithm: Gradient Ascent on Input

1. Initialize: start with a random or blank image x_0 ✓
2. Forward pass: compute neuron activation $a_{\square}(x)$ ✓
3. Compute gradient: $\partial a_{\square} / \partial x$ — how each pixel affects activation ✓
4. Update: $x \leftarrow x + \alpha \cdot \nabla_x a_{\square}(x)$ [gradient ascent] ✓
5. Apply regularization (L2 decay, Gaussian blur, jitter) ✓
6. Repeat until convergence $\rightarrow$ visualize x^* ✓

Why Regularization Matters

Without constraints, gradient ascent produces adversarial-like noise. Regularization (total variation, L2, frequency penalization) forces the synthesized image toward naturalistic patterns — making it human-interpretable.

Simonyan et al. (2014) · Mordvintsev et al. — DeepDream (2015) · Mahendran & Vedaldi (2015) CVPR

Applications

Dream / DeepDream

Google's viral visualization — amplifying patterns recognized in an image at each layer.

Neuron Debugging

Identify dead neurons (no activation) or neurons encoding unwanted features.

Class Visualization

Generate the 'ideal' image for a class: what does the model think a 'cat' looks like?

Layer Comparison

Compare early vs. late layers — transitions from Gabor-like filters to complex part detectors.

Key Aspects of Network Dissection:

→ align hidden units with human interpretable concepts

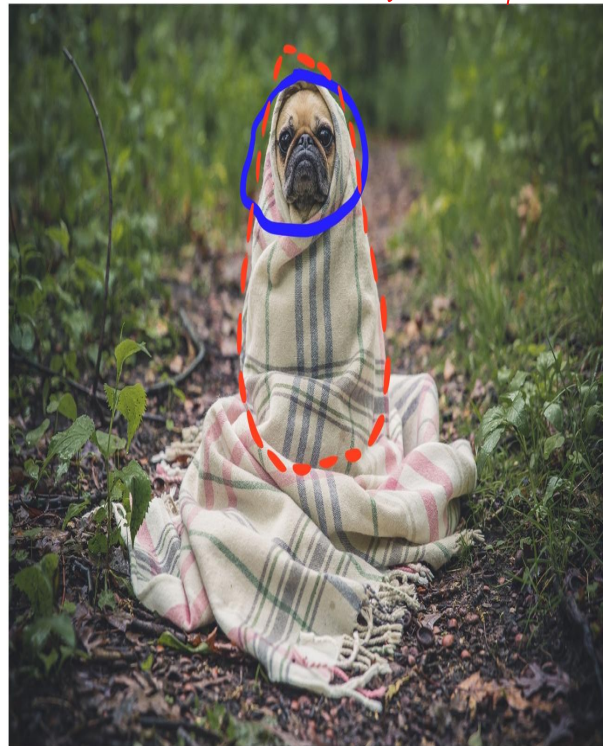
- The Network Dissection method links human concepts with individual neural network units in Deep CNN.
- Alert: Network Dissection requires extra datasets that someone has labeled with human concepts.
- The Network Dissection approach by Bau et al. (2017) quantifies the interpretability of a unit of a convolutional neural network. It links highly activated areas of CNN channels with human concepts (objects, parts, textures, colors, ...).
- Big hypothesis behind : The hypothesis is: **"Units of a neural network (like convolutional channels) learn disentangled concepts."**

- **Methodology:** It tests individual convolutional units as detectors for semantic concepts using **Intersection over Union (IoU)** scores.
- **Concepts Identified:** Units are labeled based on their highest-scoring concept: colors, textures, materials, parts, objects, and scenes. *Imp result*
- **Interpretation of Hidden Layers:** It reveals that while networks are often considered "black boxes," specific neurons can emerge as specialized detectors for specific visual concepts (e.g., a "dog ear" detector) during training, even without explicit supervision.
- **Applications:** It is used to compare different CNN architectures, analyze the impact of training (e.g., depth, width, regularizations), and provide explanations for model predictions.
- **Key Findings:** Interpretability is not uniform across all neurons, and it is higher than a random basis, suggesting that neural networks tend to decompose complex input into human-understandable semantic concepts.

- Disentangled features mean that individual network units **detect specific real-world concepts**.
- Convolutional channel 394 might detect skyscrapers, channel 121 dog snouts, channel 12 stripes at a 30-degree angle, ... The opposite of a disentangled network is a completely entangled network.
- In a completely entangled network, for example, there would be no individual unit for dog snouts. All channels would contribute to the recognition of dog snouts.
- Disentangled features imply that the network is highly interpretable

disentangled

→ have units for specific features



○ = Human annotated ground truth

○ = Top activated area

○ = Area of Intersection

○ = Area of Union

The Intersection over Union (IoU) is computed by comparing the human ground truth annotation and the top activated pixels.

01 LEARNED FEATURES : Network Dissection

Network Dissection quantifies interpretability by measuring alignment between individual hidden units and human-defined visual concepts. It uses the Broden dataset — a broad, densely annotated segmentation dataset covering scenes, objects, parts, textures, and colors.



Key Findings

- GoogLeNet trained on Places205 had 60% of conv5 units aligned to human concepts
- Texture units dominate early layers; object-part units dominate deeper layers
- Randomizing labels degrades interpretability — concepts are learned, not arbitrary
- GAN generators also develop interpretable units (Bau et al., 2019)

Limitations

- Polysemantic neurons: many units respond to multiple unrelated concepts
- IoU threshold is arbitrary — concept alignment is not binary
- Dataset coverage limits: Broden doesn't cover all possible concepts
- Units are not independent — features are distributed across many neurons

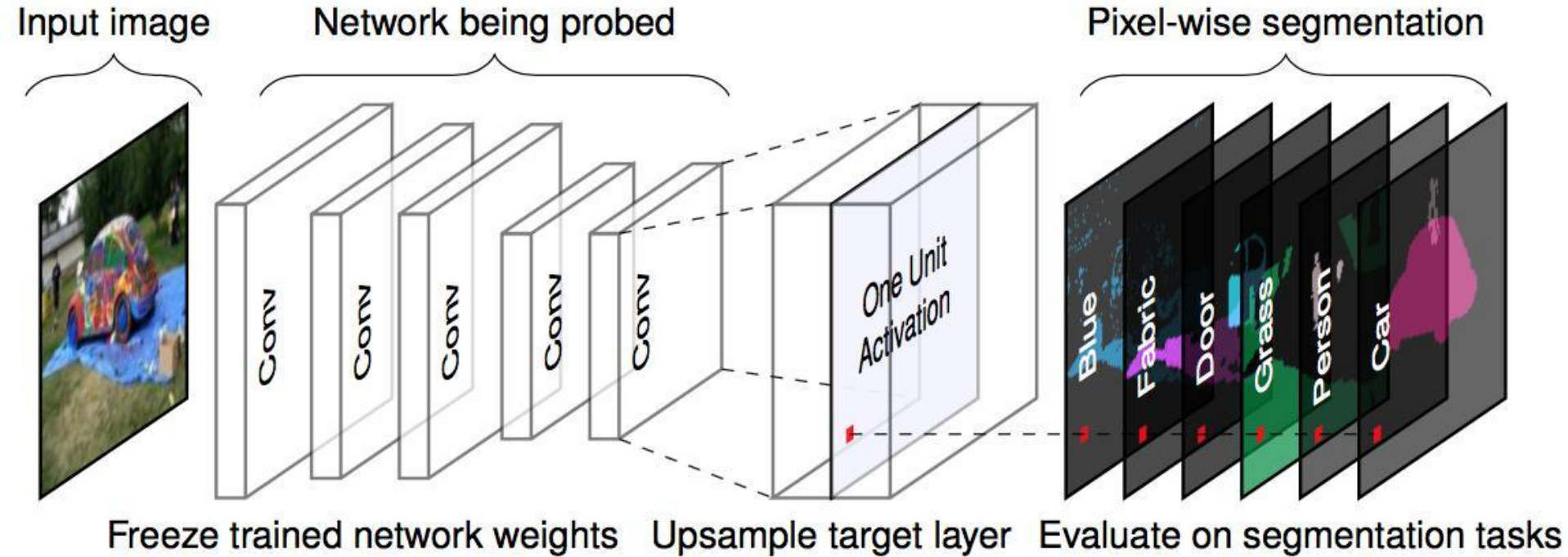


Fig : For a given input image and a trained network (fixed weights), we propagate the image forward to the target layer, upscale the activations to match the original image size, and compare the maximum activations with the ground truth pixel-wise segmentation. Figure originally from <http://netdissect.csail.mit.edu/>

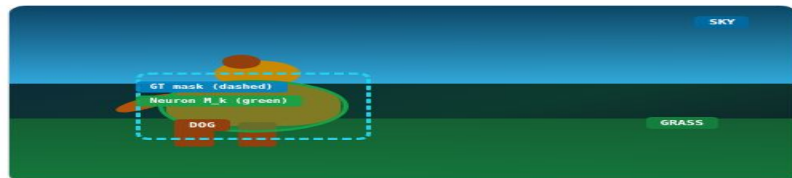
Network Dissection

Mapping hidden neuron units – human-interpretable semantic concepts via Intersection over Union (IoU)

STEP-BY-STEP PIPELINE



WORKED EXAMPLE – DOG SCENE (CONV5 UNIT_34)



- Ground truth annotation (Broden dataset)
- Neuron k activation mask (thresholded at top 1%)
- overlap region = Intersection ($\cap$)

INTERSECTION OVER UNION – IOU CALCULATION



$$IoU = \frac{|GT \cap M_k|}{|GT \cup M_k|} = \text{Overlap} \div \text{Total Area}$$

For unit_34 (Dog Detector): IoU = 0.72

GT pixels: 2,400 M_k pixels: 2,100 $\cap$ pixels: 1,728
– IoU = 1728 $\div$ (2400+2100-1728) = 1728 $\div$ 2772 = 0.72 ✓

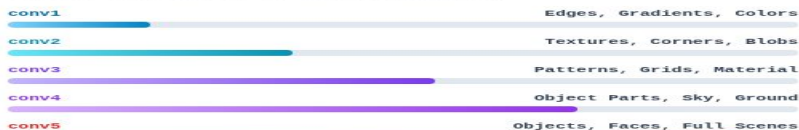
Semantic Alignment Rule

$IoU \geq 0.04$ – Unit is a concept detector for that label
Higher IoU = stronger & more precise semantic alignment

DISSECTED UNITS – PLACES365-ALEXNET CONV5 LAYER EXAMPLES



CONCEPT COMPLEXITY BY LAYER (ALEXNET)



Bar width = relative semantic complexity of detected concepts

KEY INSIGHTS FROM NETWORK DISSECTION

- Emergent concept detectors:** CNN units spontaneously learn to respond to semantically meaningful regions – without any explicit supervision on concept labels during training.
- IoU as alignment metric:** Spatial overlap between neuron binary mask M_k and Broden ground-truth mask measures how well a unit detects a concept (textures, objects, scenes, parts).
- Depth = abstraction:** Deeper layers detect progressively more complex concepts – conv1 detects edges, conv5 detects entire objects & scenes.
- Threshold rule $IoU \geq 0.04$:** A unit is labelled a concept detector above this threshold. unit_221 (Sky, IoU=0.83) is a near-perfect detector example.

Olah et al. (2020, Distill) proposed that features are not stored in individual neurons but in circuits — small subgraphs of neurons and weights that implement specific algorithms. This reframes XAI from 'what does this neuron detect?' to 'what computation does this circuit perform?'

Circuits Hypothesis

- ▶ Features are universally shared across models trained on similar data
- ▶ Curve detectors → curve assemblies → object parts — hierarchical compositionality
- ▶ High-low frequency detectors help detect image borders and textures
- ▶ Multimodal neurons (e.g. 'Spider-Man' in CLIP) respond to concept across modalities
- ▶ Circuits can be reverse-engineered like logical circuits in hardware

Polysemanticity Problem

- ▶ Single neurons often respond to multiple unrelated concepts (e.g., cats AND cars)
- ▶ Caused by superposition: models store more features than they have neurons
- ▶ Sparse, high-dimensional activations allow concepts to coexist without interference
- ▶ Anthropic's Toy Models of Superposition (2022) formalized the phenomenon
- ▶ Monosemanticity is the goal — each neuron for one concept — rarely achieved

Probing Classifiers & Representational Similarity

Linear Probing

Train a simple logistic regression on frozen activations of layer l to predict label y . If linear probe succeeds $\rightarrow$ information is linearly decodable at that layer. Widely used to benchmark representation quality in self-supervised learning (SimCLR, DINO).

Centered Kernel Alignment (CKA)

Measures similarity between activation matrices of two different layers or models. Scale-invariant and permutation-invariant. Reveals which layers are 'doing the same thing' across architectures — foundational for transfer learning analysis.

Imp

t-SNE / UMAP Embedding

Project high-dimensional activation spaces into 2D for visualization. Reveal clustering structure — classes that are close in activation space are 'confused' by the model. Used diagnostically to catch label leakage and class imbalance effects.

Representational Dissimilarity Matrices (RDMs)

From neuroscience — compare pairwise distances between all stimulus representations. Can compare neural networks to actual brain activity (EEG/fMRI), enabling cross-disciplinary XAI research.

Imp

Raw gradient ascent produces high-frequency noise rather than recognizable images. Regularization imposes prior beliefs about what natural images look like, constraining the optimization to produce more interpretable results. This is the core engineering challenge in feature visualization.

Regularization techniques

L2 Weight Decay

Penalize: $\lambda ||x||^2$

Prevents extreme pixel values. Produces desaturated, low-contrast images. Simple but insufficient alone.

Gaussian Blur

Apply: $G_{\sigma} * x$

Smooths high-frequency artifacts. Applied periodically during optimization. Blurs fine detail — trade-off.

Jitter & Padding

$x \leftarrow \text{roll}(x, \delta)$

Randomly shifts image each step. Prevents optimization from exploiting fixed-pixel artifacts. Olah et al. key trick.

Total Variation

Penalize: $TV(x) = \sum |\nabla x|$

Encourages piecewise-smooth images with sharp edges. Produces cartoon-like but interpretable patterns.

Frequency Penalization

Penalize high-freq modes

Work in Fourier space; down-weight high frequencies. Produces the most naturalistic visualizations (Olah 2017).



Image Parametrization

$x = D(z)$, optimize z

Parametrize images as outputs of a decoder (VAE/GAN). The decoder's implicit prior forces natural images. Strongest results.

What We Know — and What Remains Unsolved

Established Knowledge

- ✓ Networks develop hierarchical representations from edges → textures → parts → objects
- ✓ Many features are universal across different architectures and training datasets
- ✓ Network Dissection shows ~60% of deep units align with human-interpretable concepts
- ✓ Circuit-level analysis can reverse-engineer specific computational algorithms

Open Challenges

- ? Polysemanticity: most neurons respond to multiple unrelated concepts — distributed representations
- ? Scale problem: GPT-4/large models have billions of parameters — circuit analysis doesn't yet scale
- ? Causality gap: feature visualization shows correlation with activations, not causal mechanisms
- ? Evaluation: no ground truth for 'correct' visualization — quality is subjective

Future Direction

Mechanistic interpretability (Anthropic, DeepMind, MIT) — systematically reverse-engineer the algorithms implemented by neural networks, treating them as software to be decompiled. Recent work on sparse autoencoders (Bricken et al., 2023) decomposes MLP activations into interpretable monosemantic features, offering a path around polysemanticity.

Hands on

02

Pixel Attribution

Which pixels drove the model's prediction? ✓?



Pixel attribution assigns importance scores to each input pixel for a given prediction. Methods range from gradient-based approaches to propagation and perturbation-based techniques. Unlike global feature visualization, these are local explanations — specific to one prediction on one image.

Vanilla Gradients

2013 · Simonyan et al.

$\partial \text{output} / \partial \text{input}$ — gradient of the loss w.r.t. the input. Fast but noisy.

Integrated Gradients

2017 · Sundararajan et al.

Averages gradients along path from baseline to input. Satisfies completeness.

GradCAM

2017 · Selvaraju et al.

Gradients into last conv layer → class-discriminative localization maps.

LIME

2016 · Ribeiro et al.

Perturbs superpixels → local linear surrogate → influential regions.

SHAP (DeepSHAP)

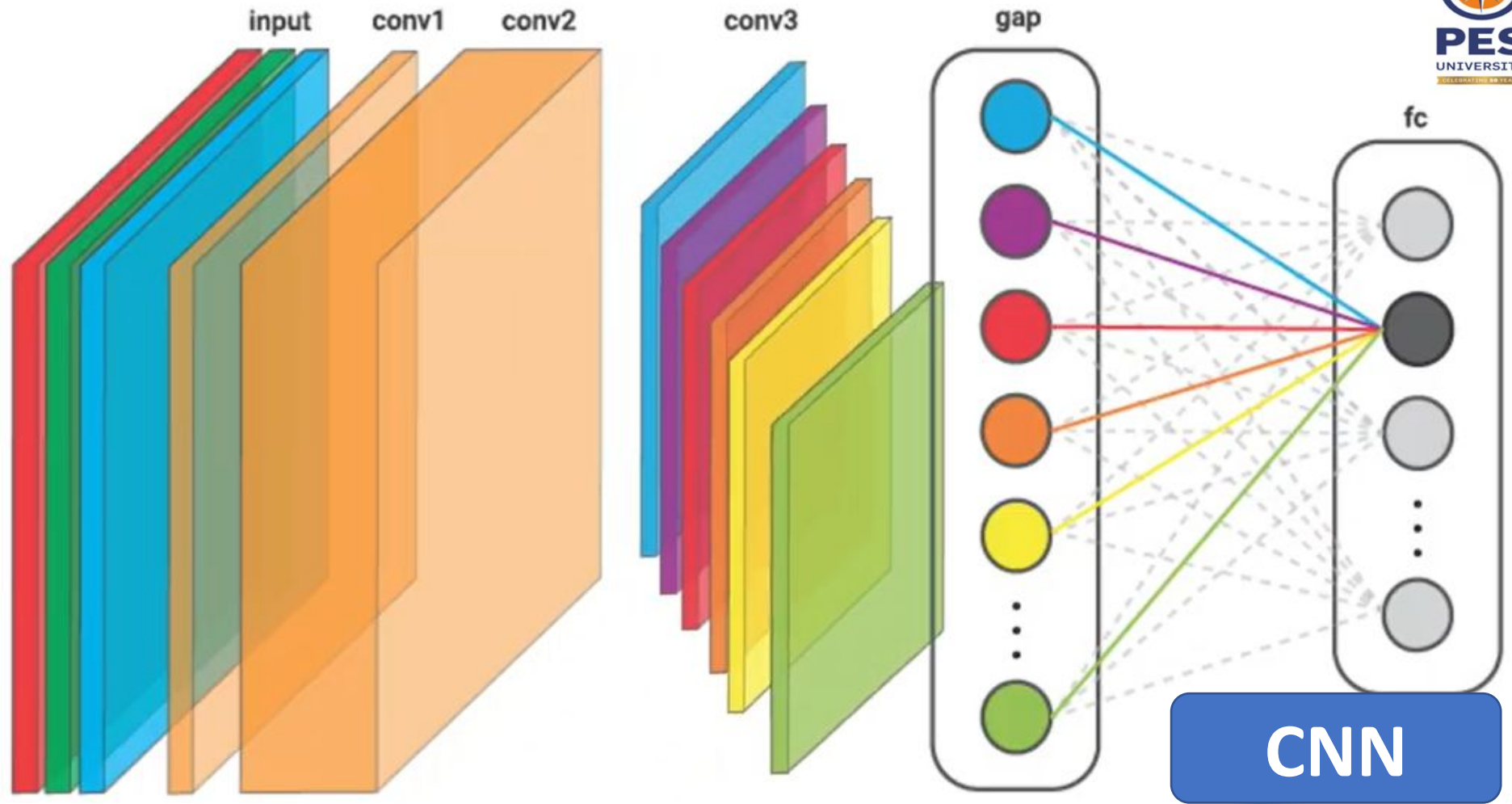
2017 · Lundberg & Lee

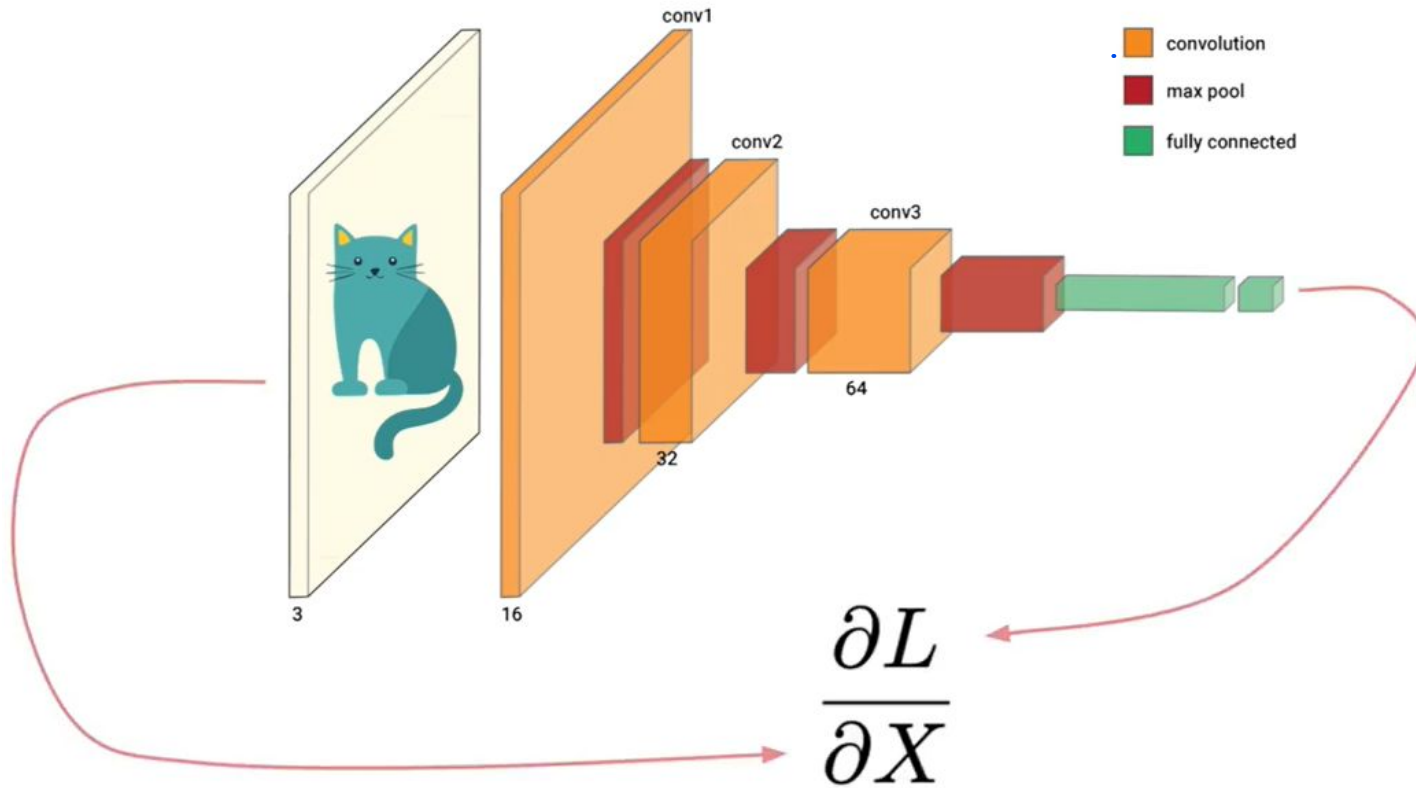
Shapley values: fair distribution of prediction credit across pixels.

SmoothGrad

2017 · Smilkov et al.

Adds noise, averages maps → reduces gradient noise for cleaner saliency.



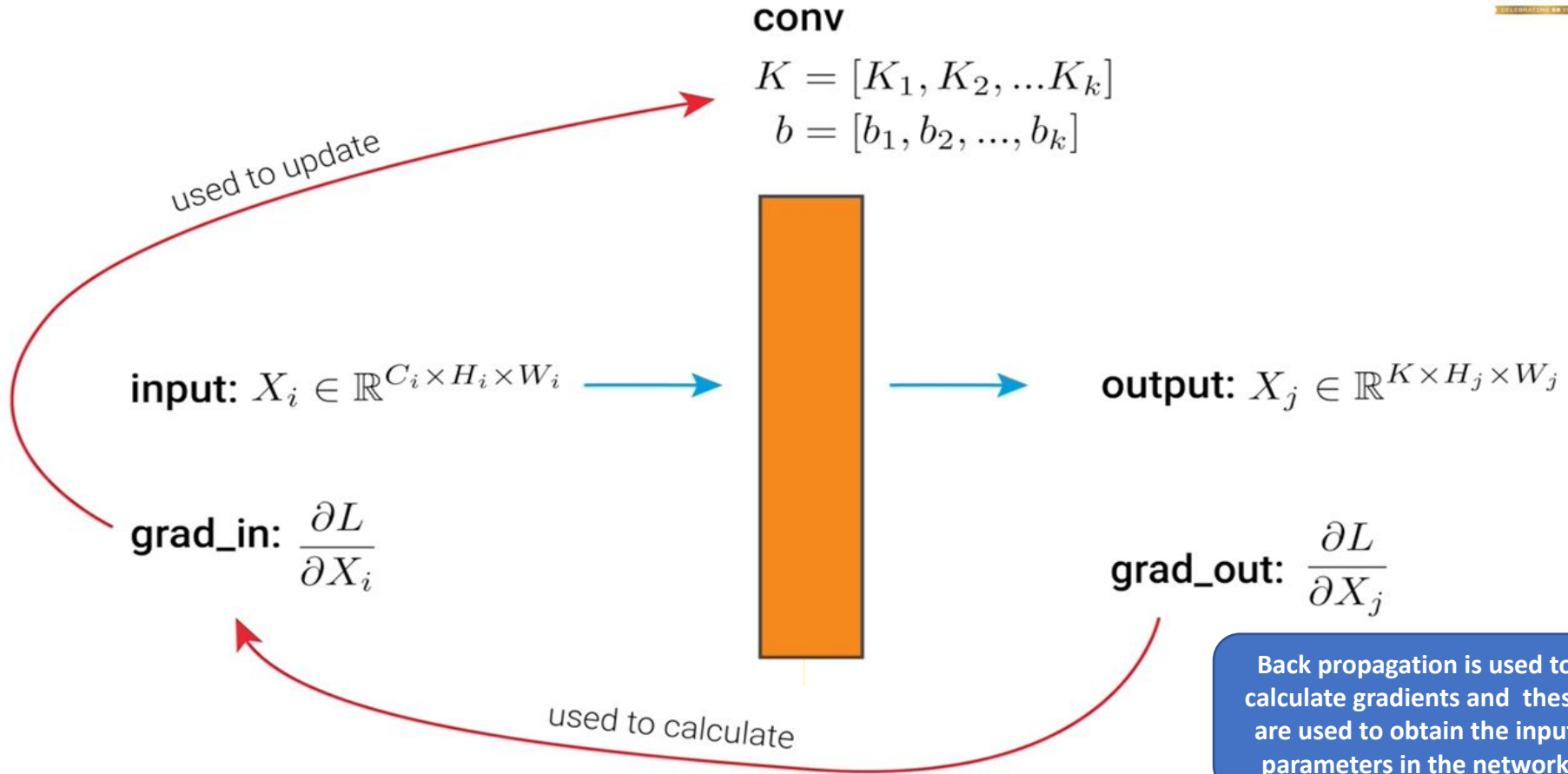


VANILLA GRAD

Vanilla gradients are the gradients of the output with respect to the input image

For every input image, find the gradients of the loss with respect to the pixels in the input image

Vanilla Gradient

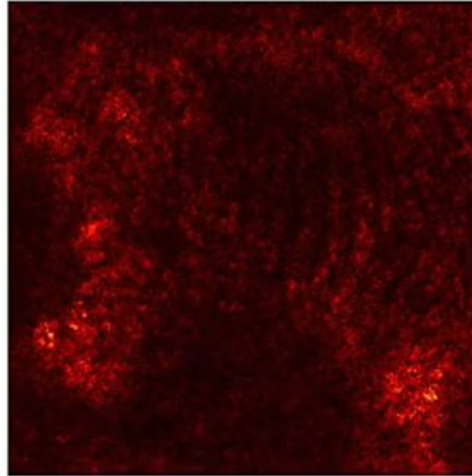


Limitations of Vanilla Gradient

Input Image



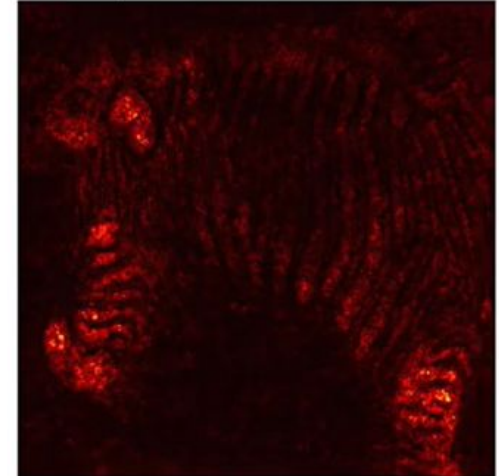
Gradients



X

=

Input x Gradients



To avoid noisy saliency maps which makes it difficult to understand what features the model is using to make predictions. One option is to multiply the gradients with the inputs to get visually sharper saliency maps.

Limitations of Vanilla Gradient

Noisy gradients

Extremely sensitive to small input changes

Irrelevant information

Highlight regions that suppress the target prediction

Saturated gradients

Important pixels can have near-zero gradients

Vanishing gradients

Gradients can diminish as they propagate to earlier layers

Lack of context awareness

Only measure local sensitivity of individual pixels

Hands on Vanilla Gradient

Why this code satisfies the "Vanilla Gradient" requirement:

1. **A Specific Task:** The model is tasked with learning a linear relationship ($Y = 3X$).
2. **Calculating Gradients:** I used `tape.gradient(loss, [W, b])`. This is the core "Vanilla" step. It calculates the slope of the error curve.
3. **Backpropagation:** The `GradientTape` tracks the operations during the forward pass and "backpropagates" the error through the math chain when `tape.gradient` is called.
4. **Weight Update:** Instead of using a high-level tool like `model.fit()`, I manually subtracted the gradient from the weights: `W.assign_sub(learning_rate * grad_W)`. This is the mathematical definition of **Vanilla Gradient Descent**.

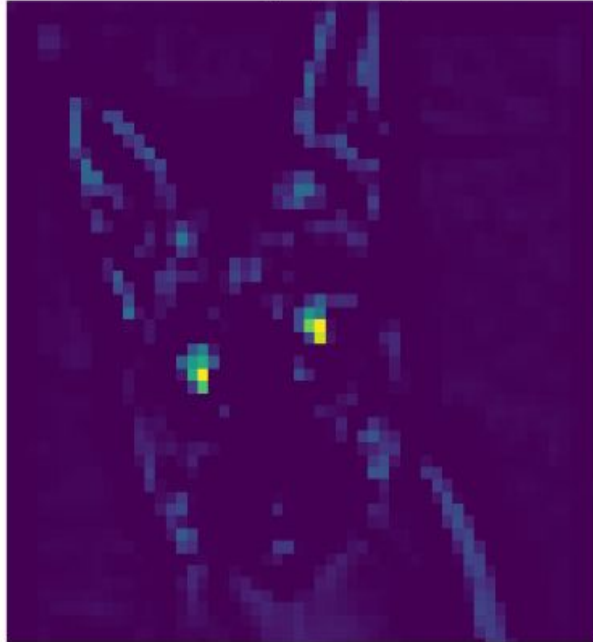
Out come..... Vanilla Gradient

Interpretation: Notice how the 'Early' image looks like a sharp outline (edges), while the 'Deep' image is more abstract and blobby (high-level parts).

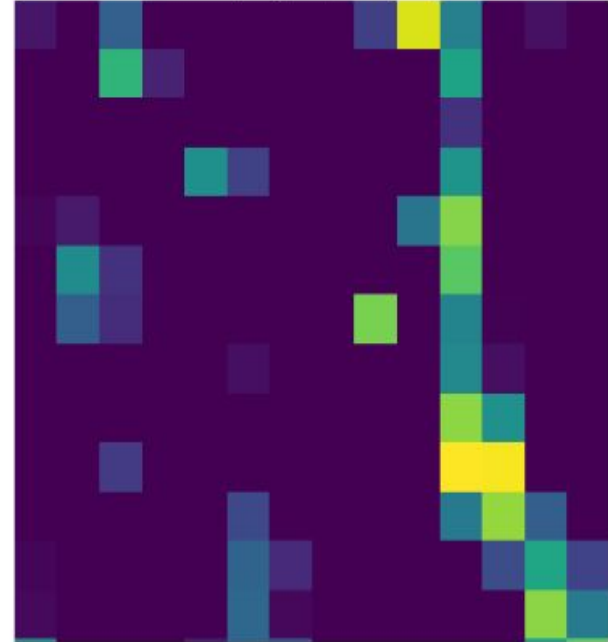
Early Layer (Edges)



Mid Layer (Textures)



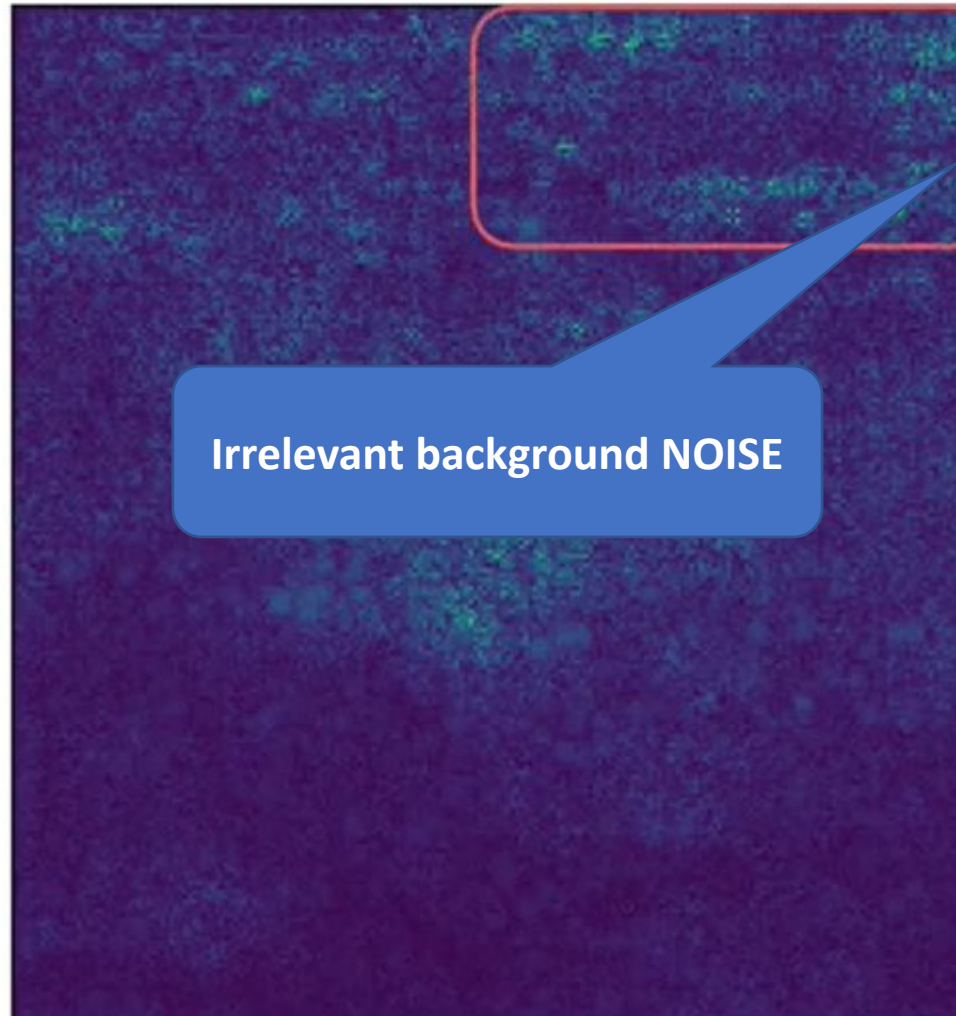
Deep Layer (Concepts)



Input Image



Smooth Gradients



Irrelevant background NOISE



SMOOTH GRAD

First create multiple
augmentations of the same
image by adding random noise

SMOOTH GRAD

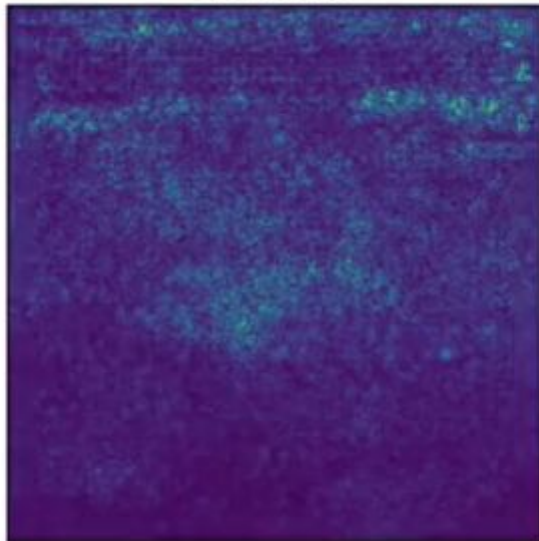
Then add explanation methods to get a saliency map for these augmented images. Then take an average of the saliency maps to get the final saliency map

$$\hat{M}_c(X) = \frac{1}{n} \sum_n M_c(X + \mathcal{N}(0, \sigma^2))$$

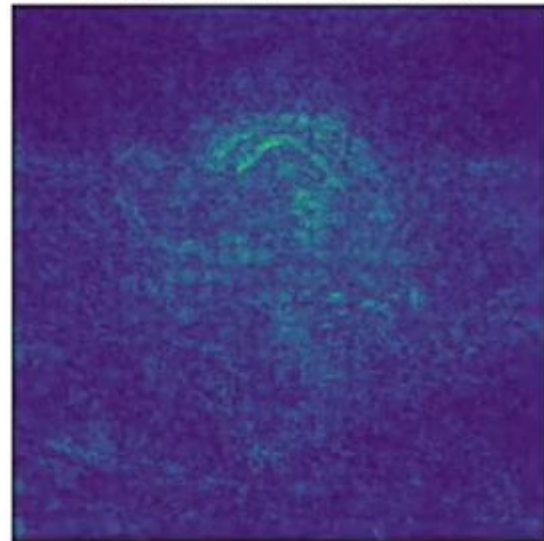
Input Image



Gradients



SmoothGrad + Gradients



Hands on Smooth Gradient

What you will see in the results:

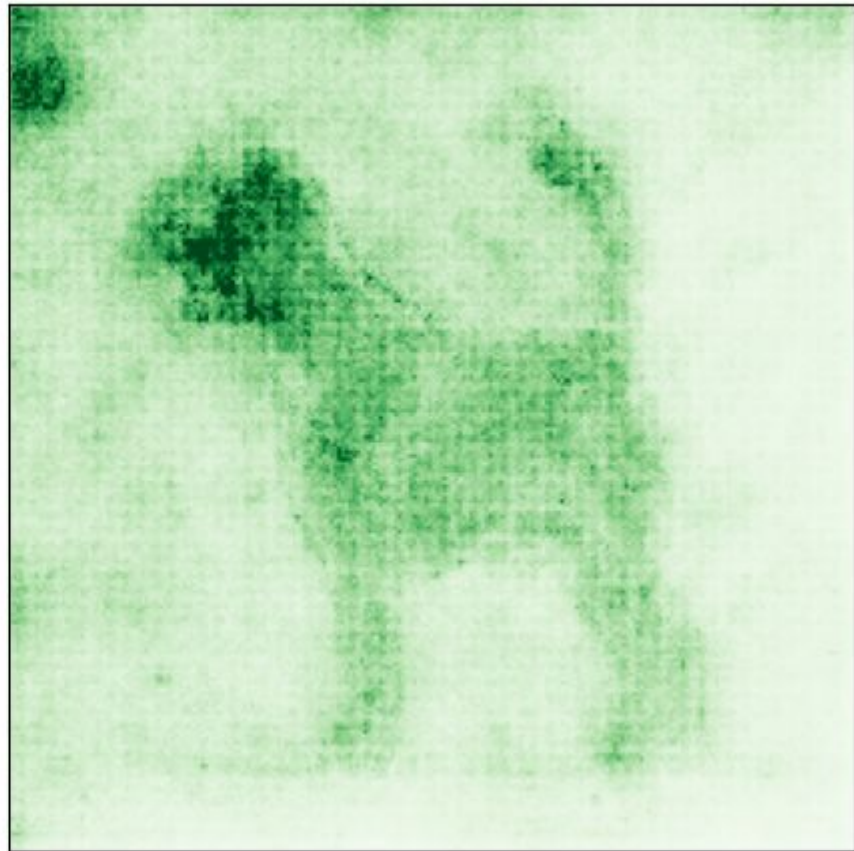
- **For the Dog:** The heatmap will likely light up around the **ears, nose, and tail shape**. These are the "discriminative" pixels that differentiate a dog from other dog breeds.
- **For the Cat:** You will notice focus on the **eyes and whiskers**. Because SmoothGrad reduces noise, you will see sharp lines around the facial features rather than random dots across the white background.
- **For the Apple:** The model will focus on the **stem and the specific curvature** of the fruit.
- **For the Mangoes:** The attribution should highlight the **yellow-to-orange gradient and the oval silhouette**.

Out Come Smooth Gradient

Input Image: dog.webp



SmoothGrad Explanation (Class: 167)

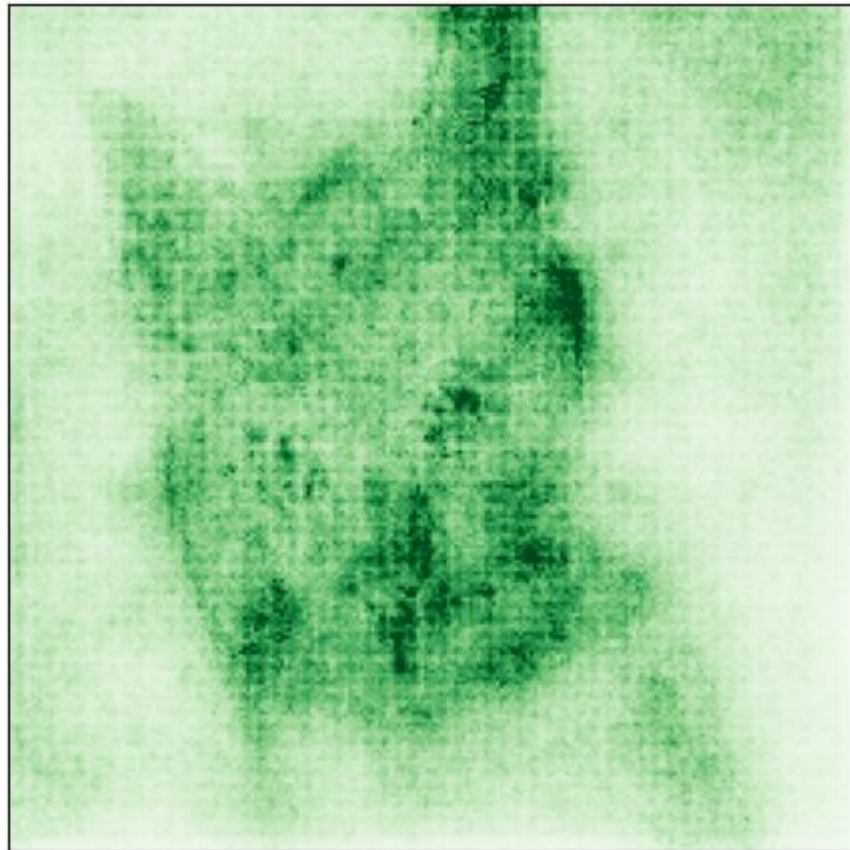


Out Come Smooth Gradient

Input Image: cat.jpg



SmoothGrad Explanation (Class: 285)

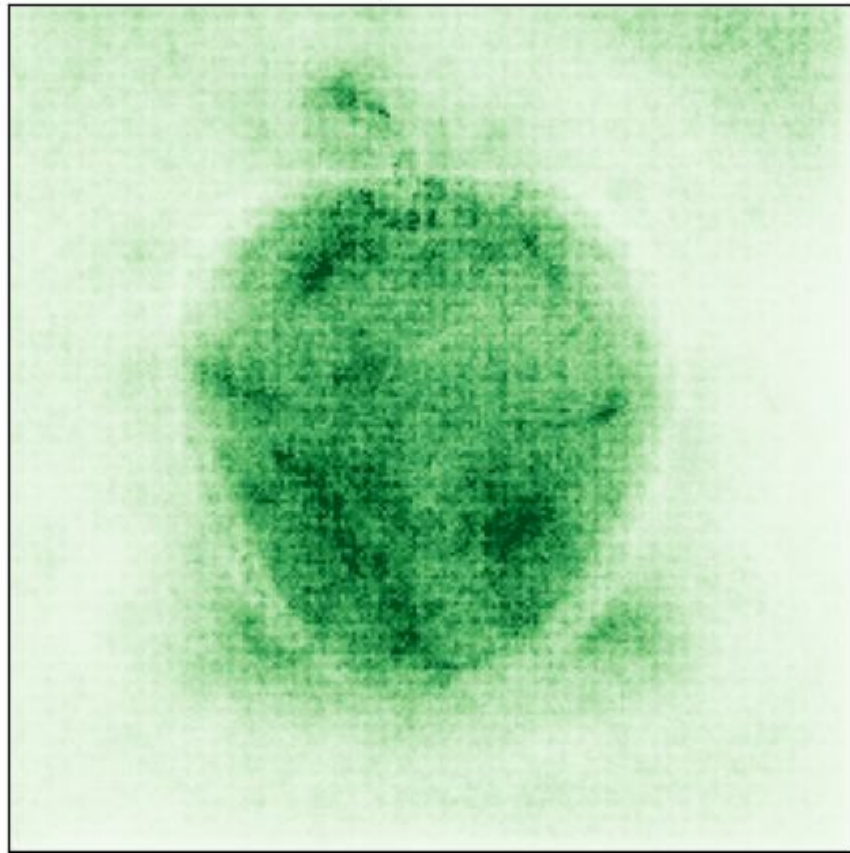


Out Come Smooth Gradient

Input Image: apple.avif



SmoothGrad Explanation (Class: 948)

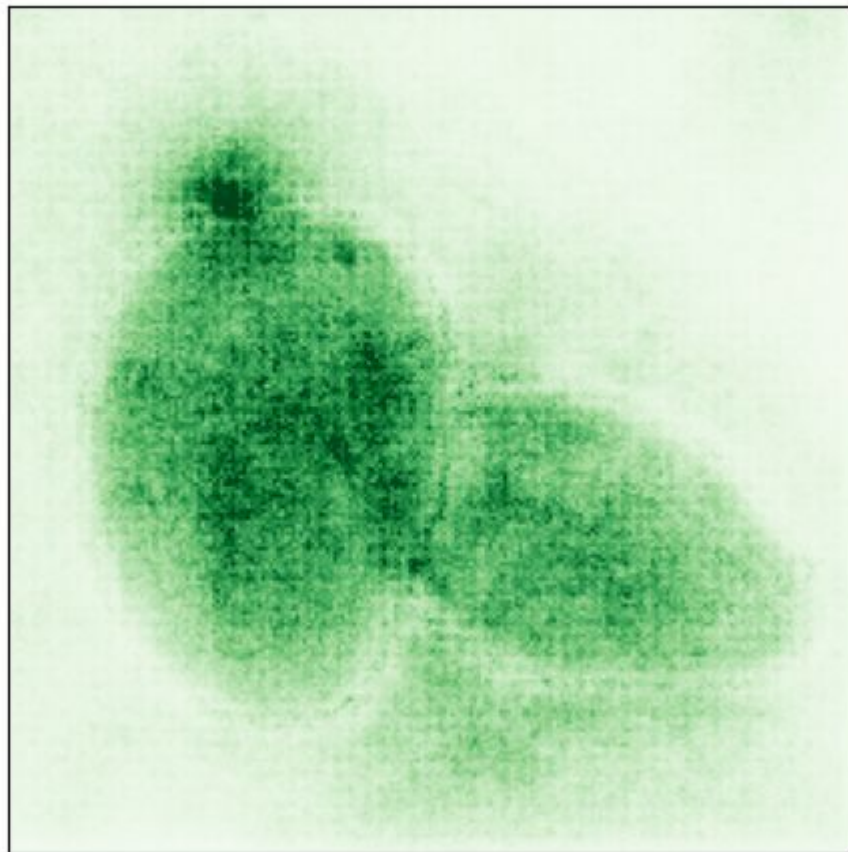


Out Come Smooth Gradient

Input Image: mango.png



SmoothGrad Explanation (Class: 940)



Integrated Gradient

Step 1



Step 2



Step 3



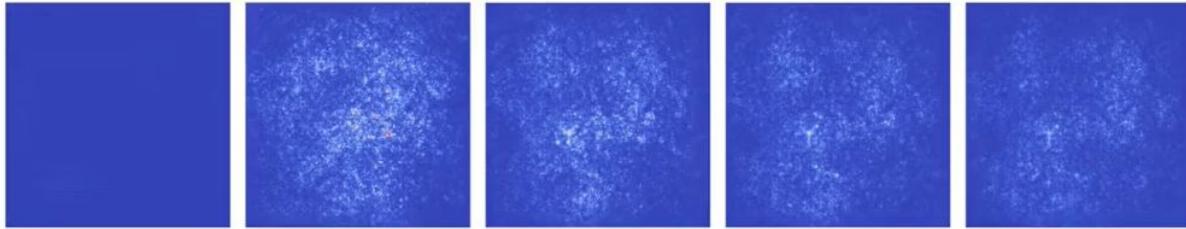
Step 4



Step 5

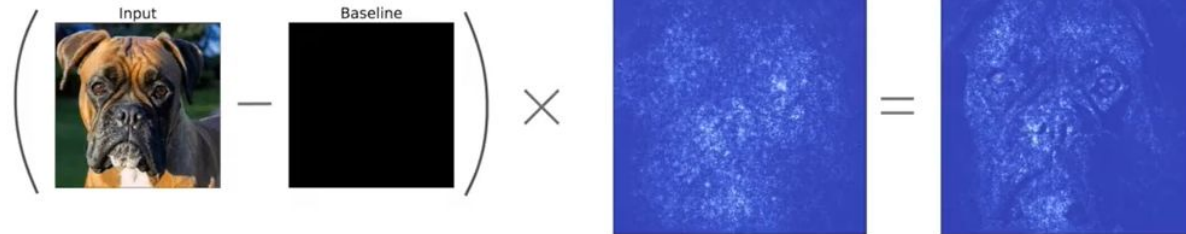


$$\frac{\partial y_c}{\partial X}$$



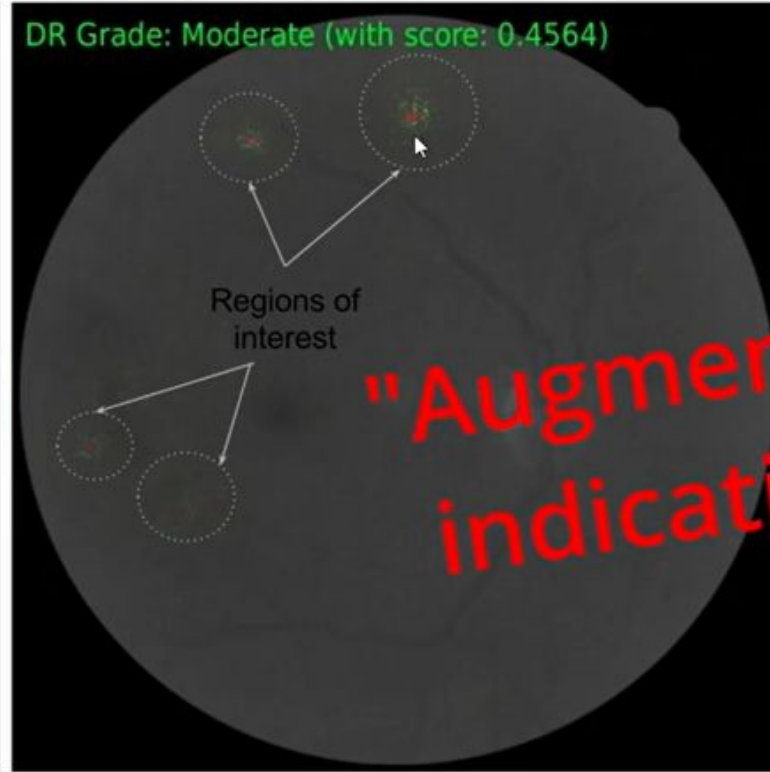
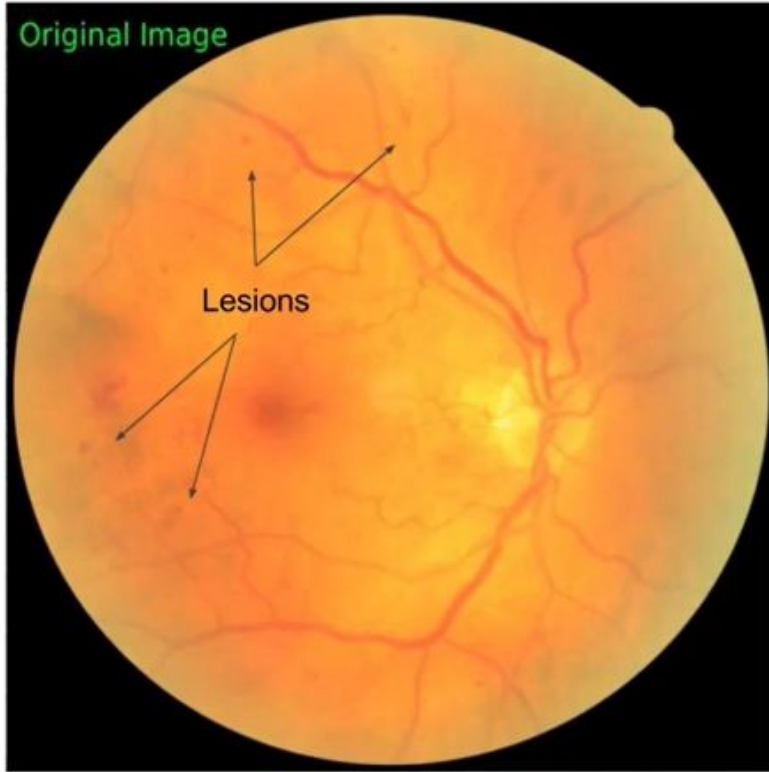
Average

Integrated Gradients

$$\left(\begin{array}{c} \text{Input} \\ \text{Baseline} \end{array} \right) \times \text{Average} = \text{Integrated Gradients}$$


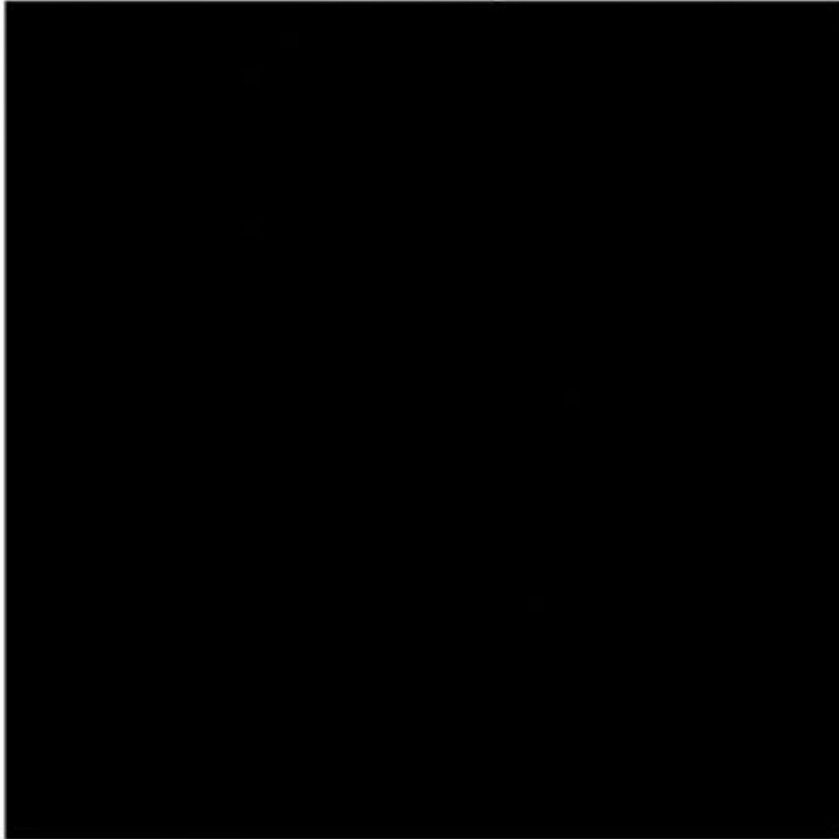
This method considers the gradients of many images along a path from the baseline to the input image

Integrated Gradient



Integrated Gradient

Baseline image

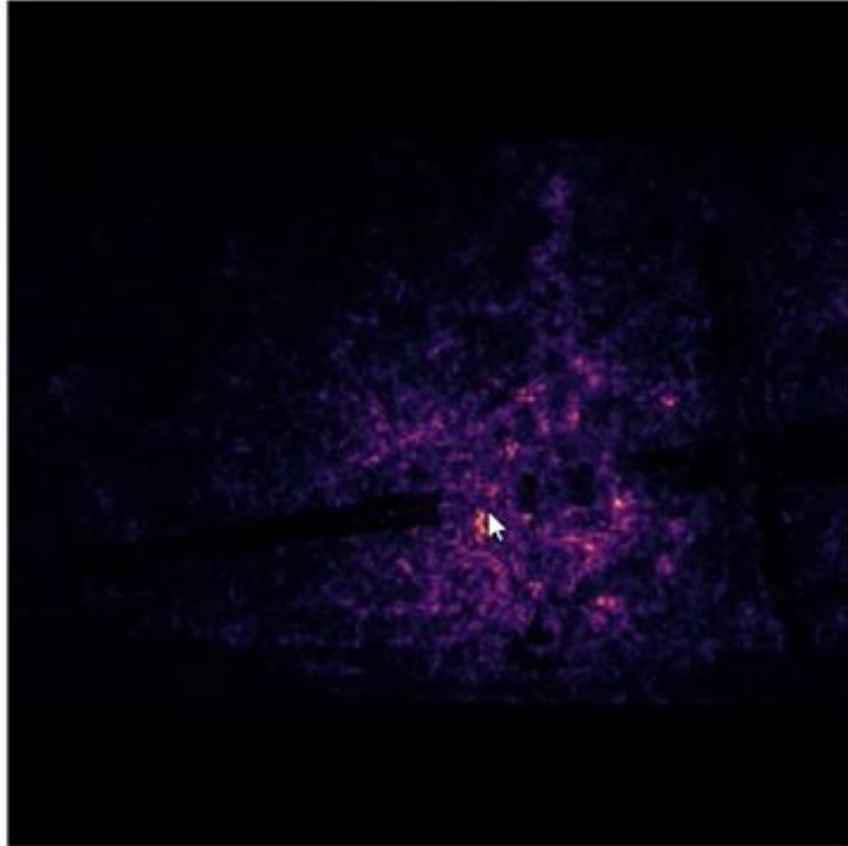


Original image



Integrated Gradient

Attribution mask



Overlay



Hands on Integrated Gradient

How this code demonstrates Integrated Gradients:

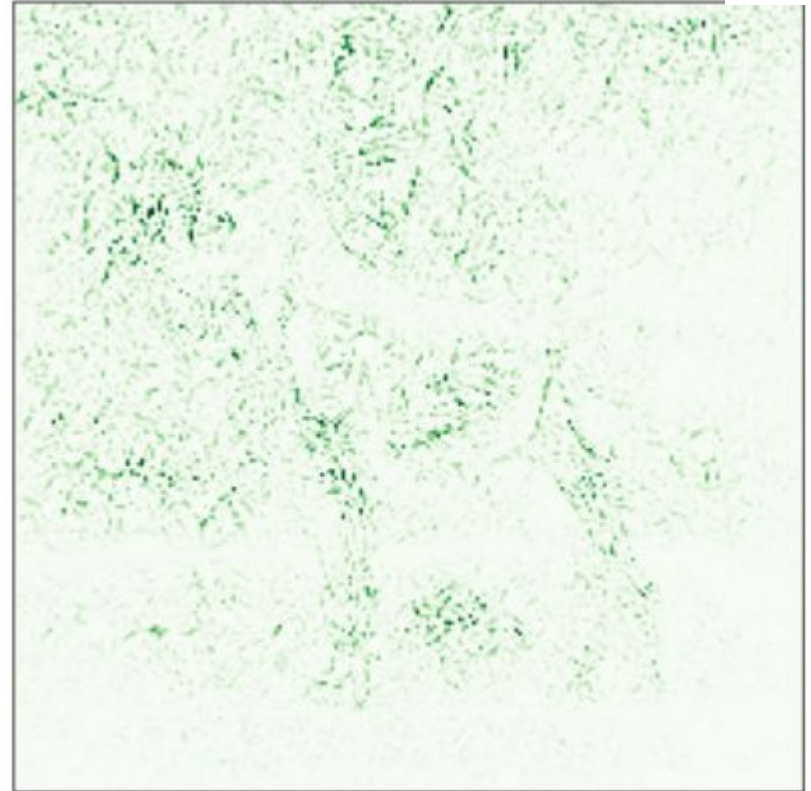
1. **The Baseline (x'):** Per Slide 4, we use a **Black Image (all zeros)** as the baseline. This represents "no information." IG calculates how much "credit" each pixel gets as we move from a black image to the actual image.
2. **The Path Integral:** Instead of looking at just the local gradient (like Vanilla Gradients), IG calculates gradients at multiple points (steps) along the path from the baseline to the input. We use `n_steps=50` to approximate this integral using **Riemann Summation**.
3. **Axiom of Completeness:** The code calculates the **Convergence Delta**. This value proves that the sum of all pixel attributions equals the difference between the model's prediction for the image and its prediction for the black baseline.
4. **Sensitivity:** By integrating along the path, IG ensures that if a pixel is important, it *will* receive attribution, avoiding the "gradient saturation" problem discussed on Slide 3.

Out Come Integrated Gradient

Input: dog.webp



IG Attribution (Target Class: 167)



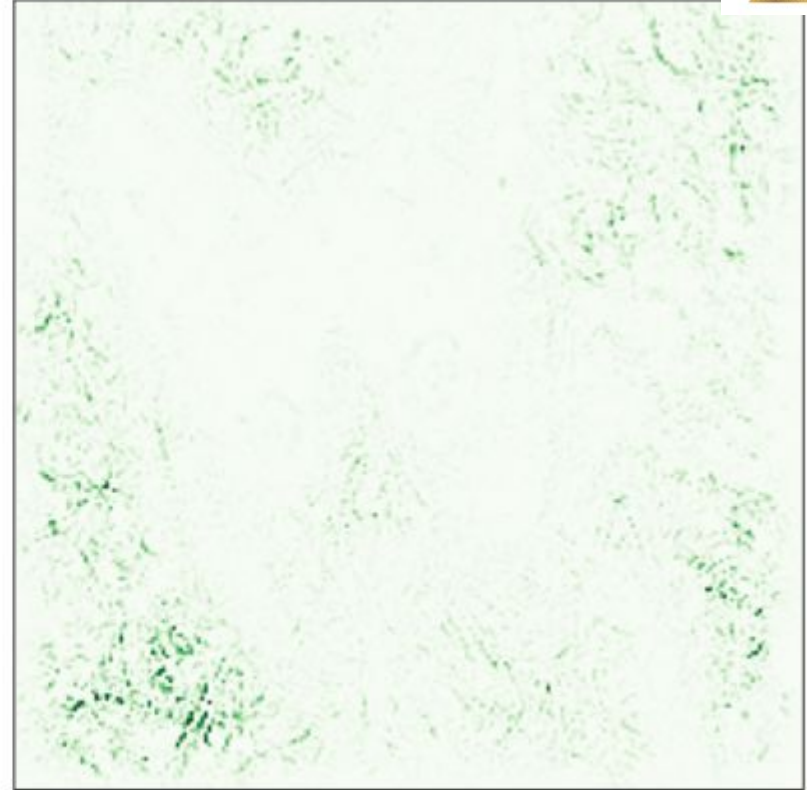
- ✓ Analysis for dog.webp:
- Convergence Delta: 2.469206
 - Result: Delta is near 0, justifying the 'Axiom of Completeness'.

Out Come Integrated Gradient

Input: cat.jpg



IG Attribution (Target Class: 285)



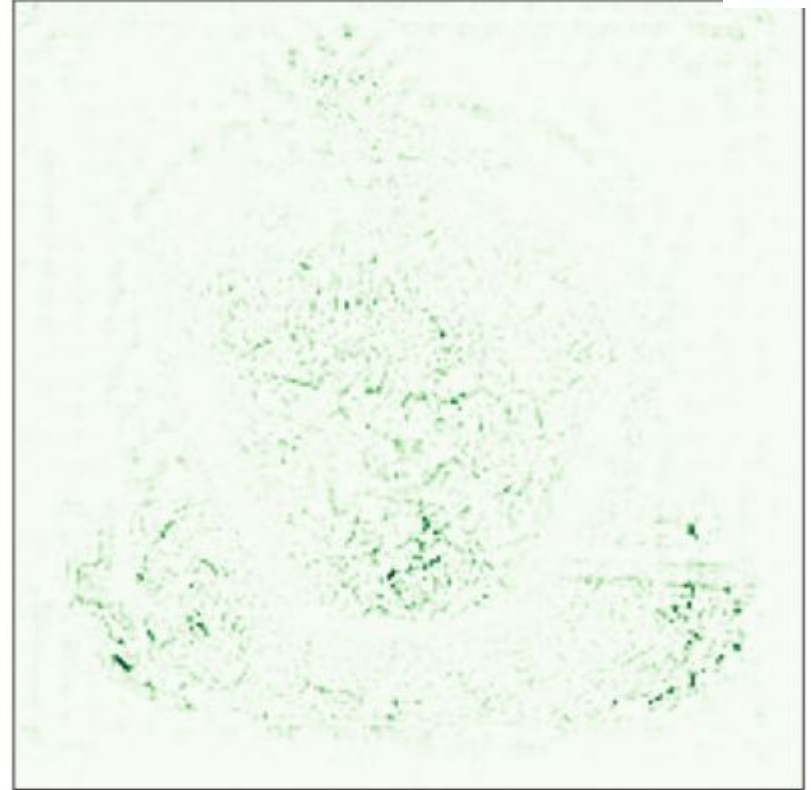
- ✓ Analysis for cat.jpg:
 - Convergence Delta: 2.118891
 - Result: Delta is near 0, justifying the 'Axiom of Completeness'.

Out Come Integrated Gradient

Input: apple.avif



IG Attribution (Target Class: 948)



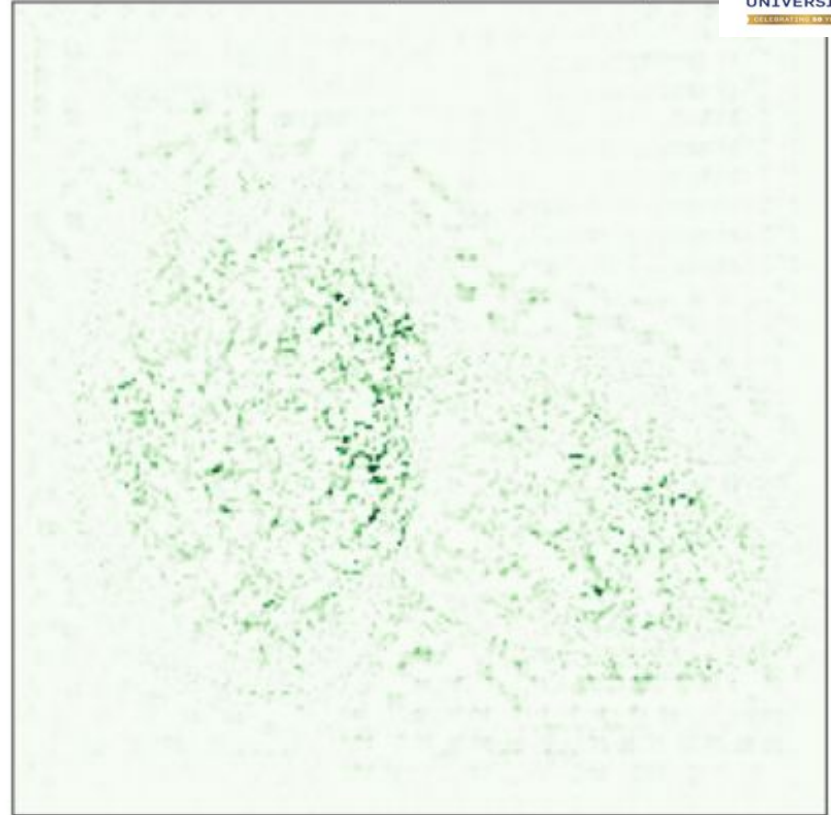
- ✓ Analysis for apple.avif:
 - Convergence Delta: 0.473137
 - Result: Delta is near 0, justifying the 'Axiom of Completeness'.

Out Come Integrated Gradient

Input: mango.png



IG Attribution (Target Class: 940)



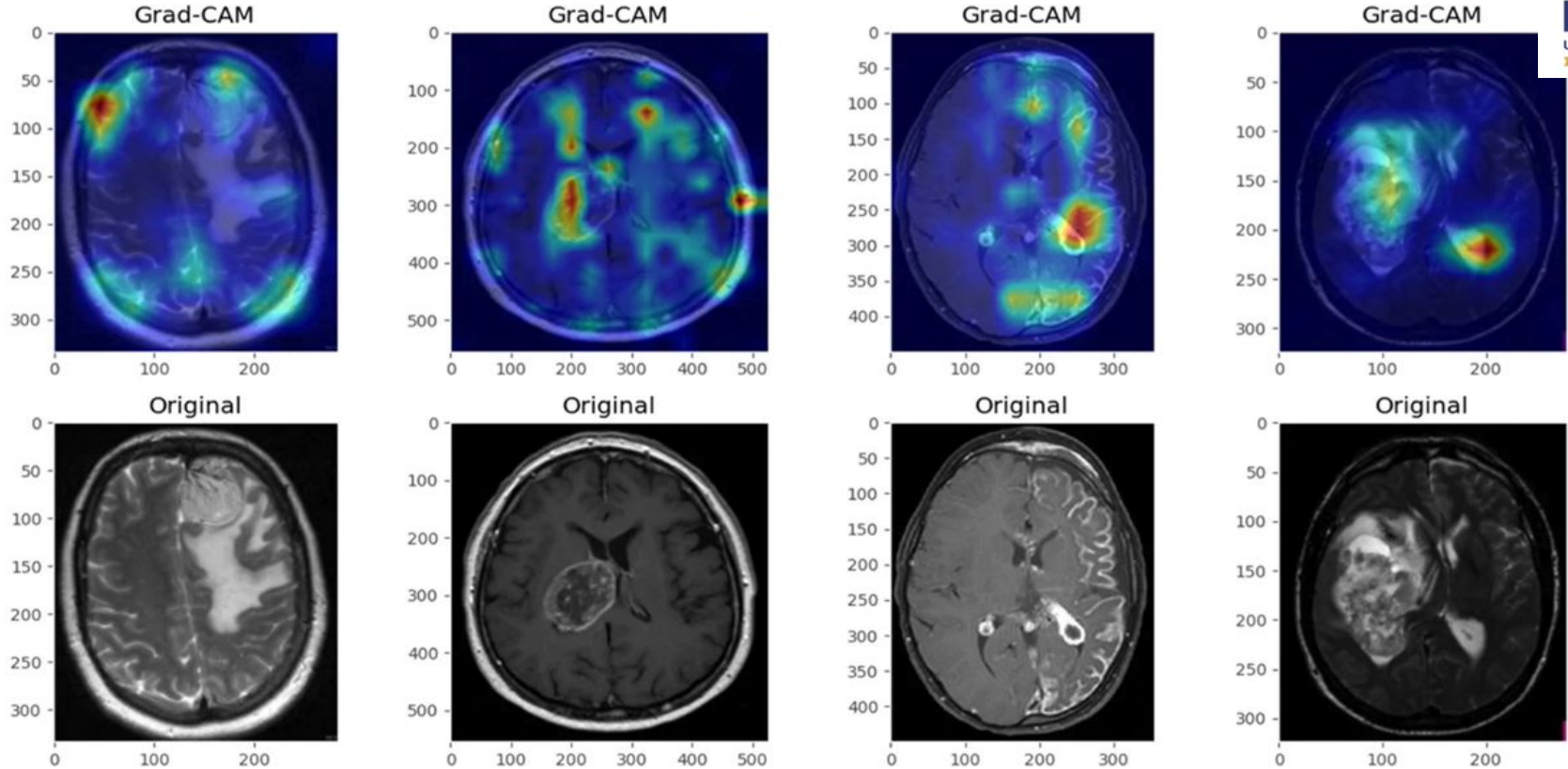
- ✓ Analysis for mango.png:
 - Convergence Delta: 0.858294
 - Result: Delta is near 0, justifying the 'Axiom of Completeness'.

GRAD CAM

Gradient Class activation maps

It shows which portion of an input image is used to classify that image as a given class. It does so by weighting feature maps in a module's convolutional layers by that class's gradients

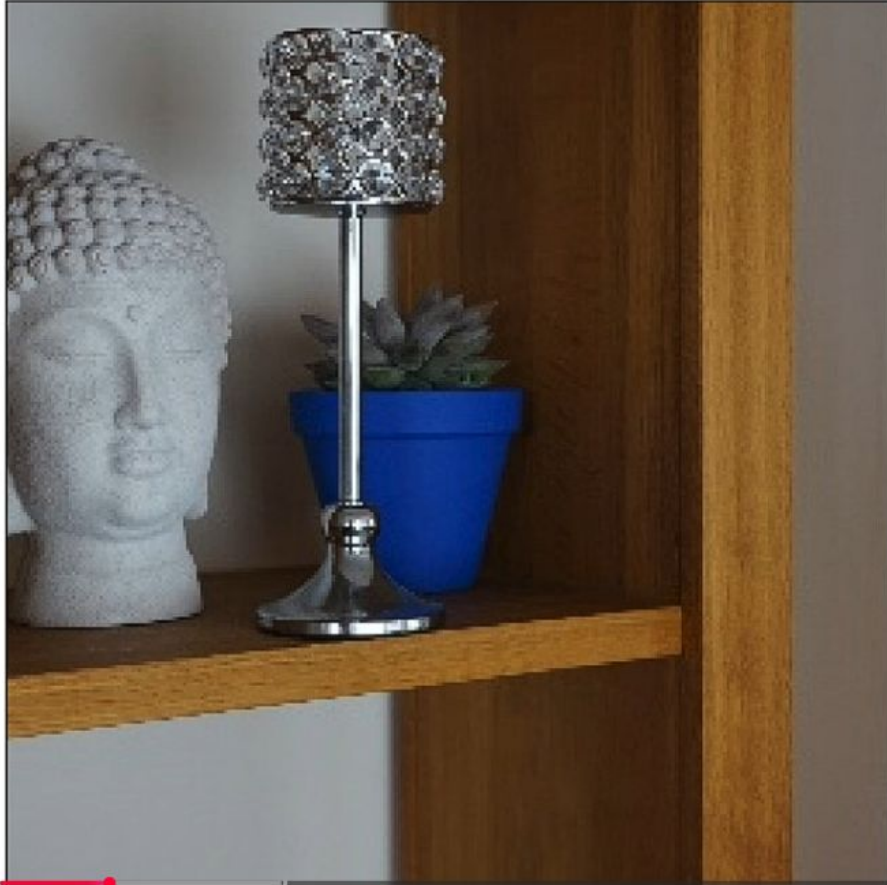
Grad Cam



source: T. R, M., V, V.K. and Guluwadi, S., 2024. Enhancing brain tumor detection in MRI images through explainable AI using Grad-CAM with Resnet 50. BMC medical imaging, 24(1), p.107.

Grad Cam

Input



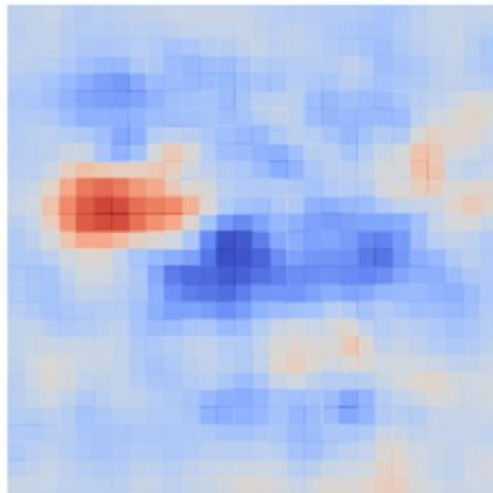
GradCAM Heatmap



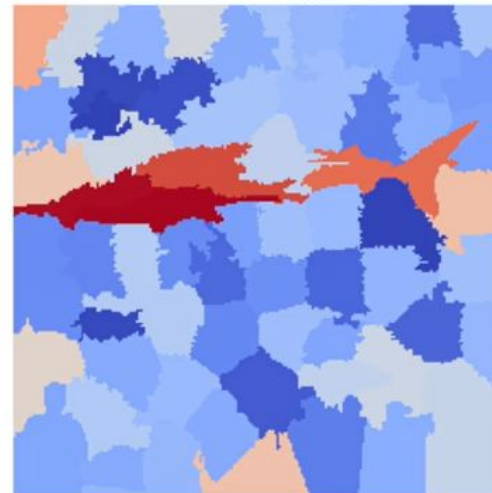
Input Image



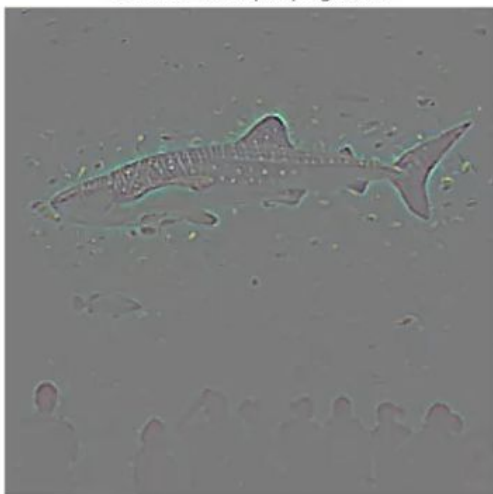
Occlusion



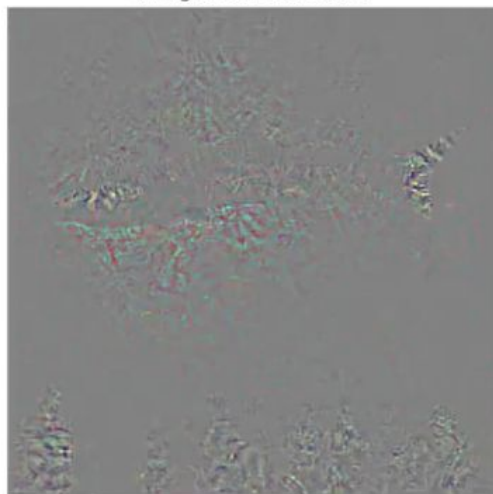
KernelShap



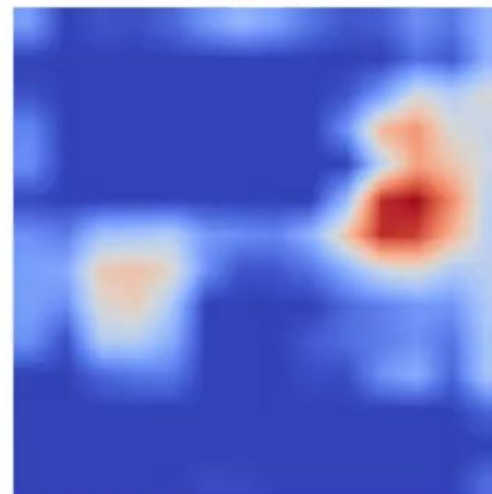
Guided Backpropagation

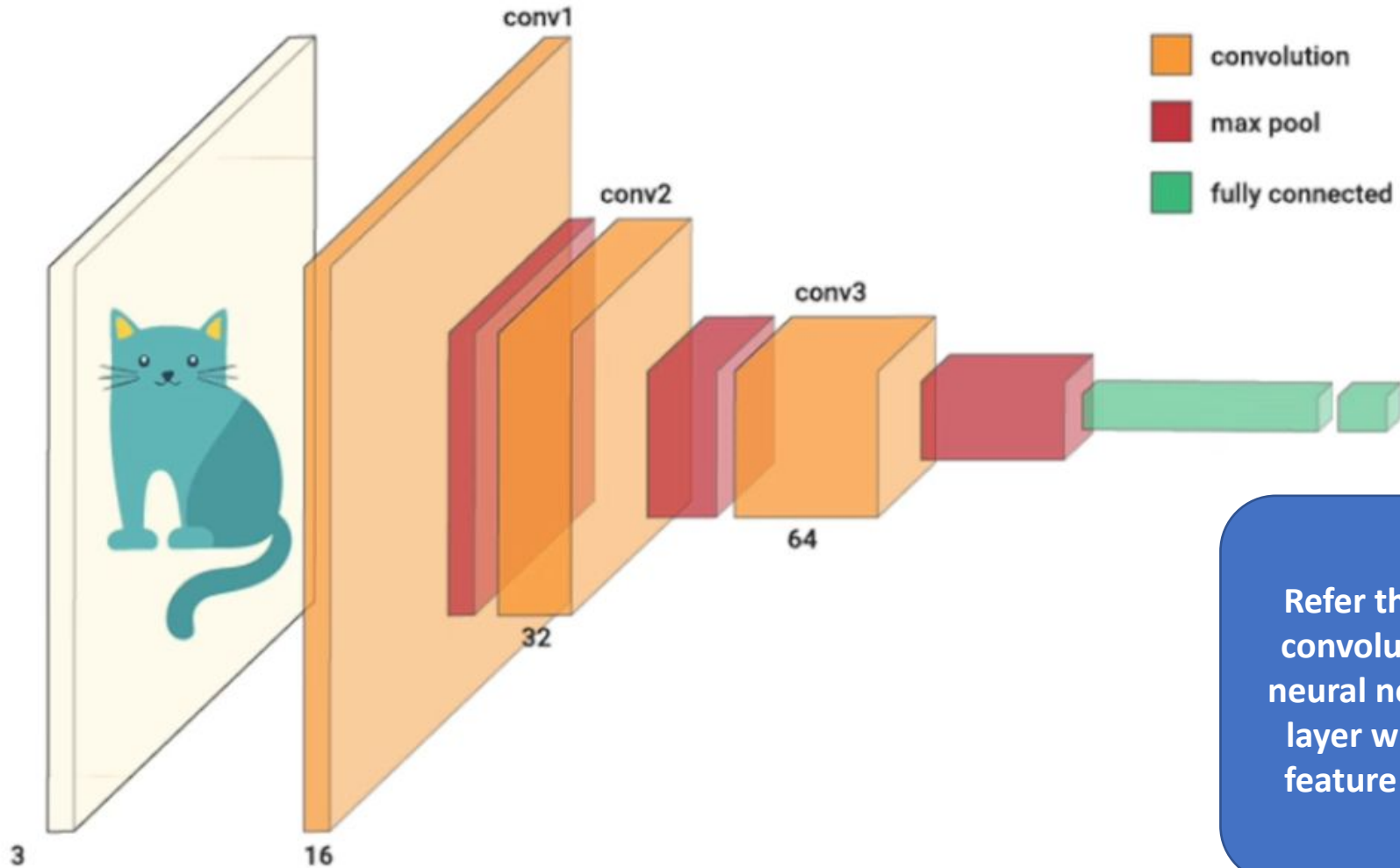


Integrated Gradients



Grad-CAM





Refer the last
convolutional
neural network
layer with 64
feature maps

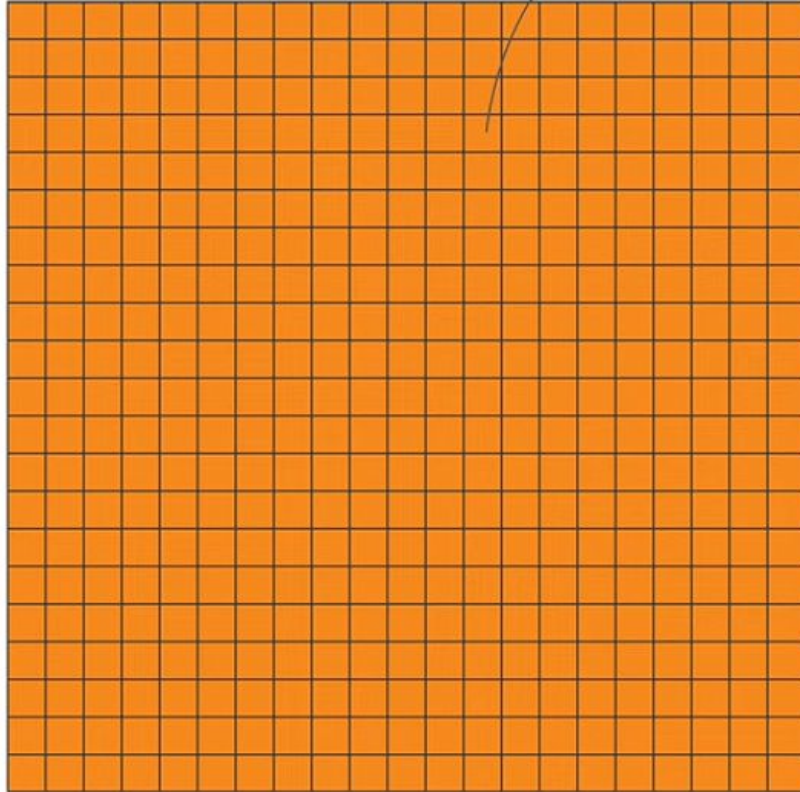
$$a_k^c A^k$$

$$\sum_k a_k^c A^k$$



We refer to arbitrary map A_k with each feature weight and then we do element wise summation to get one matrix of elements

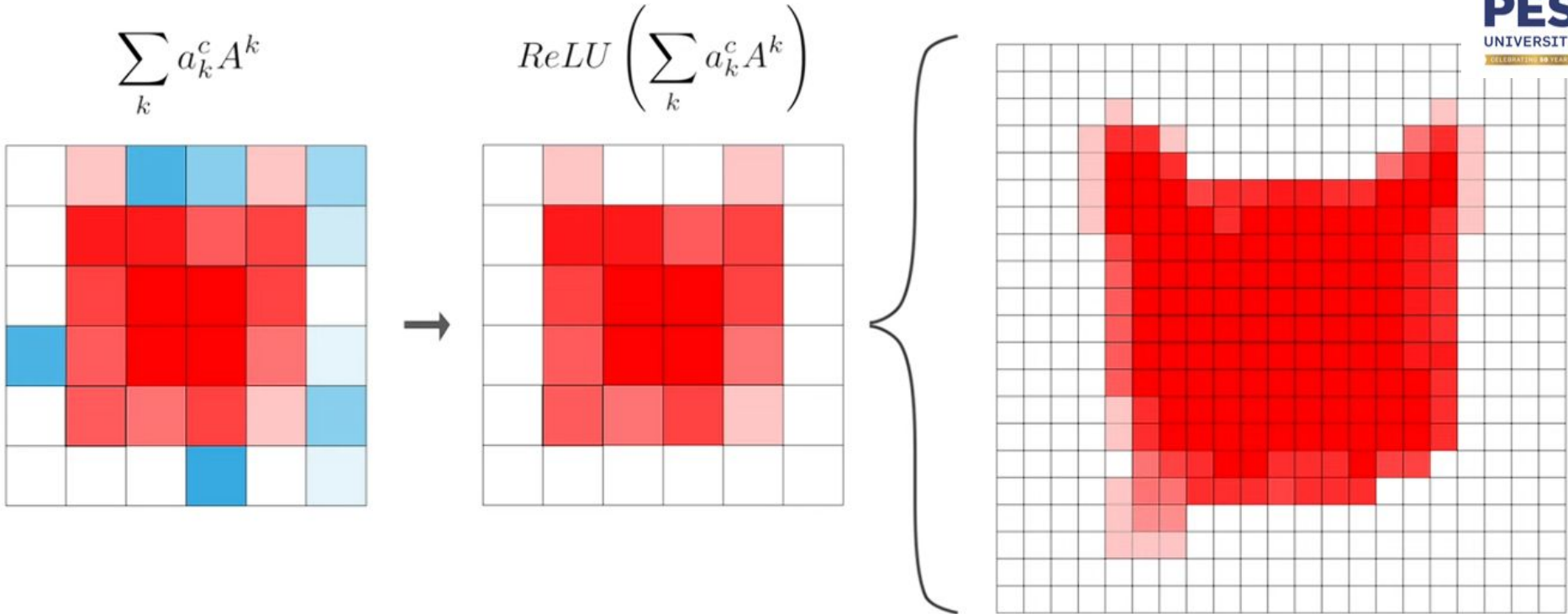
A^k



A_{ij}^k

$$\frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

We sum the
gradient of every
feature map and
divided by total
number of
elements



Elements with negative values in the weighted feature map are set to zero using ReLU activation function resulting in a coarse heat map. Then resizing of the image is done by upsampling using interpolation techniques.

$$\frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

$$L_{Grad-CAM}^c = ReLU \left(\sum_k \boxed{a_k^c} A^k \right)$$


Thus GRAD CAMS is obtained by using the ReLU activation function to the weighted sum of the feature maps in the final convolutional layer where the weight of each feature map is the average gradient off the logit for the predicted class with respect to the elements in the feature map.

Hands on Grad cam

How this code justifies and demonstrates Grad-CAM:



1. **Architecture-Agnostic (Slide 5):** Unlike the original CAM, Grad-CAM works on any CNN (like ResNet50) without needing a specific "Global Average Pooling" layer setup.
2. **Targeting the "Last Conv Layer" (Slide 5):** The code specifically targets `model.layer4`, the final convolutional block. This layer contains the most complex "semantic" information (e.g., "this is a dog's ear") while still keeping track of **where** that feature is in the image.
3. **The Weighted Combination (L^c):** The algorithm calculates gradients to see which "neurons" in that last layer were most excited by the specific class (Beagle, Cat, etc.).
4. **ReLU (Positive Influence):** Per Step 4 on Slide 5, we apply a ReLU to the result. This ensures we only highlight pixels that **positively** contribute to the prediction, ignoring those that might represent other classes.
5. **Upsampling (Slide 5, Step 4):** Since the last conv layer is much smaller than the input image (usually 7×7), the code **interpolates** (bilinear upsampling) the heatmap to stretch it back to the original 224×224 size.

Out Come.....

Input Image: dog.webp



Grad-CAM Localization (Layer 4)



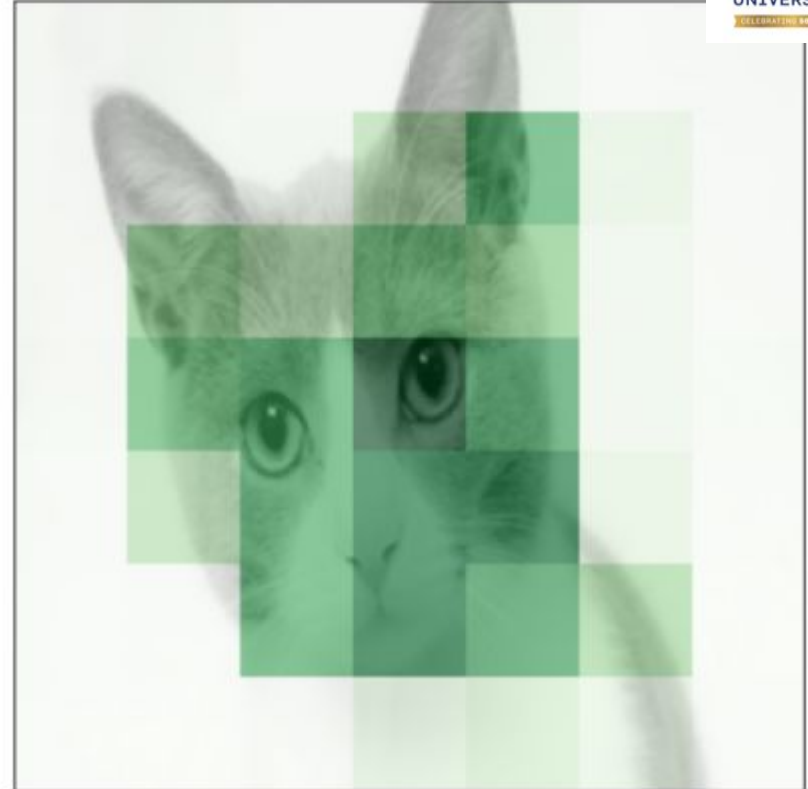
- ✓ Grad-CAM Analysis for dog.webp:
- Methodology: Used gradients flowing into the last conv layer.
 - Justification: Highlighted the regional features (blobs) that define the object.

Out Come.....

Input Image: cat.jpg



Grad-CAM Localization (Layer 4)



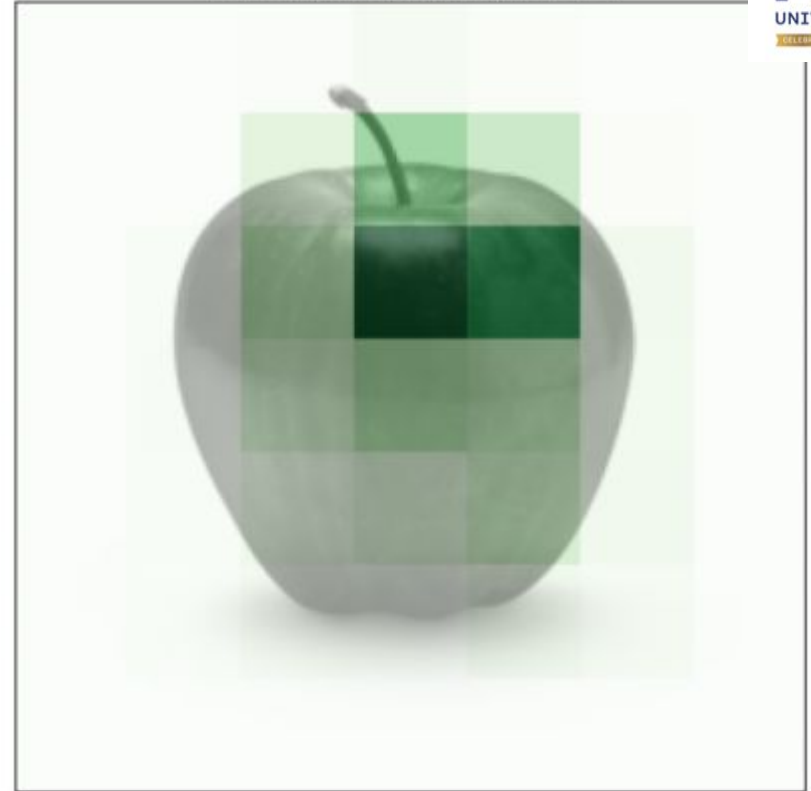
- ✓ Grad-CAM Analysis for cat.jpg:
 - Methodology: Used gradients flowing into the last conv layer.
 - Justification: Highlighted the regional features (blobs) that define the object.

Out Come.....

Input Image: apple.avif



Grad-CAM Localization (Layer 4)



- ✓ Grad-CAM Analysis for apple.avif:
 - Methodology: Used gradients flowing into the last conv layer.
 - Justification: Highlighted the regional features (blobs) that define the object.

Out Come.....

Input Image: mango.png



Grad-CAM Localization (Layer 4)



- ✓ Grad-CAM Analysis for mango.png:
- Methodology: Used gradients flowing into the last conv layer.
 - Justification: Highlighted the regional features (blobs) that define the object.

“Why Should I Trust You?”

“Why Should I Trust You?” Explaining the Predictions of Any Classifier

Marco Tulio Ribeiro
University of Washington
Seattle, WA 98105, USA
marcotcr@cs.uw.edu

Sameer Singh
University of Washington
Seattle, WA 98105, USA
sameer@cs.uw.edu

Carlos Guestrin
University of Washington
Seattle, WA 98105, USA
guestrin@cs.uw.edu

The original LIME paper

Published on arXiv:

<https://arxiv.org/abs/1602.04938>

17000+ citations

ABSTRACT

Despite widespread adoption, machine learning models remain mostly black boxes. Understanding the reasons behind predictions is, however, quite important in assessing trust, which is fundamental if one plans to take action based on a prediction, or when choosing whether to deploy a new model. Such understanding also provides insights into the model, which can be used to transform an untrustworthy model or prediction into a trustworthy one.

In this work, we propose LIME, a novel explanation technique that explains the predictions of any classifier in an interpretable and faithful manner, by learning an interpretable model locally around the prediction. We also propose a method to explain models by presenting representative individual predictions and their explanations in a non-redundant way, framing the task as a submodular optimization problem. We demonstrate the flexibility of these methods by explaining different models for text (e.g. random forests) and image classification (e.g. neural networks). We show the utility of explanations via novel experiments, both simulated

how much the human understands a model's behavior, as opposed to seeing it as a black box.

Determining trust in individual predictions is an important problem when the model is used for decision making. When using machine learning for medical diagnosis [6] or terrorism detection, for example, predictions cannot be acted upon on blind faith, as the consequences may be catastrophic.

Apart from trusting individual predictions, there is also a need to evaluate the model as a whole before deploying it “in the wild”. To make this decision, users need to be confident that the model will perform well on real-world data, according to the metrics of interest. Currently, models are evaluated using accuracy metrics on an available validation dataset. However, real-world data is often significantly different, and further, the evaluation metric may not be indicative of the product's goal. Inspecting individual predictions and their explanations is a worthwhile solution, in addition to such metrics. In this case, it is important to aid users by suggesting which instances to inspect, especially for large datasets.

In this paper, we propose providing explanations for indi-

What is LIME

Local

Interpretable

Model-agnostic

Explanations

Local neighborhood of the instance

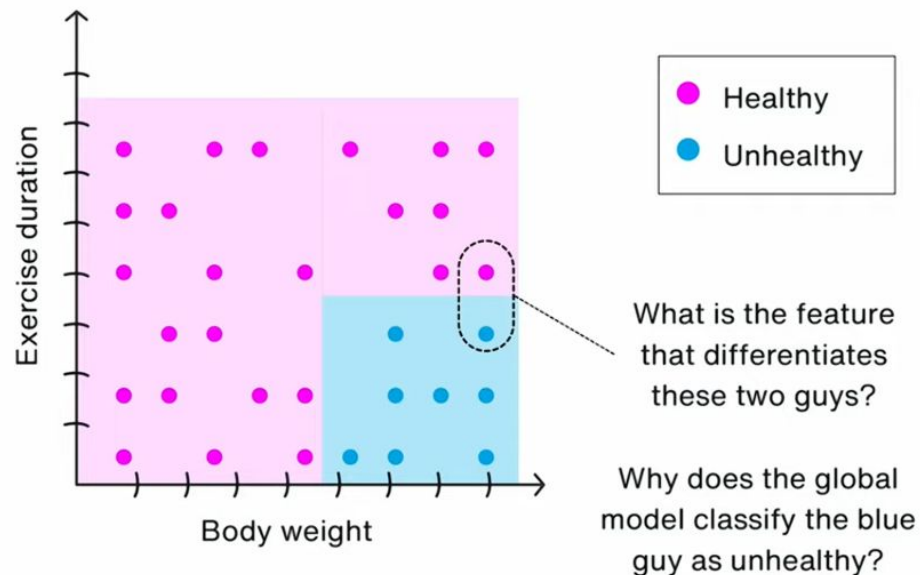
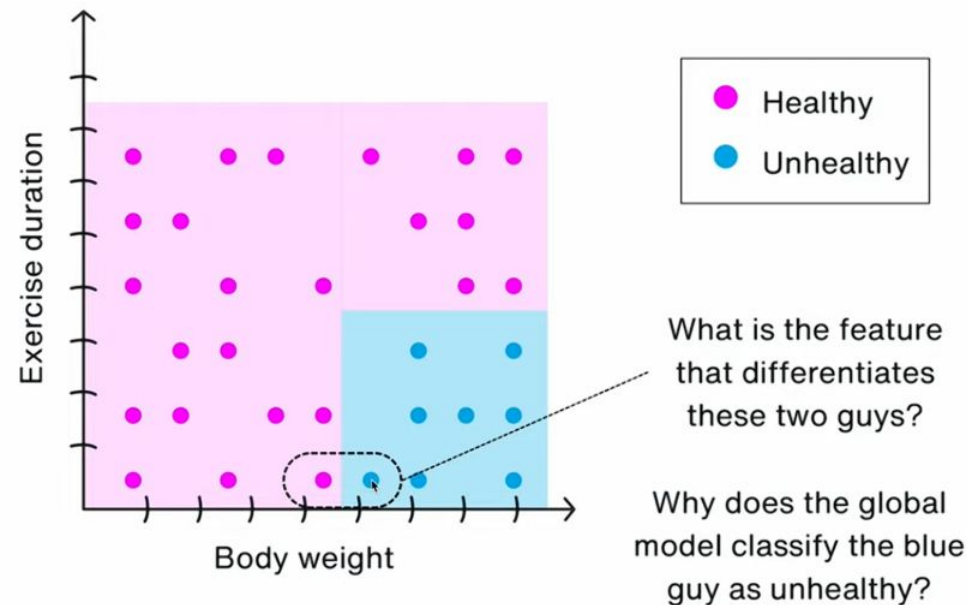
A human should be able to interpret

Applicable for all models

Explanation that helps the interpretation

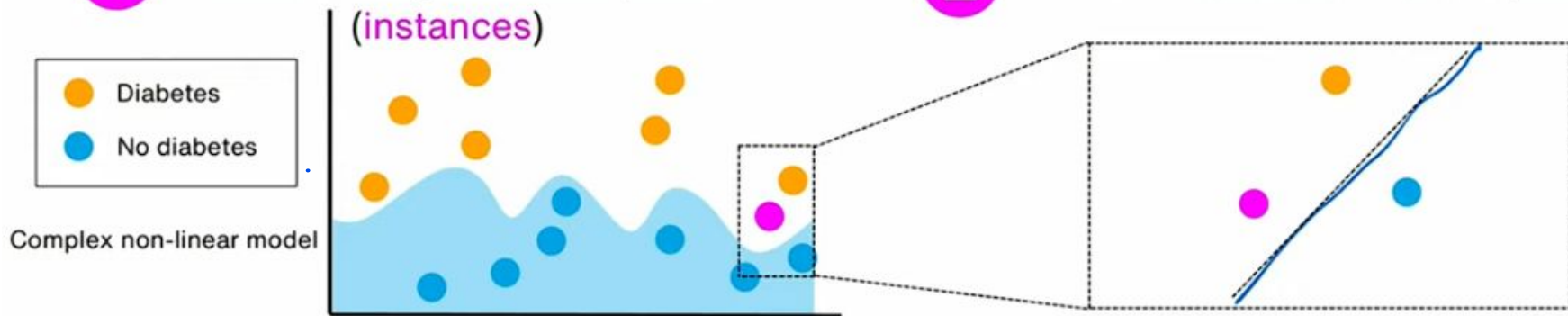
What is LIME

Global and Local Interpretability

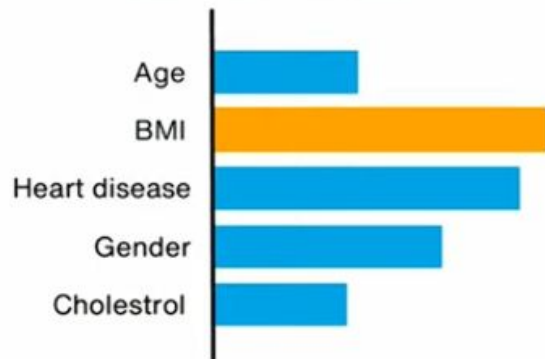
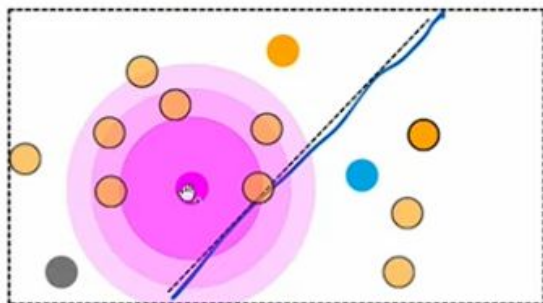


LIME Implementation in 4 Steps

- 1 Global model, local data point
- 2 Local linear model near the point

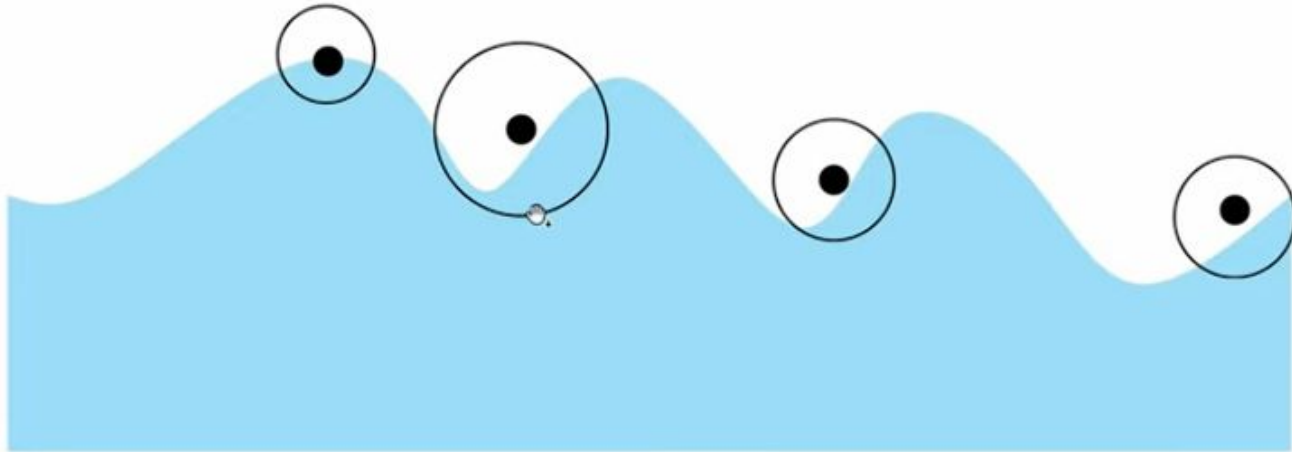


- 3 Perturbed data weighted according to the distance to the local point
- 4 Features relevant to the data point

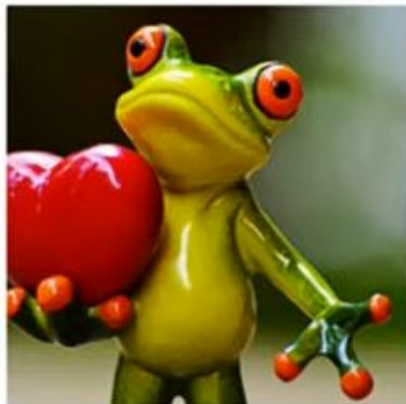


The local border conditions

Local vs. Global Interpretability: LIME explains individual predictions, which might not provide comprehensive insights into the model's overall behavior.



LIME Applied to Images



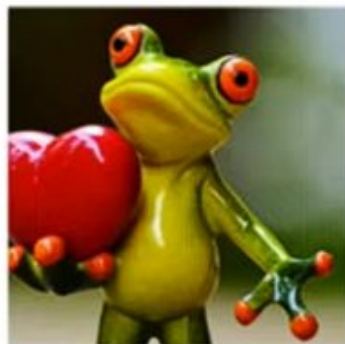
Original Image



Interpretable
Components

Subdivide the image to interpretable components

What Does The Neural Network See?



$$P(\text{Image of frog on leaf}) = 0.54$$



$$P(\text{Image of pool balls}) = 0.07$$



$$P(\text{Image of hot air balloon}) = 0.05$$



LIME Math: Optimization Problem

$$\tilde{\zeta}(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

Diagram illustrating the LIME Math Optimization Problem:

- $\tilde{\zeta}(x)$: Predicted instance we want to explain
- $g \in G$: Family of interpretable models
- $\mathcal{L}(f, g, \pi_x)$: Complex model being explained (Simple/surrogate model)
- $\Omega(g)$: Complexity measure of g (Neighbourhood of x)

$$\mathcal{L}(f, g, \pi_x) = \sum_{z \in \mathcal{Z}} \pi_x(z) (f(z) - g(z))^2$$

Hands on LIME

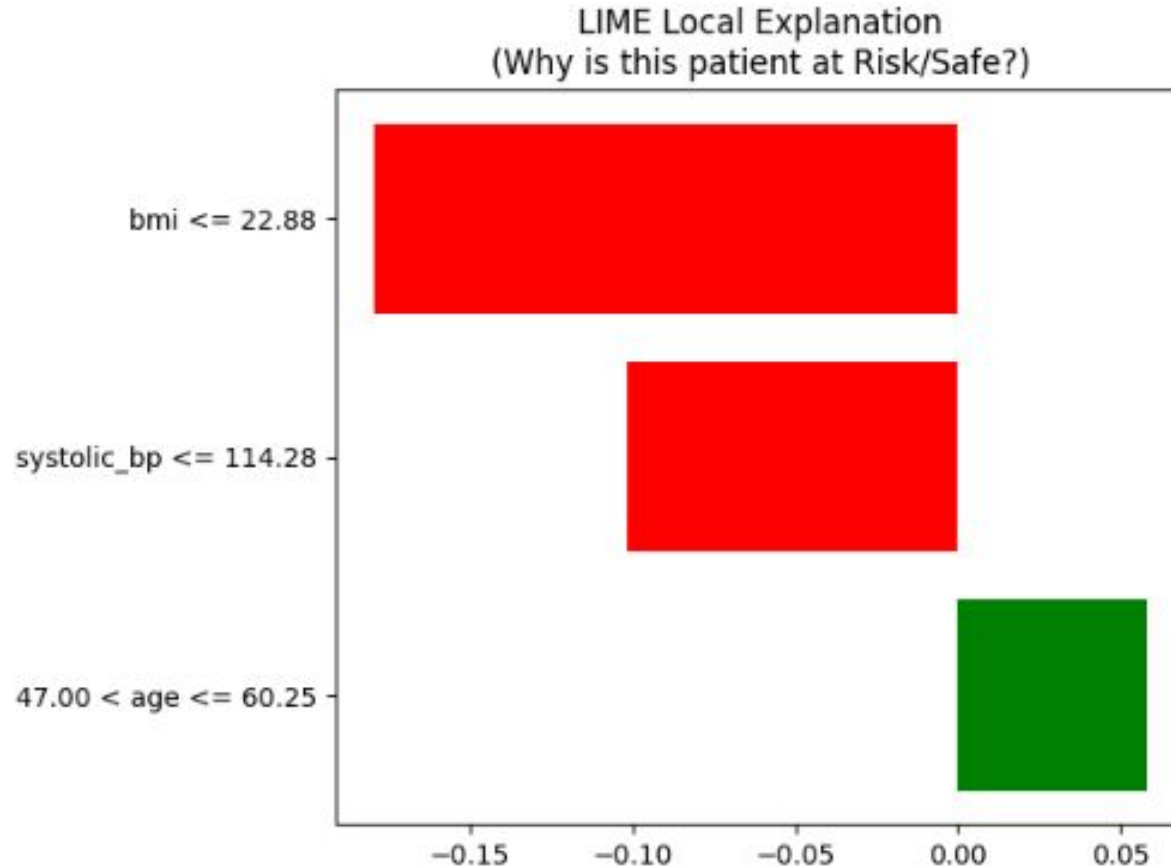
How this code justifies the LIME methodology :

- 1.Local Explanation (Slide 2):** Notice that we didn't explain the whole model. We picked **one patient** (`idx=0`) and explained why the model made *that specific* decision. LIME is "local," not "global."
- 2.Perturbation (Slide 2):** Under the hood, the `explain_instance` function creates thousands of variations of that patient (e.g., slightly different BMI or Age) to see which feature makes the prediction "flip."
- 3.Model-Agnostic:** I used a `RandomForestClassifier`, but you could replace it with a Neural Network or an XGBoost model. LIME doesn't care about the internal math of the model; it only cares about the **Input** → **Output** relationship.
- 4.Feature Importance:** The resulting plot gives you a simple bar chart. For example, it might show: "*This person has high risk BECAUSE their Systolic BP is > 130.*" This is the "Influential Regions" concept mentioned on Slide 2.

Comparison with Pixel Attribution:

- Grad-CAM/IG:** Look at "where" in an image the model looks.
- LIME:** Tells you "which factor" (Age, BMI, or BP) in a spreadsheet drove the model's prediction. It turns complex math into a simple "Reason Code" for a doctor to read.

Out come LIME





SHAP

A Unified Approach to Interpreting Model Predictions

Scott M. Lundberg
Paul G. Allen School of Computer Science
University of Washington
Seattle, WA 98105
slund1@cs.washington.edu

Su-In Lee
Paul G. Allen School of Computer Science
Department of Genome Sciences
University of Washington
Seattle, WA 98105
suinlee@cs.washington.edu

Abstract

Understanding why a model makes a certain prediction can be as crucial as the prediction's accuracy in many applications. However, the highest accuracy for large modern datasets is often achieved by complex models that even experts struggle to interpret, such as ensemble or deep learning models, creating a tension between accuracy and interpretability. In response, various methods have recently been proposed to help users interpret the predictions of complex models, but it is often unclear how these methods are related and when one method is preferable over another. To address this problem, we present a unified framework for interpreting predictions, SHAP (SHapley Additive exPlanations). SHAP assigns each feature an importance value for a particular prediction. Its novel components include: (1) an importance value for a particular prediction. Its novel components include: (1) the identification of a new class of additive feature importance measures, and (2) theoretical results showing there is a unique solution in this class with a set of desirable properties. The new class unifies six existing methods, notable because several recent methods in the class lack the proposed desirable properties. Based on insights from this unification, we present new methods that show improved computational performance and/or better consistency with human intuition than

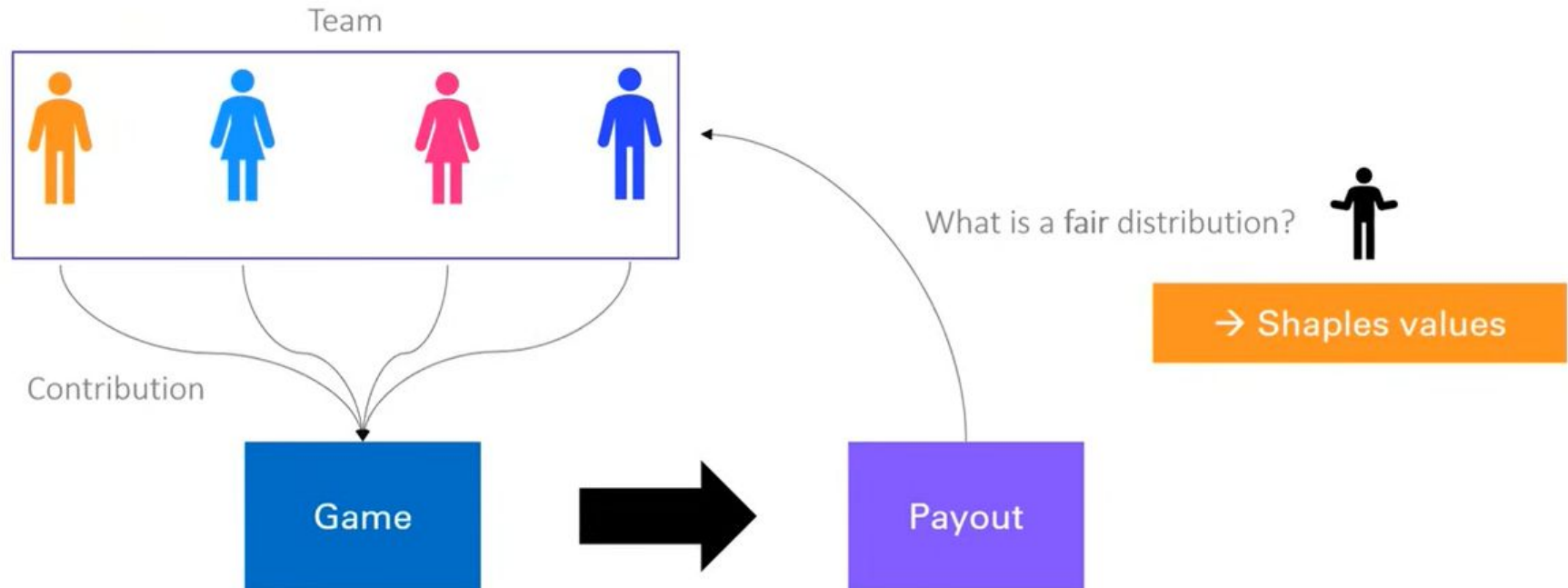
Published on arXiv:

<https://arxiv.org/abs/1705.07874>

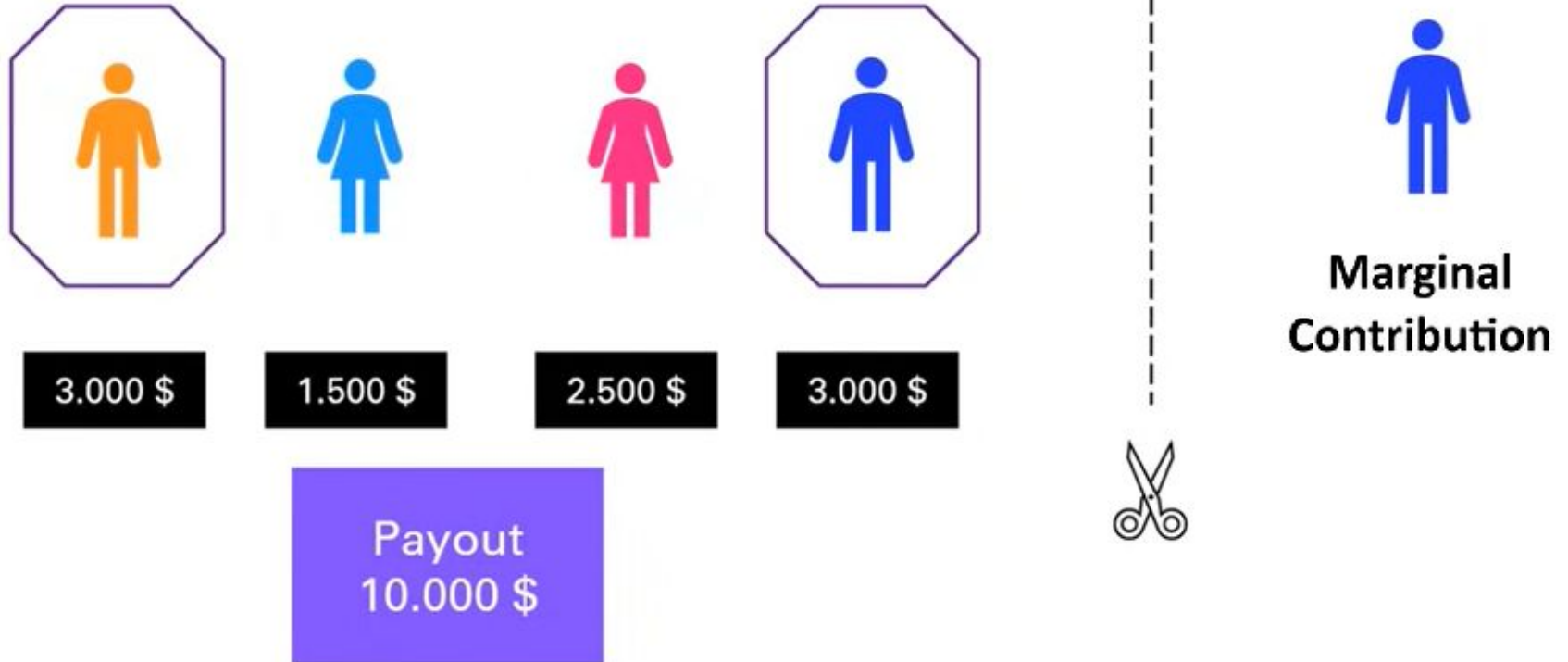
5.07874v2 [cs.AI] 25 Nov 2017

Game pay-out distribution for a team

SHAP (SHapley Additive exPlanations) provides a game theory-based method for explainable AI (XAI), attributing an importance score to each feature in a specific prediction.



Game pay-out distribution for a team



Calculating shapley values

Blackbox model

Input datapoint

Shapley value
for feature i

Subset

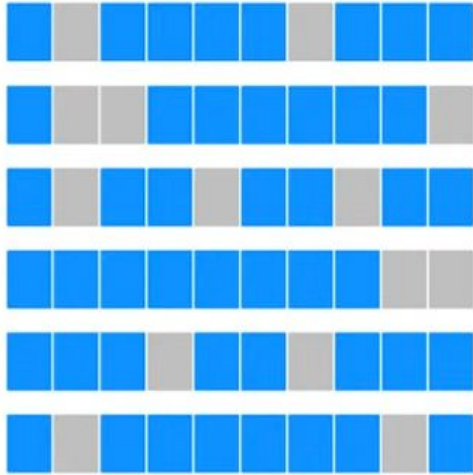
Simplified
data input

Weighting

Contribution

$$\phi_i(f, x) = \sum_{z' \subseteq x'} \frac{|z'|!(M - |z'| - 1)!}{M!} [f_x(z') - f_x(z' \setminus i)]$$

Calculating shapley values



...

$$2^{10} = 1024$$

2^n = total number of
subsets of a set



Kernel SHAP

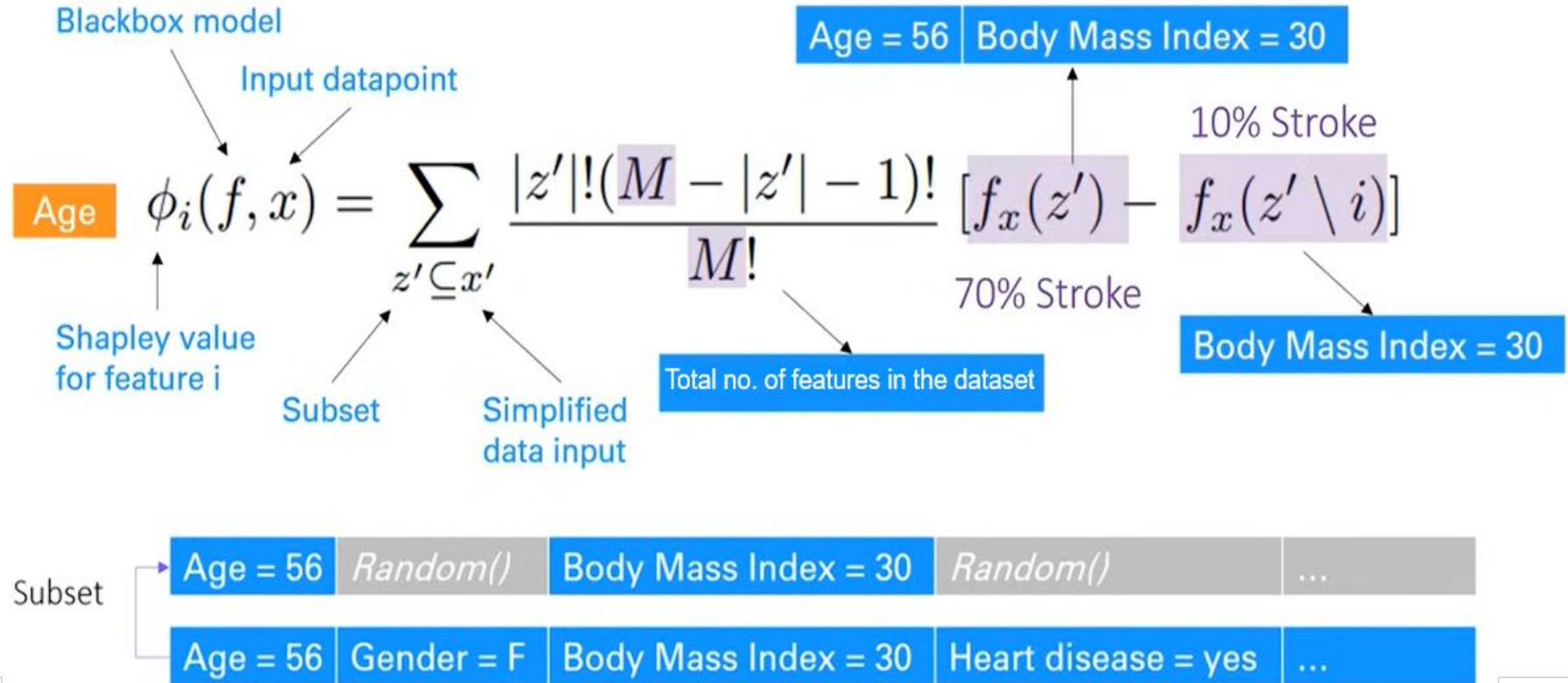


Tree SHAP

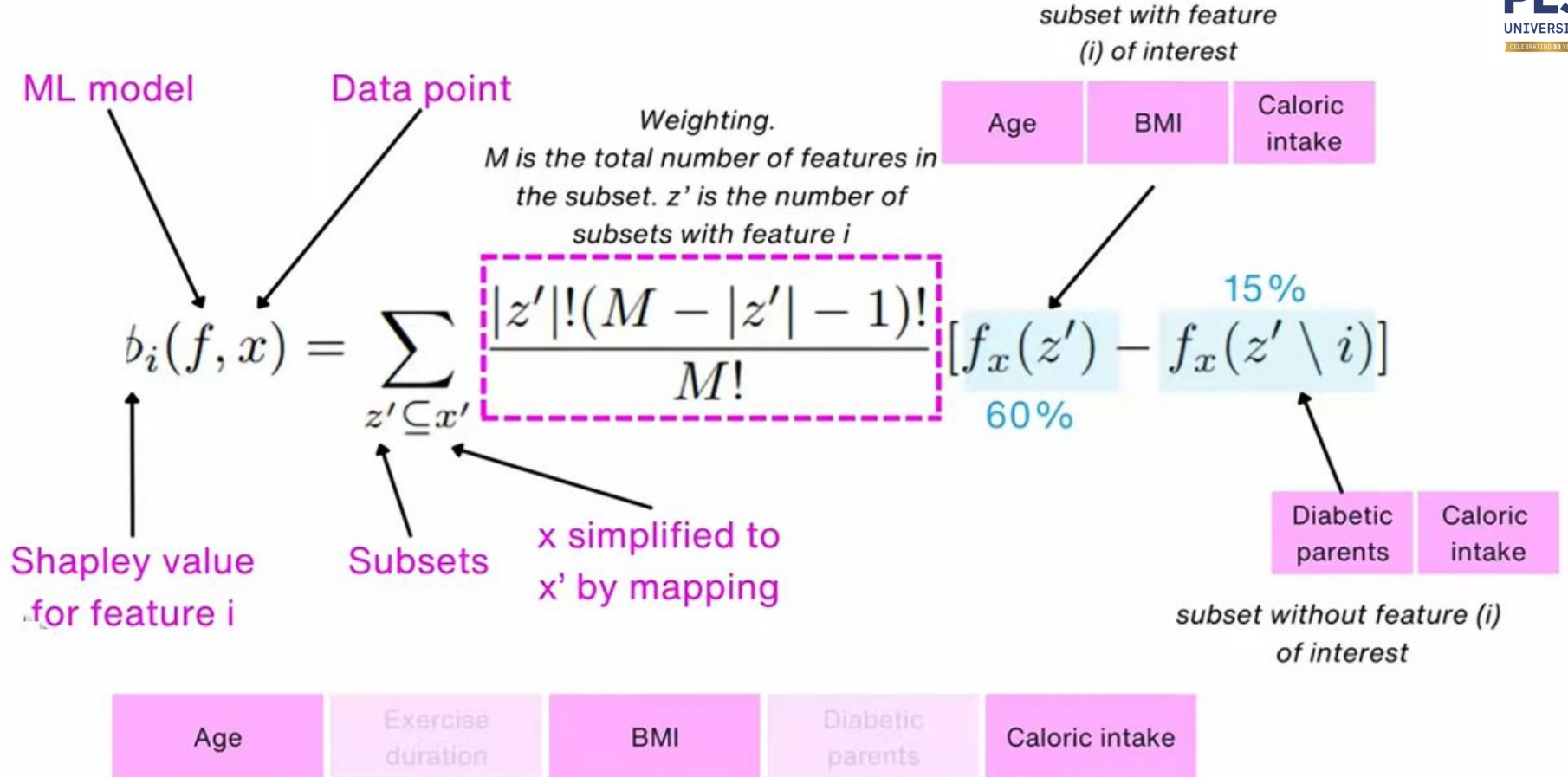


Deep SHAP

Our 'disease-risk' data set example

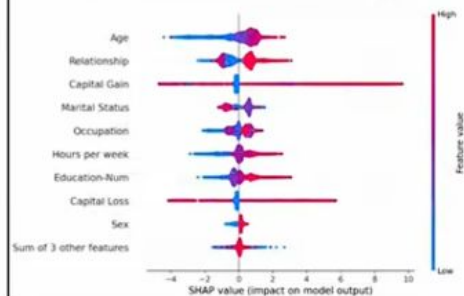


SHAP Math

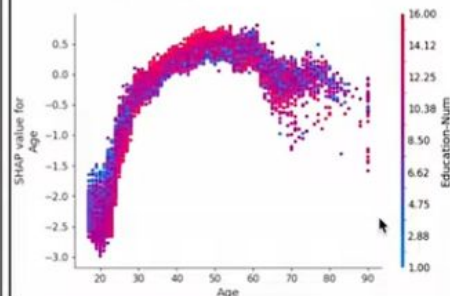


Types of SHAP Plots

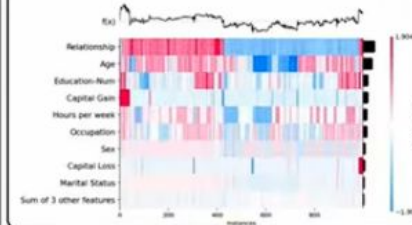
(1) Beeswarm summary plot



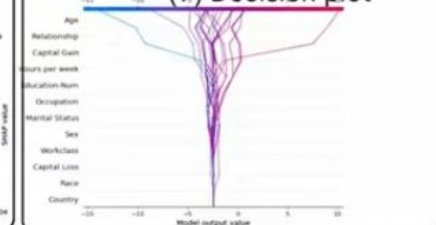
(2) Dependence plot



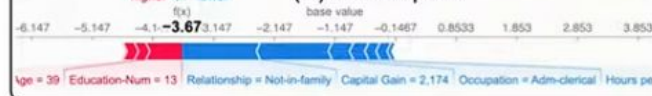
(6) Heatmap



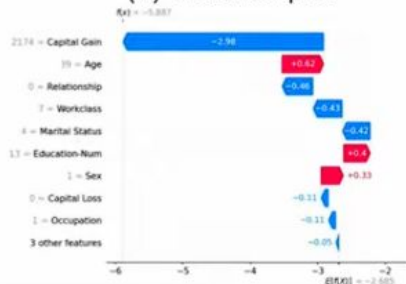
(7) Decision plot



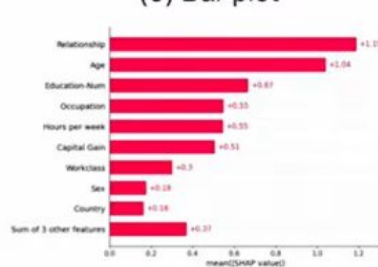
(3) Force plot



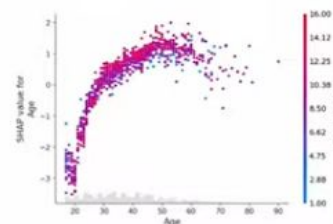
(4) Waterfall plot



(5) Bar plot



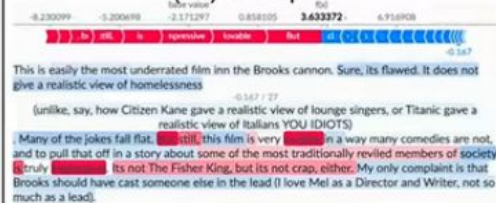
(8) Scatter plot



(9) Image plot

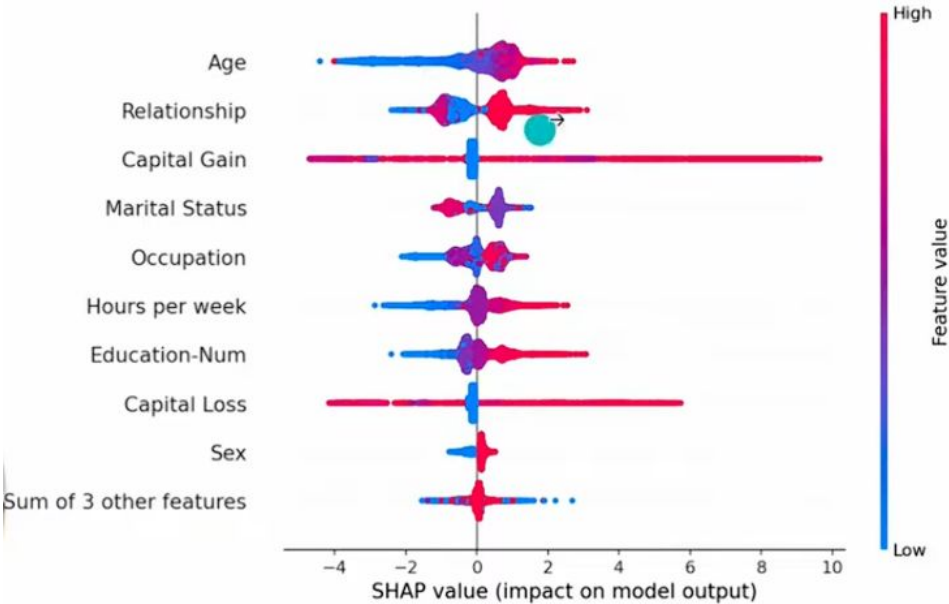


(10) Text plot

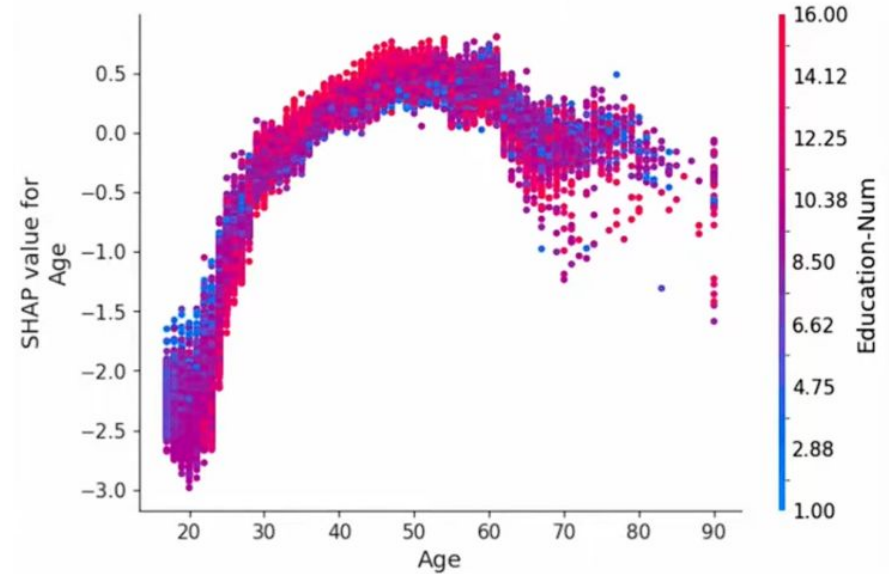


Plots demonstrating SHAP

1) Beeswarm Summary Plot

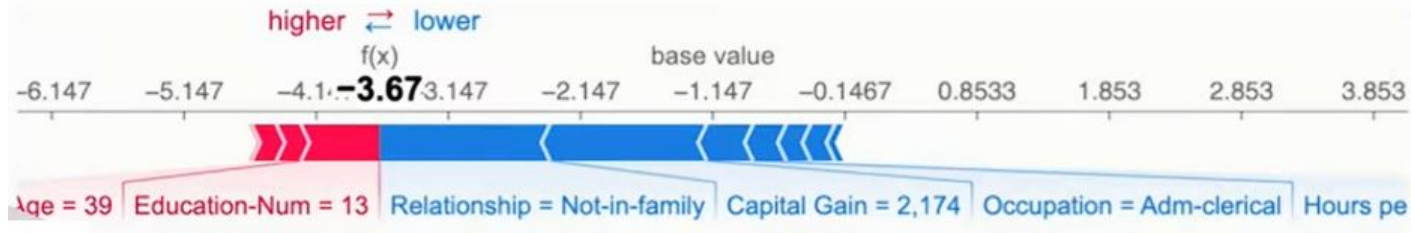


2) Dependence Plot



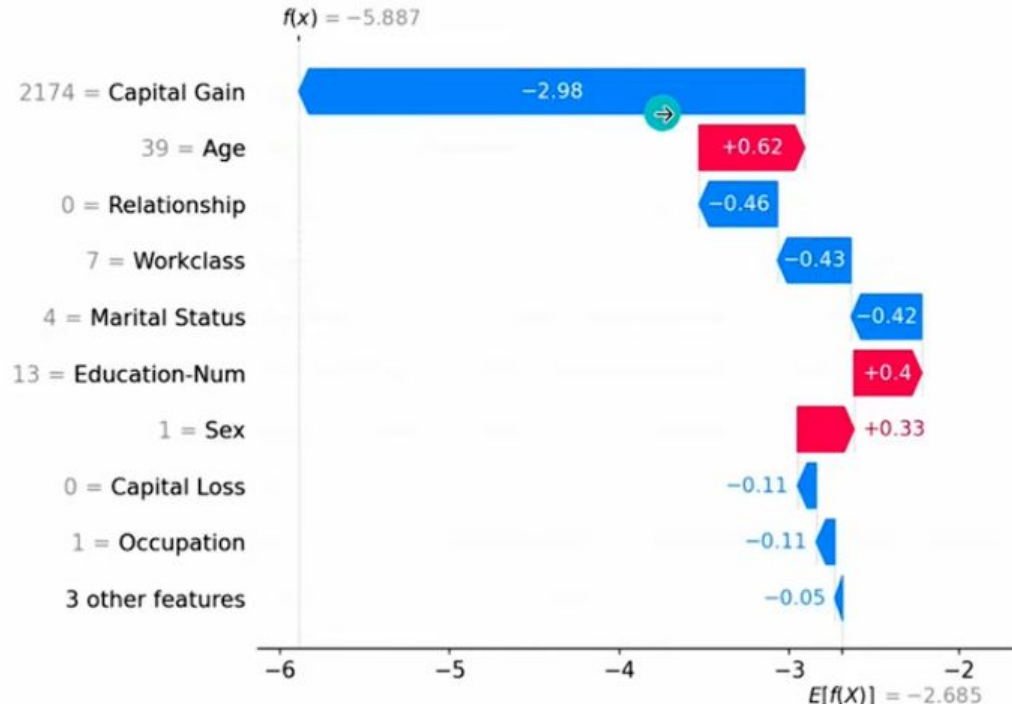
Plots demonstrating SHAP

3) Force Plot



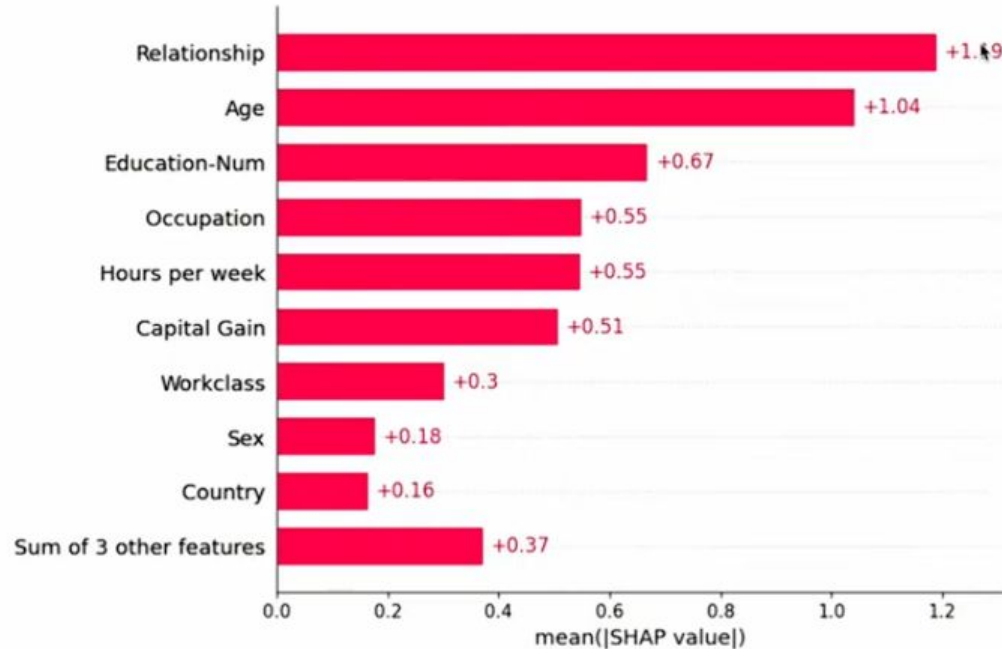
Plots demonstrating SHAP

4) Waterfall Plot



Plots demonstrating SHAP

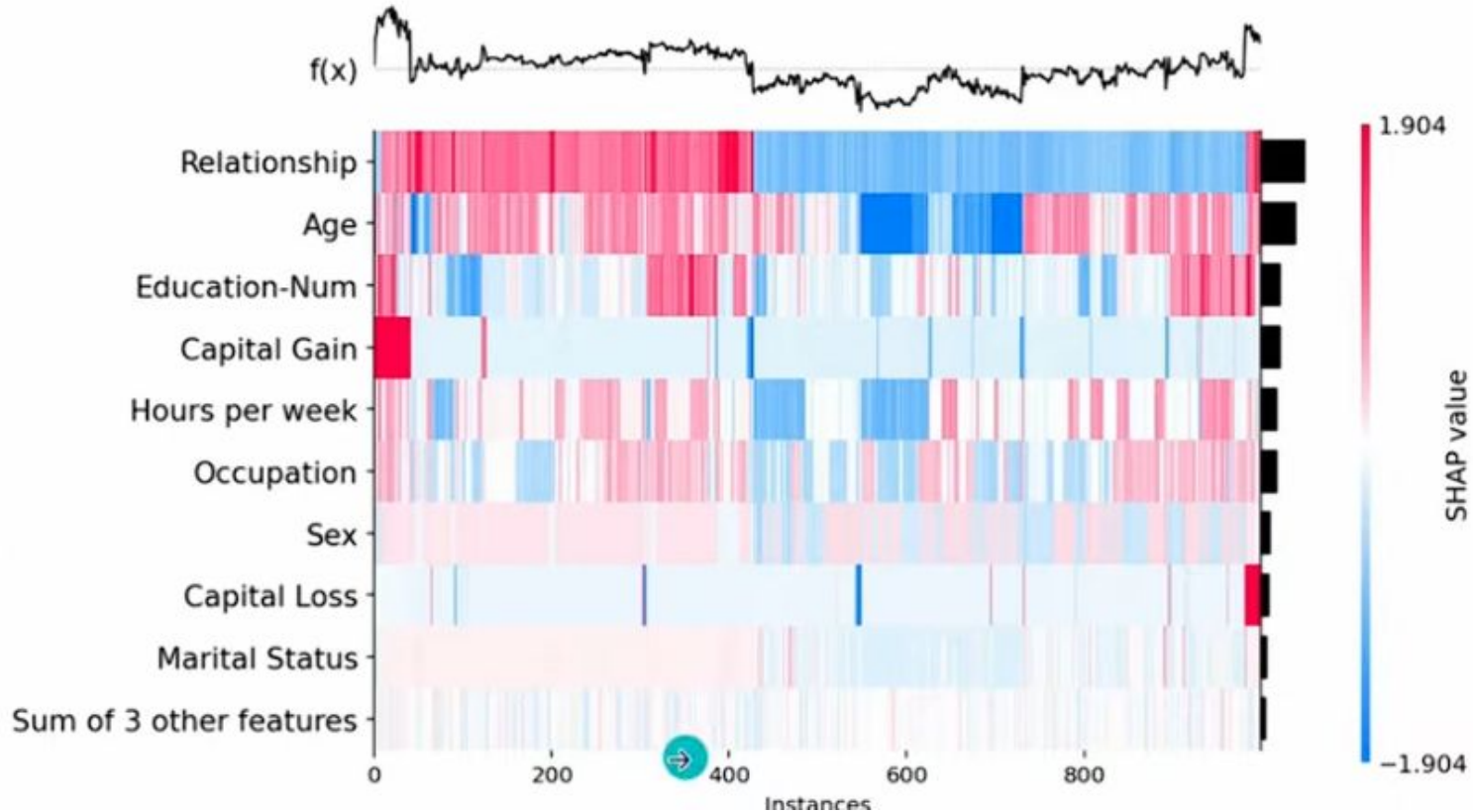
5) Bar Plot



- Description: Displays the average absolute SHAP value for each feature, representing global feature importance.

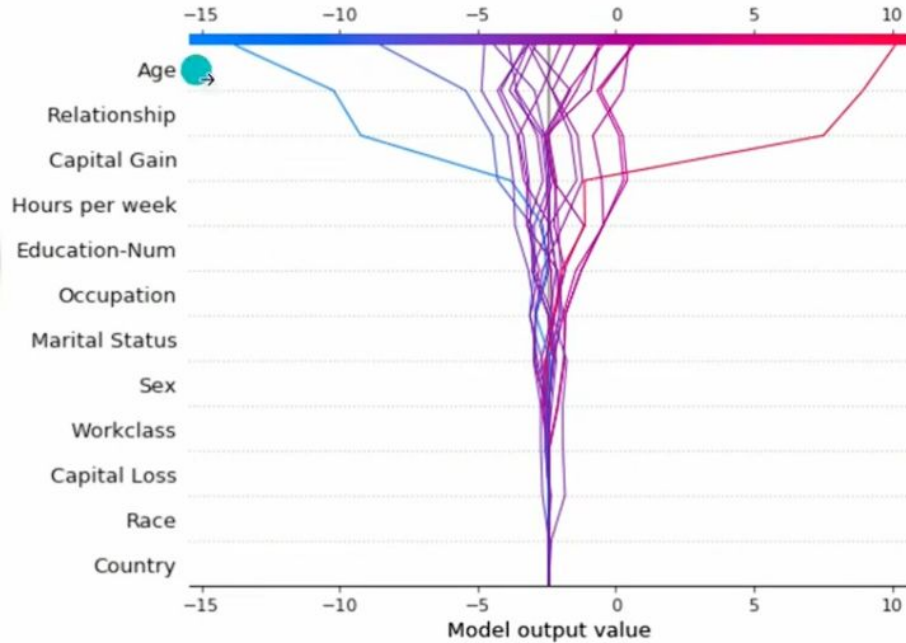
Plots demonstrating SHAP

6) Heatmap



Plots demonstrating SHAP

7) Decision Plot



8) Image Plot



American_egret

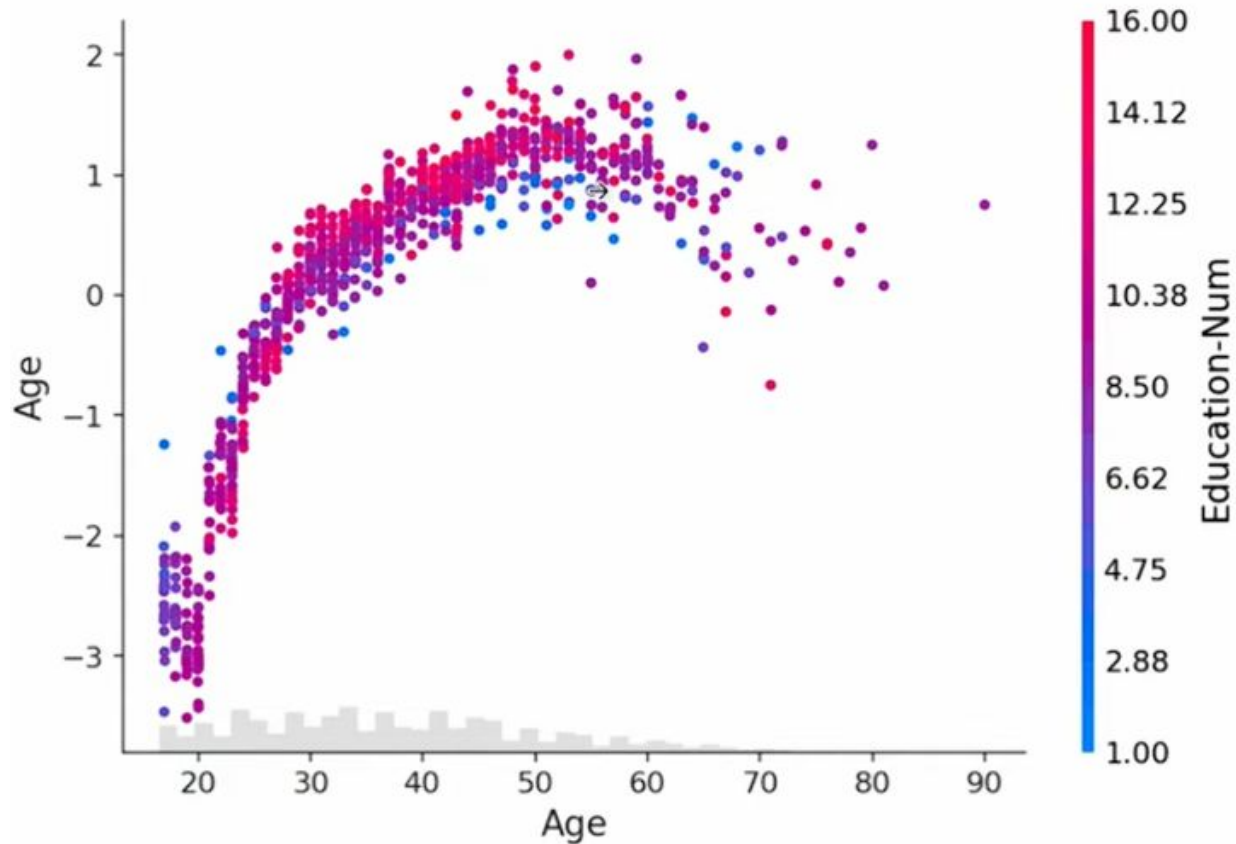


speedboat



Plots demonstrating SHAP

9) Scatter Plot



Plots demonstrating SHAP

10) Text Plot



This is easily the most underrated film inn the Brooks cannon. Sure, its flawed. It does not give a realistic view of homelessness

-0.167 / 27

(unlike, say, how Citizen Kane gave a realistic view of lounge singers, or Titanic gave a realistic view of Italians YOU IDIOTS)

. Many of the jokes fall flat. But still, this film is very lovable in a way many comedies are not, and to pull that off in a story about some of the most traditionally reviled members of society is truly impressive. Its not The Fisher King, but its not crap, either. My only complaint is that Brooks should have cast someone else in the lead (I love Mel as a Director and Writer, not so much as a lead).

Limitations of SHAP

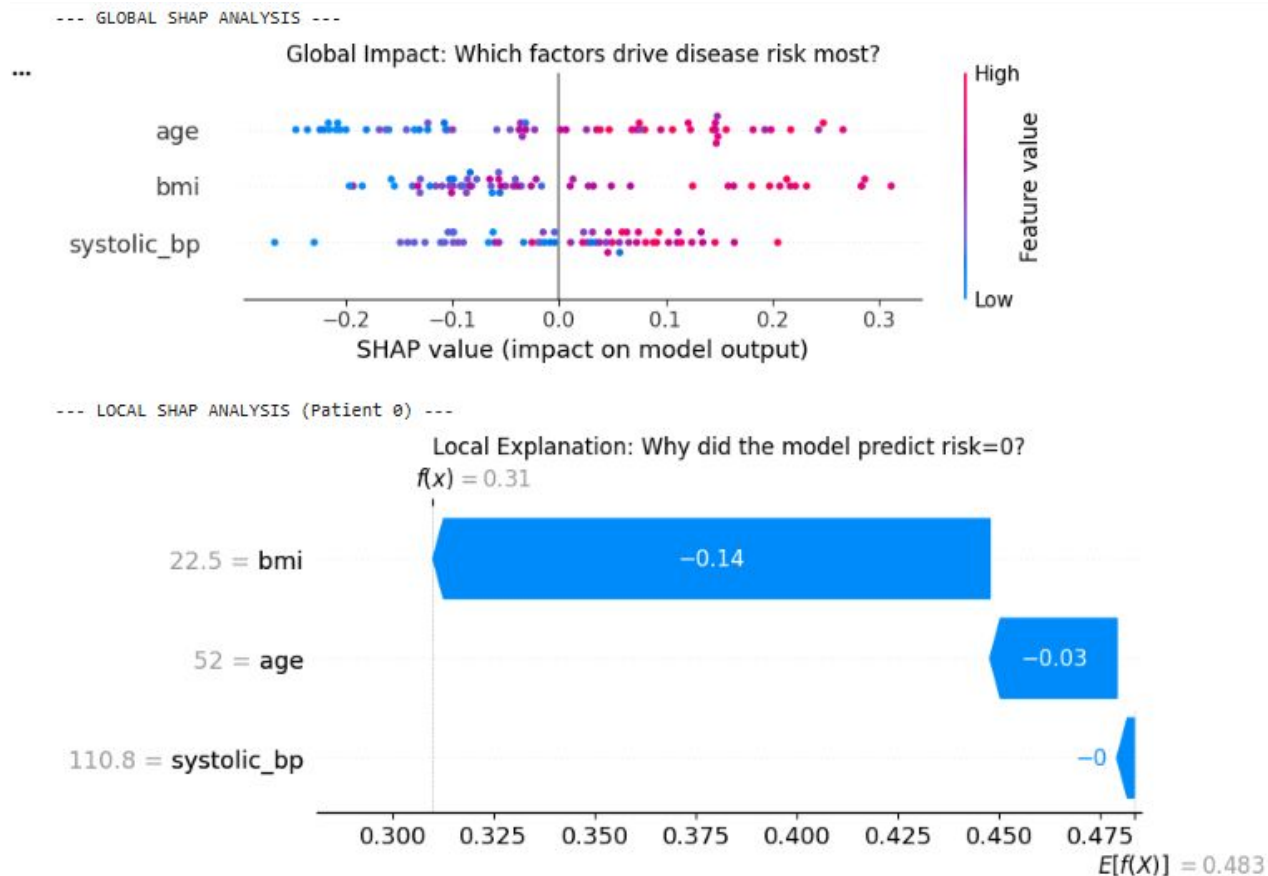
1. **Computational Complexity**: Calculating SHAP values is computationally expensive due to the need to evaluate all possible feature subsets.
2. **Approximation Methods**: Using approximations like KernelSHAP or TreeSHAP can lead to inaccuracies in explanations.
3. **Model Assumptions**: SHAP methods like TreeSHAP assume specific model structures, which might not apply to all models.
4. **Independence Assumption**: SHAP assumes features are independent, which is often not the case in real-world data.
5. **Global Interpretability**: Aggregating local SHAP values to provide global model insights is difficult.

Hands on SHAP

How this script justifies the SHAP method:

- **Local Fairness (Slide 2):** In the **Waterfall Plot**, you will see a $E[f(X)]$ (Base Value). This is the average risk. The arrows show exactly how much age, bmi, or systolic_bp added to or subtracted from that average to get the final result.
- **Global Insight (Slide 6):** The **Beeswarm Plot** shows the "marginal contribution." You can see the density of the data points—for example, if many patients with high BMI (red dots) are pushed to the right, SHAP is proving that BMI is a strong driver for the "Disease" class.
- **Accuracy (Slide 8):** Unlike LIME (which is an approximation), SHAP values are mathematically consistent. If a patient's Blood Pressure goes up, their SHAP value for BP will never go down, satisfying the **Axiom of Monotonicity**.

Out come SHAP



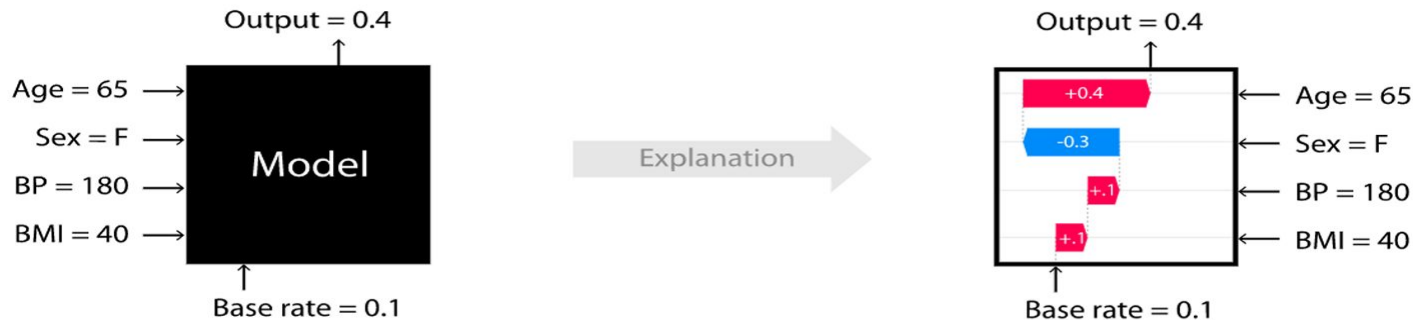
SHAP – SHapley Additive exPlanations

SHAP is a model agnostic explainer:

- Its purpose is to "imitate" the model used.
- It gives an understandable explanation of a local prediction of a model by assigning to each feature a value.
- Based on **Shapely** values from **cooperative game theory**

The *Shapley value* is a mathematical concept in game theory that was introduced by Lloyd Shapley in 1951 [72], later awarded the Nobel Prize in Economics for this work.

SHAP is a method that enables a fast computation of Shapley values and can be used to explain the prediction of an instance x by computing the contribution (Shapley value) of each feature to the prediction.

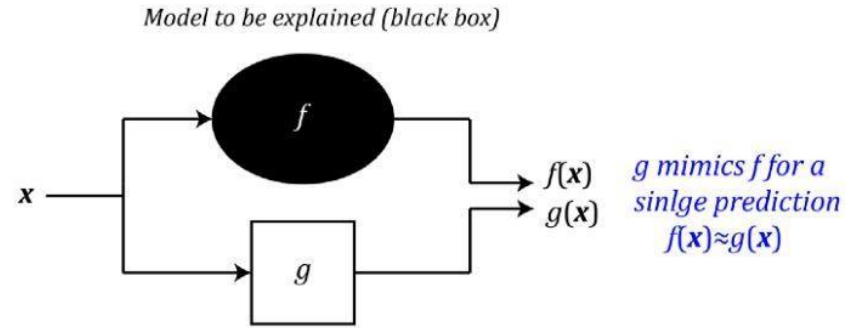


SHAP-How it works?

How to achieve this?

It uses the concept of **Shapley Value**.

The *Shapley value* is a mathematical concept in game theory that was introduced by Lloyd Shapley in 1951 [72], later awarded the Nobel Prize in Economics for this work.



Explainer model (interpretable)



-worth of a coalition on which is Shapley value based on

SHAP-How it works?

- Let's consider a game in co-operative with m numbered players and call F the set of such players.

$$F = \{1, 2, 3, 4, \dots, m\}$$

- We then define an S coalition as a subset of F , which also includes the empty set without players.
- An example of the possible coalitions with 3 players

$$\{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$$

SHAP-How it works?

- Let us now define a function v , called a **characteristic function**, which associates each coalition with a real number. The value $v(S)$ will be called the **worth of the coalition** S and represents the total gain obtained by the coalition if the members act together.
- Obviously in the trivial case:

$$v(\emptyset) = 0$$

- At this point, for the calculation of the value of $\{3\}$, for example, you can proceed by subtracting:

$$v(\{1, 2, 3\}) - v(\{1, 2\})$$

The aim now is to assess what each player's contribution is in the total payoff.

SHAP-How it works?

Considering the number of permutations and summing up on all other possible combinations we get:

$$\checkmark \quad \phi_i = \sum_{S \subseteq F - \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} (\nu(S \cup \{i\}) - \nu(S))$$

where $|F|!$ is the total number of permutations of the grand coalition, S indicates the coalition and $S \cup \{i\}$ indicates the coalition with the addition of i .

The ϕ_i value is called **Shapley Value** and represents the average contribution of the player, or variable, i .

Shapley Values in Machine Learning?

How translate this in Machine Learning?

- **SHAP** (Shapley Additive Explanation)

Input data	x	z'	Simplified local inputs (generally $z' \in (0,1)$)
Input model	$f(x)$	$g(z')$	Explanatory models

$$\text{If } x \approx z' \text{ then } f(x) = g(z')$$

Shapley Values in Machine Learning

We assume:

$$g(z') = c_0 + \sum_{i=1}^M c_i z'_i$$

SHAP values

Additive Feature Attribution

- c_i represent the contribution of each feature.
- c_0 is the average output of the model.
- Easy to interpret.

Shapley Values in Machine Learning

How we calculate SHAP values?

Player $\longleftrightarrow$ Feature

$$\nu(\emptyset) = 0 \longleftrightarrow f(NA, NA, \dots, NA) = E[f(x)] = \frac{1}{k} \sum_{j=1}^k f(x^{(j)})$$

$$f_S(x_S) = E[f(x)|x_S]$$



$$\nu(S) = f_S(x_S) - E[f(x)]$$

Shapley Values in Machine Learning

Finally we get:

$$\phi_i(f, x) = \sum_{z' \subseteq x'} \frac{(|z'| - 1)(|x'| - |z'|)!}{|x'|!} (E[f(z)|z_S] - E[f(z)|z_{S \setminus \{i\}}])$$

Shapley values calculated using the conditional expectations are called **SHAP** values.

SHAP – Desirable Properties

SHAP describes the following three desirable properties:

It is proved that the function $g(z')$ has the following property:

Local Accuracy:

- If $x \approx z'$ then $f(x) = g(z')$.
- *SHAP values add up to the difference between the expected model output and the actual output for a given input. This means that SHAP values provide an accurate and local interpretation of the model's prediction for a given input.*

Missingness:

- If $z' = 0$, then $\phi_i = 0$.
- *SHAP values are zero for missing or irrelevant features for a prediction. This makes SHAP values robust to missing data and ensures that irrelevant features do not distort the interpretation.*

Consistency:

- If feature contribution changes, the feature effect cannot change in the opposite way.
- *SHAP values do not change when the model changes unless the contribution of a feature changes. This means that SHAP values provide a consistent interpretation of the model's behavior, even when the model architecture or parameters change.*

SHAP Explainability Explained

"Who Actually Did the Work?" - A Capstone Project Mystery

The Scenario: Final Capstone Project

Project: "AI-Powered Recipe Recommender App"

Team: 4 students

Final Grade: A- (92%)

The Question: How much did each team member contribute to achieving this grade?

🤔 **What is SHAP?**

SHAP (SHapley Additive exPlanations) is like being the *professor* who needs to figure out exactly how much each student contributed to the final project grade. It's based on game theory and answers: *"What happens to the outcome if we add or remove this person?"*

"In machine learning, SHAP does the same thing but for features instead of people!"

AI-Powered Recipe Recommender



SHAP Analysis: Individual Contributions

Base value (starting point without anyone): **70%** (a functioning but basic app)

Final grade: **92%**

Total improvement to explain: **+22 points**

Team Member	SHAP Value	Impact	Interpretation
Alex 🧑	+12 points	★★★★	The ML algorithm was the core innovation
Sam 🧑	+8 points	★★★	Great UX impressed the professors
Jordan 🧑	+5 points	★	Good data enabled the algorithm
Taylor 👤	-3 points	⚠️	Coordination emails were... unnecessary spam

Visual Breakdown:

Alex's Contribution

+12 points (55%)

Sam's Contribution

+8 points (36%)

Jordan's Contribution

+5 points (23%)

Taylor's Contribution

-3 points
(-14%)



The Magic of SHAP

SHAP considers ALL possible team combinations!

What if Alex worked alone? → 82%

What if Sam worked alone? → 78%

What if Alex + Jordan worked together? → 87%

What if everyone except Taylor worked together? → 95%!

SHAP looks at every possible subset of the team and calculates each person's average marginal contribution across all those scenarios. It's like running every possible "what if" experiment!

Capestone Project Example : Python Implementation

```
# Install if needed
# pip install shap scikit-learn pandas numpy matplotlib

import numpy as np
import pandas as pd
import shap
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

# -----
# 1. Create Synthetic Dataset
# -----
# Each column = contribution score [0-10 scale for example]

np.random.seed(42)

data_size = 200

data = pd.DataFrame({
    "Alex_ML_Algorithm": np.random.uniform(5, 10, data_size),
    "Sam_UI_Interface": np.random.uniform(4, 10, data_size),
    "Tayhen_Dataset": np.random.uniform(3, 10, data_size),
    "Taylor_Coordination": np.random.uniform(1, 8, data_size)
})
```



```
# Create target (Final Grade %)
# Assume weighted importance (hidden true logic)
data["Final_Grade"] = (
    0.4 * data["Alex_ML_Algorithm"] +
    0.3 * data["Sam_UI_Interface"] +
    0.2 * data["Tayhen_Dataset"] +
    0.1 * data["Taylor_Coordination"]
) * 10 + np.random.normal(0, 2, data_size)

# -----
# 2. Train ML Model
# -----
X = data.drop("Final_Grade", axis=1)
y = data["Final_Grade"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

print("Model Test Score:", model.score(X_test, y_test))

# -----
# 3. SHAP Explainability
# -----
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
```



Capestone Project Example : Python Implementation

```
# -----
# 4. Global Contribution Plot
# -----
shap.summary_plot(shap_values, X_test)

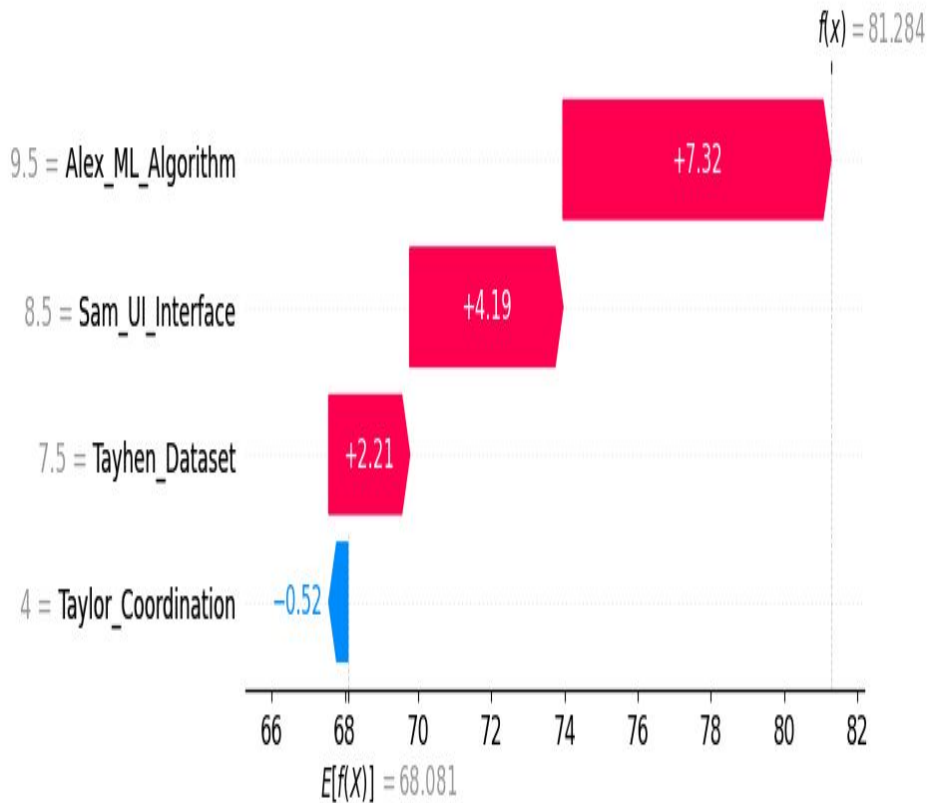
# -----
# 5. Explain One Specific Project (Capstone Team Case)
# -----
# Example: Your actual team scenario

team_project = pd.DataFrame({
    "Alex_ML_Algorithm": [9.5],
    "Sam_UI_Interface": [8.5],
    "Tayhen_Dataset": [7.5],
    "Taylor_Coordination": [4.0]
})

predicted_grade = model.predict(team_project)
print("Predicted Final Grade:", predicted_grade[0])

# SHAP values for this project
team_shap = explainer.shap_values(team_project)

# Waterfall plot (individual contribution)
shap.plots.waterfall.waterfall_legacy(
    explainer.expected_value,
    team_shap[0],
    team_project.iloc[0]
)
```



SHAP Explainers

The SHAP library offers several explainers, each optimized for different types of machine learning models to approximate Shapley values efficiently. The appropriate explainer depends on the specific model architecture you are using.

The main explainers are:

- shap.Explainer**: This is the primary, unified interface for the SHAP library. It can automatically select the appropriate underlying algorithm (e.g., permutation, partition, tree, linear, deep) based on the model and masker you provide. This is the recommended starting point for most users.

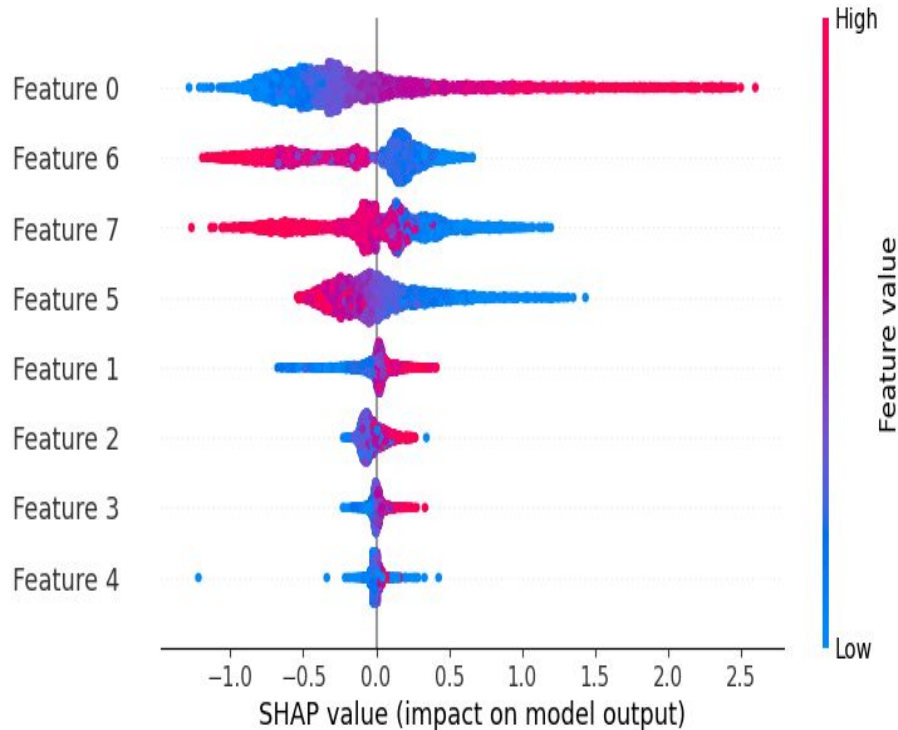
MODEL-SPECIFIC

- shap.TreeExplainer**: This explainer is highly optimized for tree-based models, such as decision trees, random forests, XGBoost, LightGBM, and CatBoost. It is a model-specific method and significantly faster than the model-agnostic alternatives for these model types.
- shap.DeepExplainer**: Tailored for deep learning models (e.g., TensorFlow/Keras, and some PyTorch models), DeepExplainer builds upon the DeepLIFT algorithm to approximate SHAP values efficiently for high-dimensional data like images.
- shap.GradientExplainer**: Also used for deep learning models, this explainer uses expected gradients to compute explanations, which is an extension of integrated gradients. Both DeepExplainer and GradientExplainer are primary options for neural networks.
- shap.LinearExplainer**: This explainer is optimized for linear models, such as

MODEL-AGNOSTIC

- shap.KernelExplainer**: This is a model-agnostic explainer that can be used with any machine learning model or Python function. It uses an approximation method (local surrogate models) to estimate Shapley values, but it is computationally slower than model-specific explainers, especially with a large number of features.
- shap.PermutationExplainer**: This is another model-agnostic explainer that estimates SHAP values by permuting features.
- shap.PartitionExplainer**: This uses Owen values and is optimized for models that can handle hierarchical masking (e.g., in natural language processing with the shap.maskers.Text masker).
- shap.ExactExplainer**: This computes the exact SHAP values but has an exponential time complexity $O(2^m)$ where m is the no. of features

SHAP Visualization



This plot is made of all the dots in the train data. It demonstrates the following information:

- 1.Feature importance:** Variables are ranked in descending order.
- 2. Impact:** The horizontal location shows whether the effect of that value is associated with a higher or lower prediction.
- 3. Original value:** Color shows whether that variable is high (in red) or low (in blue) for that observation.
- 4. Correlation:** A high level of the "Type_of_Cab" content has a low and Positive impact on the "Surge_Pricing_Type". The "low" comes from the blue color, and the "positive" impact is shown on the X-axis. Similarly, the "Trip_Distance" negatively correlates with the target variable.

Red indicates a higher, and **blue** indicates a lower. We can verify the impact (Positive or Negative) from the X-axis for that specific data.

SHAP Visualization Plots

SHAP (SHapley Additive exPlanations) visualization plots are tools used to interpret machine learning model predictions by showing how much each feature contributes to the output.

They are divided into global interpretability (overall model behavior) and local interpretability (individual predictions).

- **Global Interpretation Plots**
- **Local Interpretation Plots**
- **Feature Relationship Plots**
 - Dependence Plot, Interaction Plot
- **Specialized Plots**
 - Image plot, text plot, monitoring plot

Common Use Cases:

- **Identifying Top Features:** Bar and Beeswarm plots.
- **Debugging Individual Predictions:** Waterfall and Force plots.
- **Understanding Feature Trends:** Dependence plots.

https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/plots

1. Global Interpretation Plots(Dataset Level)

Beeswarm Plot (`shap.plots.beeswarm`): The standard plot for summarizing feature importance. It displays the distribution of SHAP values for each feature across all samples. Colors represent feature values (red=high, blue=low), allowing you to see if a high value increases or decreases the prediction.

Summary Plot (`shap.summary_plot`): Similar to the beeswarm, it provides a comprehensive view of feature importance and the direction of their impact.

Bar Plot (`shap.plots.bar`): Displays the mean absolute SHAP values for each feature, showing which features are most important on average.

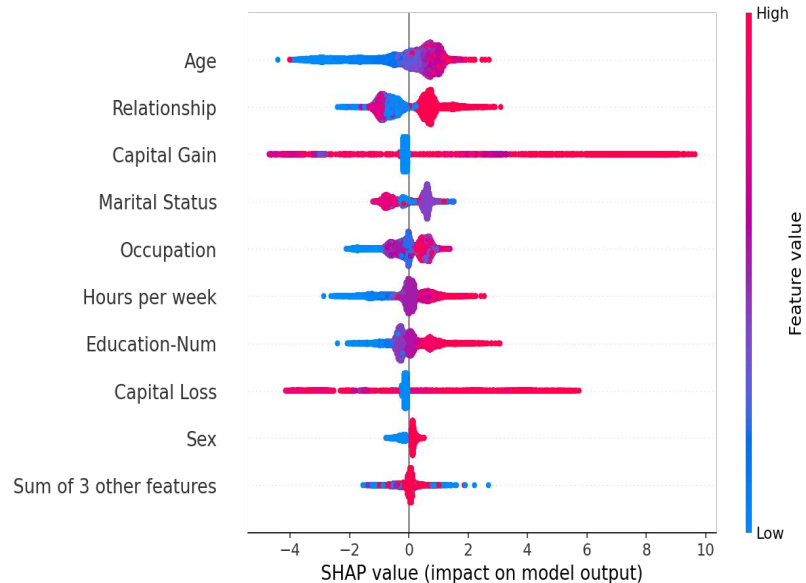
Heatmap (`shap.plots.heatmap`): Shows the population structure of a dataset using supervised clustering, which helps identify patterns in feature contributions across instances.

Uses an XGBoost model trained on the classic UCI adult income dataset (which is a classification task to predict if people made over \">\$50k in the 1990s).

1. Global Interpretation Plot : Beeswarm Plot

- **(shap.plots.beeswarm):** The standard plot for summarizing feature importance.
- It displays the distribution of SHAP values for each feature across all samples.
- Each instance the given explanation is represented by a single dot on each feature row. The x position of the dot is determined by the SHAP value (`shap_values.value[instance,feature]`) of that feature, and dots “pile up” along each feature row to show density.
- Color is used to display the original value of a feature (`shap_values.data[instance,feature]`).
- Summary plot also similar to Beeswarm plots

In the plot below we can see that Age is the most important feature on average, and than young (blue) people are less likely to make over \\$50k.



By default the maximum number of features shown is ten, but this can be adjusted with the `max_display` parameter: `shap.plots.beeswarm(shap_values, max_display=20)`

1. Global Interpretation Plot : Bar Plot

- **(shap.plots.bar)**: Displays the mean absolute SHAP values for each feature, showing which features are most important on average.
- Passing a matrix of SHAP values to the bar plot function creates a global feature importance plot, where the global importance of each feature is taken to be the mean absolute value for that feature over all the given samples.

```
import xgboost
```

```
import shap
```

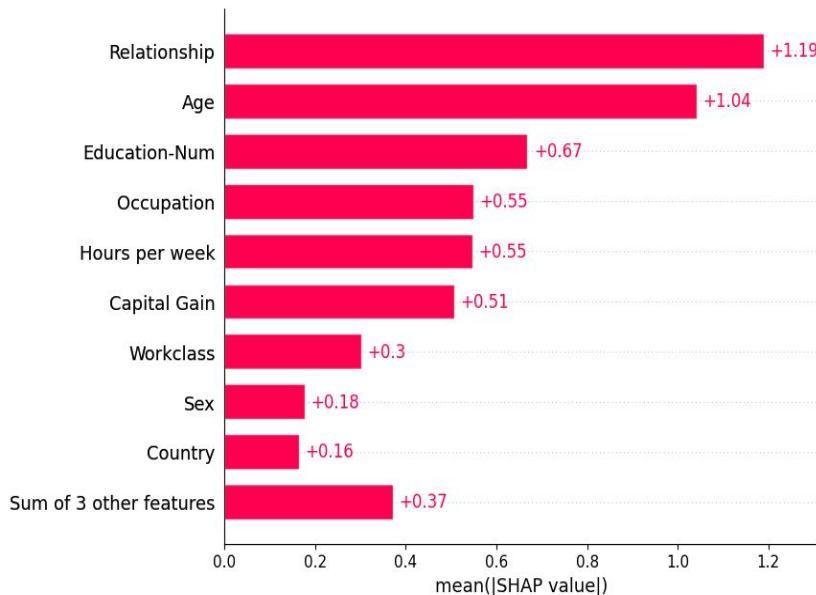
```
# train XGBoost model
```

```
X, y = shap.datasets.adult(n_points=2000)
model = xgboost.XGBClassifier().fit(X, y)
```

```
# compute SHAP values
```

```
explainer = shap.Explainer(model, X)
shap_values = explainer(X)
```

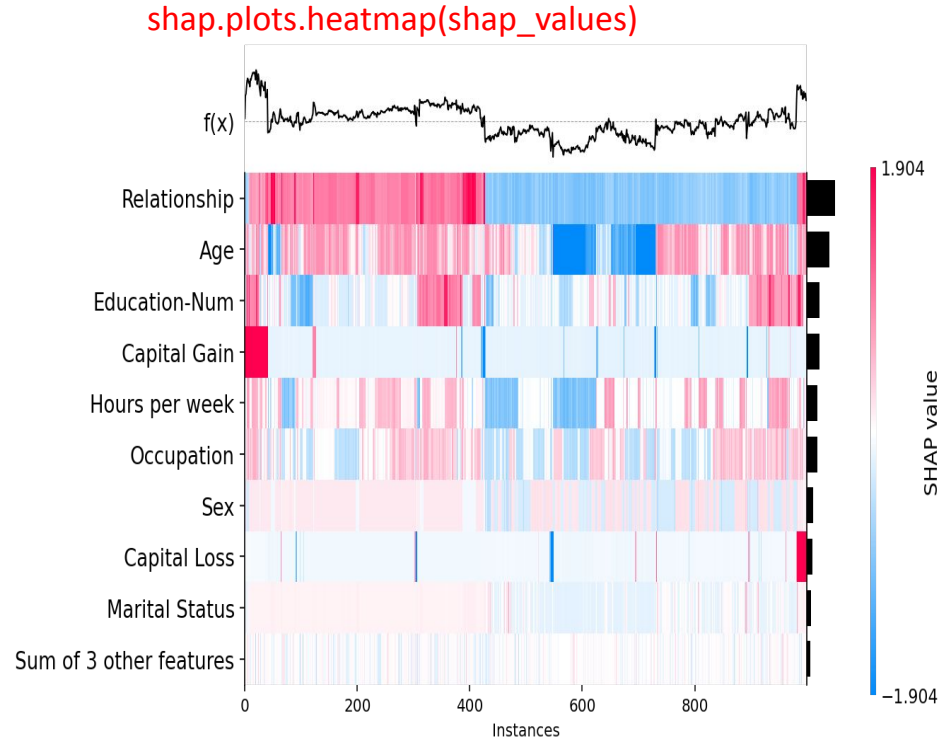
In the plot below we can see that Age is the most important feature on average, and then young (blue) people are less likely to make over \$50k.



By default the maximum number of features shown is ten, but this can be adjusted with the `max_display` parameter: `shap.plots.bar(shap_values, max_display=12)`

1. Global Interpretation Plot : Heat Map

- **(shap.plots.heatmap):** Shows the population structure of a dataset using supervised clustering, which helps identify patterns in feature contributions across instances.
- Passing a matrix of SHAP values to the heatmap plot function creates a plot with the instances on the x-axis, the model inputs on the y-axis, and the SHAP values encoded on a color scale.
- The output of the model is shown above the heatmap matrix (centered around the explanation's `.base_value`), and the global importance of each model input shown as a bar plot on the right hand side of the plot (by default this is the `shap.order.abs.mean` measure of overall importance).

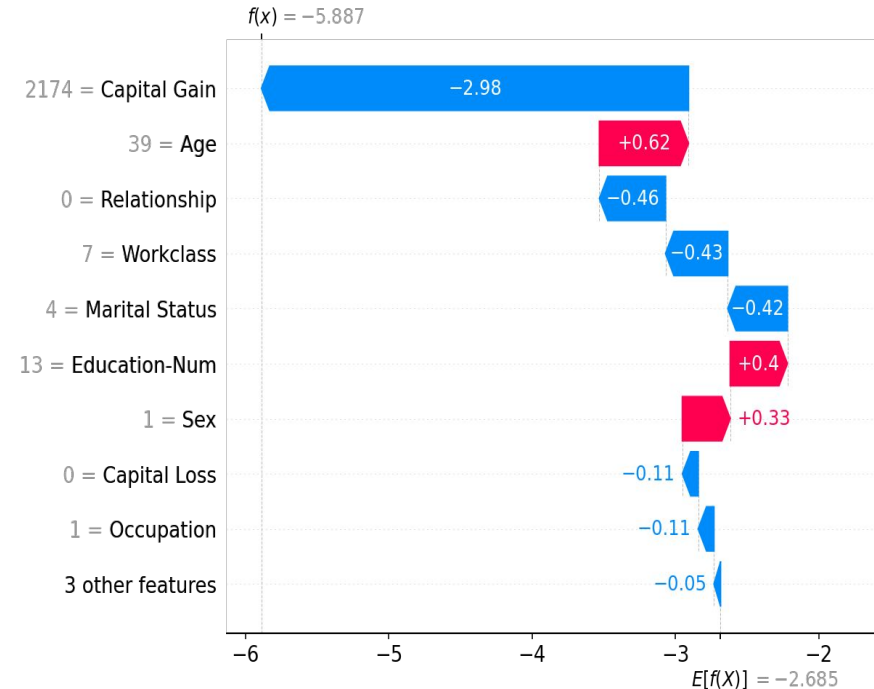


2. Local Interpretation Plot : Waterfall plot

- **(shap.plots.waterfall):** Explains a single prediction by showing how each feature moves the model output from the base value (average) to the final predicted value.
- Waterfall plots are designed to display explanations for individual predictions, so they expect a single row of an Explanation object as input.
- The bottom of a waterfall plot starts as the expected value of the model output, and then each row shows how the positive (red) or negative (blue) contribution of each feature moves the value from the expected model output over the background dataset to the model output for this prediction.

Note that by default SHAP explains XGBoost classifier models in terms of their margin output, before the logistic link function. That means the units on the x-axis are log-odds units, so negative values imply probabilities of less than 0.5 that the person makes over \$50k annually. The gray text before the feature names shows the value of each feature for this sample.

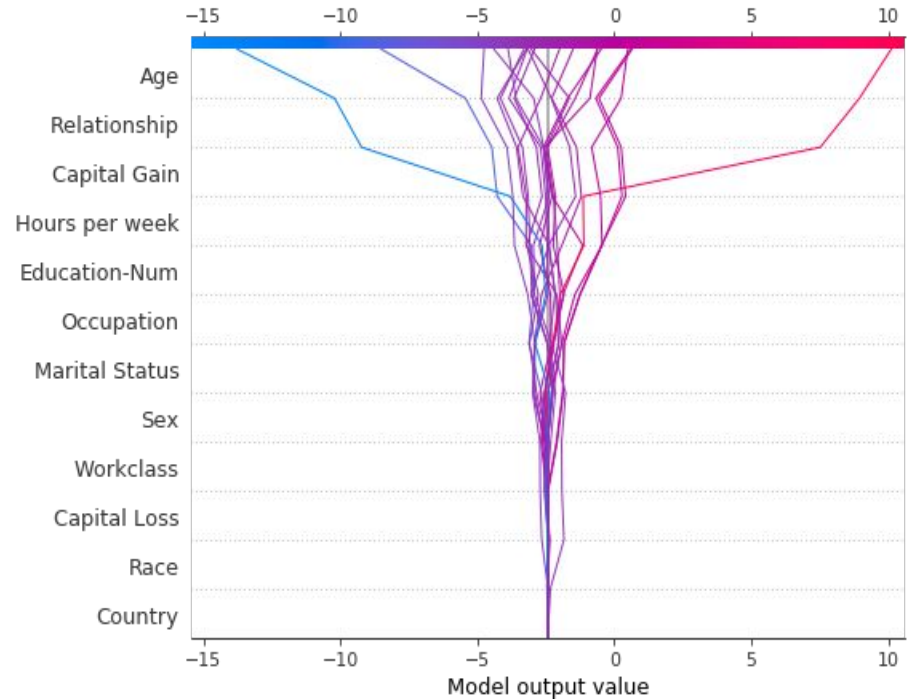
`shap.plots.waterfall(shap_values[0])`



2. Local Interpretation Plot : Decision Plot

- SHAP decision plots show how complex models arrive at their predictions (i.e., how models make decisions). This notebook illustrates decision plot features and use cases with simple
- Each plotted line on the decision plot shows how strongly the individual features contributed to a single model prediction, thus explaining what feature values pushed the prediction.
- Note: This plot isn't informative by itself; we use it only to illustrate the primary concepts. * The x-axis represents the model's output. In this case, the units are log odds. * The plot is centered on the x-axis at explainer.expected_value. All SHAP values are relative to the model's expected value like a linear model's effects are relative to the intercept. * The y-axis lists the model's features.
- Each observation's prediction is represented by a colored line
- At the bottom of the plot, the observations converge at explainer.expected_value.

`shap.decision_plot(expected_value, shap_values, features_display)`





THANK YOU

Computing Gradients for Attribution

Vanilla Gradients (Simonyan et al., 2013)

Compute $S_c = \partial f_c(x) / \partial x$ (Jacobian of class score w.r.t. input). The magnitude $|S_c|$ indicates pixel importance. Problem: gradient saturation — for ReLUs, gradients vanish in already-active regions. Result: noisy, inconsistent saliency maps that highlight edge artifacts rather than semantic content.

SmoothGrad (Smilkov et al., 2017)

$$SG(x) = (1/n) \sum \partial f(x + \epsilon) / \partial x, \quad \epsilon \sim N(0, \sigma^2)$$

Averages n noisy gradient maps. Dramatically reduces noise without changing the underlying method. σ is a critical hyperparameter — too large blurs semantics, too small adds noise.

Guided Backpropagation (Springenberg et al., 2014)

Modifies backprop through ReLUs: only pass gradients where both upstream gradient AND activation are positive. Creates sharper, more focused saliency maps — but research by Adebayo et al. (2018) showed they are independent of the model's weights (model-insensitive). A cautionary tale in XAI evaluation.

⚠ Sanity Checks are Essential

Adebayo et al. (2018) 'Sanity Checks for Saliency Maps': many attribution methods produce visually appealing maps that are completely insensitive to model parameters or data labels. Always verify: does the saliency map change when you randomize the model or scramble labels?

Integrated Gradients — Sundararajan, Taly & Yan (2017)

Core Formula

$$\text{IG}_i(\mathbf{x}) = (\mathbf{x}_i - \mathbf{x}'_i) \times \int_0^1 [\partial F(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}')) / \partial \mathbf{x}_i] d\alpha$$

where $\mathbf{x}'$ is the baseline (typically a black image), $\mathbf{x}$ is the input, α interpolates between them, and the integral is approximated with $m=50-300$ steps of Riemann summation. Each pixel gets credit proportional to the average gradient along the interpolation path.

Axiomatic Guarantees

Completeness:

Attributions sum to $F(\mathbf{x}) - F(\mathbf{x}')$. All credit is accounted for.

Sensitivity:

If $F(\mathbf{x}) \neq F(\mathbf{x}')$ for pixel i , it gets non-zero attribution.

Implementation Invariance:

Two models with same I/O get same attributions.

Linearity:

Attributions are linear in the function.

Baseline Choice Matters

- **Black image (zeros):**
Standard for vision; assumes zero = no information
- **White image (ones):**
Alternative; useful for dark images
- **Gaussian noise:**
Averages over multiple noisy baselines — more robust
- **Training set mean:**
Information-theoretically justified (Sturmfels et al.)
- **Maximum distance baseline:**
Selects baseline maximizing completeness gap

Variants & Extensions

Blur IG:

Use progressively blurred image as baseline path (Xu et al., 2020)

Expected IG:

Baseline = random training sample, averaged over distribution

Guided IG:

Chooses path through input manifold, not straight-line interpolation

XRAI:

Segments image using IG, then ranks segments (Kapishnikov et al., 2019)

Localizing Evidence in Convolutional Networks

Original CAM (Zhou et al., 2016)

Class Activation Maps require a specific architecture: Global Average Pooling (GAP) $\rightarrow$ FC layer $\rightarrow$ softmax. The CAM for class c is: $M_c(x,y) = \sum_k w_c^k \cdot f_k(x,y)$ where w_c^k are FC weights and f_k are conv feature maps at location (x,y) .

Limitation: only works with GAP-based architectures. Cannot explain VGG, AlexNet, etc.

GradCAM (Selvaraju et al., 2017)

Architecture-agnostic generalization of CAM. Steps:

1. Forward pass image x
2. Compute gradient of class score y_c w.r.t. last conv feature maps A^k
3. Global average pool gradients: $\alpha_c^k = (1/Z) \sum_i \partial y_c / \partial A^k_{ij}$
4. Weighted combination: $L^c = \text{ReLU}(\sum_k \alpha_c^k \cdot A^k)$

ReLU keeps only positive influences. Upsampled to input resolution via bilinear interpolation.

GradCAM Family of Methods

- Smooth GradCAM++ (Omeiza, 2019): combines SmoothGrad noise averaging with GradCAM++
- GradCAM++ (Chattopadhyay, 2018): better localization for multiple instances of same class
- Score-CAM (Wang, 2020): removes gradient dependency, uses activation maps as perturbation masks
- Eigen-CAM (Muhammad, 2020): uses principal components of activation maps — no class-specific gradients

Shapley-Based Attribution — Game-Theoretically Grounded

The Shapley Value Formula

$$\phi_i(f, x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \cdot [f(S \cup \{i\}) - f(S)]$$
 — The average marginal contribution of feature i over all possible subsets S of other features.

KernelSHAP

Model-agnostic. Approximates Shapley values by solving a weighted linear regression over feature subsets. Each subset = masked image. Works for any model but slow (requires many model calls).

DeepSHAP

Combines DeepLIFT with Shapley values. Uses a modified backpropagation that propagates contributions relative to a reference (baseline). Fast for neural networks — 10–100x faster than KernelSHAP.

GradientSHAP

Approximates DeepSHAP by sampling baselines from a distribution and computing expected gradients. Handles model non-linearities better than pure gradient methods while retaining speed.

Partition SHAP

For images: hierarchically partitions superpixels using the Owen value (a generalization of Shapley values for hierarchically structured data). Captures spatial dependencies between pixels.

Pixel attribution maps look convincing — but 'visually appealing' $\neq$ 'faithful'. Rigorous evaluation requires metrics that test whether the highlighted regions actually drive model decisions.

Pixel Deletion / Insertion

Progressively delete (or insert) most-important pixels. Faithful maps should cause steep accuracy drop on deletion and steep rise on insertion. AOPC (Area Over Perturbation Curve) summarizes performance.

Infidelity (Yeh et al., 2019)

Measures expected squared difference: $E[(\phi \cdot \epsilon - (f(x) - f(x - \epsilon)))^2]$. Faithful attribution should match model's sensitivity to small perturbations ϵ .

Sensitivity (Max-Sensitivity)

Max change in attribution map under small input perturbation: $\max_{\|\delta\| < r} \|A(x) - A(x + \delta)\|$. Low sensitivity = robust, consistent attributions.

Sanity Checks (Adebayo, 2018)

Randomize model weights (all layers or cascaded) and check if attribution changes. Good methods should produce random-looking maps for randomized models — many popular methods fail this test.

Localization (Pointing Game)

When ground-truth object bounding boxes are available — does the max-attribution pixel fall inside the box? Higher accuracy = better localization. Tests class-discriminateness.

Human Evaluation

Show attribution maps to domain experts (radiologists, ecologists). Do they agree? Captures semantic coherence that automated metrics miss — but expensive and subjective.

Choosing the Right Attribution Method

Method	Speed	Axioms	Architecture	Best For
Vanilla Gradients	⚡ Fastest	None	Any	Quick prototyping
SmoothGrad	⚡ Fast	None	Any	Cleaner gradient maps
Integrated Gradients	🔄 Medium	Completeness, Sensitivity	Any	Production / NLP / Vision
GradCAM	⚡ Fast	None	CNNs only	Coarse spatial localization
DeepSHAP	🔄 Medium	Shapley axioms	Neural nets	Fair attribution + fairness
KernelSHAP	🐢 Slow	Shapley axioms	Any model	Tabular / small images
LIME (image)	🐢 Slow	Local fidelity	Any	Superpixel-level explanations

Recommendation: Use Integrated Gradients as a reliable default; supplement with GradCAM for spatial localization and KernelSHAP when axioms are critical for stakeholder trust.

03

Detecting Concepts

Do networks encode human-interpretable semantic concepts?

Rather than attributing predictions to raw pixels, concept-based methods test whether networks encode human-defined abstractions — 'stripes', 'age', 'fur'.

This bridges low-level features and high-level reasoning, making explanations semantically meaningful to domain experts.

TCAV — Testing with Concept Activation Vectors

Kim et al. (2018). Define a concept using positive/negative example images → train a linear probe (CAV) in activation space → measure directional sensitivity (TCAVscore) of model predictions to that concept. Example: Is the Inception model sensitive to 'stripes' when classifying zebras?

ACE — Automatic Concept-based Explanations (Ghassemi et al., 2019)

Automatically discovers concepts by segmenting and clustering images from the same class, then identifies the most important clusters using TCAV — no manual labeling required.

Kim et al. (2018) ICML · Ghassemi et al. (2019) NeurIPS · Molnar (2023) Ch. 10.3

TCAV Pipeline

1

Define Concept

Collect images exemplifying the concept and random counterexamples.

2

Extract Activations

Pass images through network; extract intermediate layer activations.

3

Train CAV

Fit a linear classifier in activation space. Normal vector = CAV.

4

Compute TCAV Score

Fraction of inputs where gradient is positive in CAV direction for a class.

TCAV Score: Mathematical Formulation

Formal Definition of TCAV Score

$$\text{TCAV}_{\{q,C,l,k\}}(X_k) = |\{x \in X_k : S_{\{C,l,k\}}(x) > 0\}| / |X_k|$$

where $S_{\{C,l,k\}}(x) = \nabla h_{\{l,k\}}(f_l(x)) \cdot v^l_C$ is the directional derivative of the class logit k in the direction of the CAV v^l_C at layer l . A TCAV score of 1.0 means all inputs of class k are positively influenced by concept C .

Statistical Significance Testing

TCAV tests conceptual sensitivity against a null distribution generated from random concept vectors. A two-sided t-test determines if the TCAV score is significantly above 0.5. This prevents false positives from random CAV directions — only statistically significant concepts pass.

Multiple testing correction (Bonferroni or FDR) is recommended when testing many concepts simultaneously.

Real-World Applications

- Medical: 'Does the model use lesion color to diagnose melanoma?' — dermatology model audit
- Fairness: 'Is the face recognition model sensitive to skin tone?' — bias detection
- Scientific: 'Does a protein structure model encode secondary structure concepts?'
- Quality assurance: Verify that a defect detector uses the expected defect morphologies

ACE and Beyond: Discovering Concepts Without Labels

ACE Pipeline (Ghassemi et al., 2019)

1. Segment class images at multiple scales (superpixels via SLIC) → 2. Extract activation vectors for each segment → 3. Cluster segments using k-means in activation space → 4. Each cluster = a discovered concept → 5. Rank concepts by TCAV importance → 6. Return top-k visual concept patches

ConceptShap · Yeh et al., 2020

Assigns Shapley values to automatically discovered concepts — combining concept discovery with game-theoretic attribution for fair credit among concepts.

Net2Vec · Fong & Vedaldi, 2018

Learns a continuous concept embedding for each neuron by projecting onto a rich concept vocabulary. Handles the polysemanticity problem by assigning partial concept weights.

Concept Bottleneck Models · Koh et al., 2020

Architecture-level solution: predict human-defined concepts at an intermediate layer, then predict the class from concepts. Fully concept-based and allows concept-level intervention.

CRAFT · Fel et al., 2023

Uses NMF on class-conditioned feature maps to discover non-negative, sparse concepts. Pairs with Sobol attribution to rank concept importance. State-of-art unsupervised concept extraction.

03 DETECTING CONCEPTS · Concept Bottleneck Models

Concept Bottleneck Models (CBMs, Koh et al. 2020) embed concept-based interpretability directly into model architecture, rather than applying it as a post-hoc explanation. The model is forced to express its reasoning in human-defined terms.



Bottleneck: c_1 = 'striped', c_2 = 'four-legged', c_3 = 'black & white', ...

Advantages

- Human-interpretable intermediate layer ✓
- Supports concept intervention at test time ✓
- Allows 'what-if' reasoning: flip c_3 = 'spotted' → Prediction changes to 'leopard' ✓
- Auditability: easily verify concept predictions are sensible ✓

Limitations & Extensions

- Requires labeled concepts — expensive annotation ✓
- Performance-interpretability trade-off (accuracy gap) ✓
- Leakage: residual connections can route information around concepts ✓
- Post-hoc CBMs: Yuksekgonul et al. (2022) retrofits CBMs without retraining ✓

Variants

- Hybrid CBMs: extra 'unsupervised' dimensions alongside concepts ✓
- Label-free CBMs: use LLM-generated concept descriptions + CLIP embeddings ✓
- Probabilistic CBMs: Bayesian concept uncertainty propagation ✓

03 DETECTING CONCEPTS · Language-Guided Concept Detection

The emergence of vision-language models (CLIP, ALIGN) enables zero-shot concept detection using natural language descriptions — eliminating the need for labeled concept images entirely.

Text-Guided TCAV

Replace concept example images with CLIP embeddings of text descriptions. 'Stripes' concept = CLIP('a photo of stripes'). No image collection needed — enables rapid, scalable concept testing.

Label-Free CBM

Yuksekgonul et al. (2022): use GPT-3 to generate descriptive concept lists for each class, then use CLIP to score each concept's alignment with image activations. Builds a concept bottleneck without any human annotation.

TCAV-ICE (Image-Concept Explanation)

Combines image segmentation with CLIP concept scores to generate visual concept heatmaps. Each pixel gets a score for each concept — richer than binary presence/absence.

Sparse Concept Bottlenecks

Use L1 regularization to enforce sparse concept selection. **Only the most relevant concepts are activated per prediction — mimicking human diagnostic reasoning with a focused set of evidence.**

Concept Embedding Models (CEM)

Zarlenga et al. (2022): represent each concept as a continuous embedding vector rather than a binary activation. Enables partial concept satisfaction and uncertainty-aware predictions.

LLM-Explained Neurons

Hernandez et al. (2022) MILAN: automatically generate natural language descriptions of neurons using an LLM — bridges mechanistic interpretability and human-readable explanations at scale.

From Pixels to Meaning: The Concept Paradigm

Category	Methods	Strengths	Limitations
Post-hoc, Supervised	TCAV, ConceptShap, Net2Vec	Flexible — test any concept on any model	Requires manual concept labeling effort
Post-hoc, Unsupervised	ACE, CRAFT, Clustering	No annotation needed — discovers concepts automatically	Concepts may not match human intuition
Architectural	CBM, CEM, Hybrid CBM	Concept interventions at test time; fully transparent	Performance gap; requires retraining
Language-Guided	Label-free CBM, MILAN, Text-TCAV	Scalable — LLM generates concept vocabulary automatically	Dependent on quality of LLM descriptions

Open Challenge: Concept completeness — for a given task, what is the minimum set of concepts that fully explain the model's behavior? Ideally, concept attributions should sum to the full prediction — analogous to completeness in SHAP.

04

Adversarial Examples

Imperceptible perturbations that fool neural networks

04 ADVERSARIAL EXAMPLES · Overview

Goodfellow et al. (2014) showed that adding carefully crafted imperceptible noise to an image causes high-confidence misclassification. This reveals that networks rely on high-frequency, non-robust features invisible to humans — exposing a fundamental gap between machine and human perception.

FGSM

Fast Gradient Sign Method · Goodfellow et al., 2014

Single-step: $x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$. Fast but weak — easy to defend against.

PGD

Projected Gradient Descent · Madry et al., 2018

Iterative FGSM with projection onto ϵ -ball. State-of-art white-box attack; robustness benchmark.

C&W

Carlini & Wagner Attack · Carlini & Wagner, 2017

Optimization attack minimizing perturbation norm while maximizing misclassification confidence.

Defenses

- ▶ Adversarial Training — augment with adversarial examples (Madry et al.)
- ▶ Certified Defenses — provable robustness via randomized smoothing
- ▶ Feature Squeezing — detect adversarial inputs via color depth reduction

XAI Connection

Adversarial examples are diagnostic tools. Saliency maps of adversarial inputs reveal which features the model actually uses vs. which humans expect. Ilyas et al. (2019): 'Adversarial examples are features, not bugs'.

Classifying Adversarial Attacks Along Four Axes

Knowledge (Threat Model)

White-box

Full access to model weights and gradients. Strongest attacks: FGSM, PGD, C&W.

Black-box

Only input-output access. Use transfer attacks or query-based optimization (ZOO, Square Attack).

Gray-box

Partial knowledge — e.g., architecture known but not weights. Ensemble-based transfer.

Perturbation Budget (Norm)

L^∞ norm

Bounded per-pixel perturbation. Most common — each pixel changes by at most ϵ .

L_2 norm

Bounded total energy. Smaller imperceptible perturbations concentrated in fewer pixels.

L_0 norm

Bounded number of pixels changed. Sparse attacks — change as few pixels as possible (Jacobian attack).

Target Specificity

Untargeted

Cause any misclassification. Easiest to generate — minimize correct class score.

Targeted

Cause a specific wrong class. Harder — maximize target class score while perturbing minimally.

Physical-world attacks (Kurakin et al., 2017): printed adversarial patches fool cameras in real environments — stop signs misclassified, self-driving cars confused.

Why Do Adversarial Examples Exist? (Ilyas et al., 2019)

The Core Claim

Adversarial perturbations are not random noise — they correspond to useful-but-non-robust features that the model has legitimately learned from the training data. These features are predictive (correlated with labels) but imperceptible and brittle. The network is doing what it was trained to do — exploiting all available predictive signals, robust and non-robust alike.

Robust vs. Non-Robust Features

Robust features: semantically meaningful, stable under perturbation — 'fur texture', 'beak shape'. These align with human perception.

Non-robust features: statistically predictive but fragile — imperceptible correlations in pixel space that change under small perturbations. Adversarial examples exploit these.

Key experiment: training on adversarially perturbed images with flipped labels (non-robust features relabeled) — models still learn to generalize! Confirms these are real features.

XAI Implications

1. Attribution maps of adversarial images reveal the non-robust features the model used — exposing what the model 'actually cares about'
2. Standard saliency methods often look similar on clean and adversarial inputs — a failure mode indicating they don't track decision-relevant features
3. Adversarial training changes which features are used — attribution maps shift from non-robust to semantic features post-robustification
4. Adversarial examples as XAI probes: design perturbations to test if model uses specific features (e.g., remove texture — does class change?)

Building Defenses Against Adversarial Attacks

Adversarial Training (AT)

Empirical

Madry et al. (2018): augment training with PGD-generated adversarial examples. Min-max optimization: $\min_{\theta} \max_{\{\delta \in \Delta\}} L(\theta, x + \delta, y)$. Strongest empirical defense — but computationally expensive (7-10x slower training) and increases standard accuracy cost.

Randomized Smoothing

Certified

Cohen et al. (2019): smooth classifier $g(x) = \operatorname{argmax}_c P(f(x + \epsilon) = c)$ for $\epsilon \sim N(0, \sigma^2 I)$. Certifies robustness within radius $r = \sigma \cdot \Phi^{-1}(\bar{p}_A)$ where $\bar{p}_A$ is the top-class probability. Provably correct — no adversarial example can fool it within this radius.

Input Preprocessing

Empirical

Feature squeezing (Xu et al.): apply bit-depth reduction and spatial smoothing, then compare predictions on original and preprocessed inputs. Inconsistency signals adversarial input. Simple but breakable by adaptive attacks that account for preprocessing.

Certified Training (IBP/CROWN)

Certified

Interval Bound Propagation: propagate input ϵ -boxes through each layer to get certified output bounds. Train to minimize certified loss — guarantees certification at training time. Tight certifications remain challenging for large models.

Beyond Digital: Real-World and Universal Adversarial Threats

Physical-World Attacks

Printed Patches (Brown et al., 2017)

A small printed patch placed anywhere in a scene causes misclassification — works across cameras, angles, lighting.

Adversarial Stop Signs (Evtimov et al., 2017)

Stickers on stop signs fool road-sign classifiers into reading them as speed limit signs — safety-critical.

Face Adversarial Glasses (Sharif et al., 2016)

Printed eyeglass frames fool face recognition systems into identifying the wearer as a target person.

3D Adversarial Objects (Athalye et al., 2018)

'Expectation over Transformations' (EOT) creates objects that fool classifiers from all viewpoints in 3D.

Universal Perturbations (Moosavi-Dezfooli et al., 2017)

A single perturbation vector δ ($\|\delta\|_2 < \xi$) causes misclassification for most images from the data distribution: $P_{x \sim D}[f(x+\delta) \neq f(x)] \geq 1-\delta$

Algorithm: iteratively aggregate per-image minimal perturbations until universal fooling rate exceeds threshold. Result: a 'trojan' perturbation that generalizes — transferable across models.

Implication: a single printed image (like a QR-code poster) can fool cameras everywhere in a space — mass adversarial attack.

Backdoor / Trojan Attacks

Chen et al. (2017): inject a trigger pattern (e.g., a specific pixel pattern) into training data with poisoned labels. The model behaves normally until the trigger appears — then misclassifies with high confidence. Supply-chain attack — affects models trained on poisoned datasets. Detection requires deep model inspection or activation clustering (Tran et al., 2018).

Adversarial Examples for Understanding Models

Beyond security, adversarial examples serve as powerful interpretability probes. By carefully designing perturbations, we can test model hypotheses and understand decision boundaries in ways traditional XAI methods cannot. ✓

Decision Boundary Exploration

Find the minimal perturbation to flip a prediction (DeepFool, C&W). The direction of this perturbation reveals the geometry of the model's decision boundary — which features are 'close' to changing the outcome.

Counterfactual Explanations

Adversarial-style optimization: find x' closest to x such that $f(x') = c_{\text{target}}$. These are contrastive explanations — 'what minimal change would make this loan application approved?' — actionable for users.

Feature Importance via Perturbation

Systematically perturb feature groups and observe prediction change (cf. LIME, SHAP). Adversarial-strength perturbations guarantee feature importance is detected even for small-effect features.

Robustness as Interpretability

Tsipras et al. (2019): **models trained for adversarial robustness develop more human-aligned feature detectors** — gradients become visually interpretable. Robustness and interpretability may be aligned goals.

Sensitivity Analysis

Input gradients (a form of adversarial perturbation) approximate local decision boundaries. Large gradients = model is sensitive = that feature is important for the local explanation.

Model Comparison via Transfer

If adversarial examples transfer across models, **those models share similar decision boundaries/features** — a quantitative measure of model similarity useful for ensemble design.

05

Influential Instances

Which training examples most shaped the model's predictions?

Data-Centric Explanations

Influential instance methods ask: 'If this training example were removed, how would the model's predictions change?' These explain through the lens of training data — connecting predictions to specific examples.

Influence Functions

Koh & Liang (2017): measures how the model's prediction on a test point changes when training point z_i is up-weighted. Uses the Hessian: $\text{Inf}(z, z_{\text{test}}) = -\nabla_{\theta} L(z_{\text{test}})^T H_{\theta}^{-1} \nabla_{\theta} L(z)$. **Reveals mislabeled training examples and model vulnerabilities.**

Prototypes & Criticisms — MMD-critic

Kim et al. (2016): Maximum Mean Discrepancy selects prototypes (representative examples) and criticisms (examples poorly represented by prototypes). Provides intuitive data summary explaining what the model 'knows' vs. where it may fail.

Applications

Label Error Detection

High negative self-influence = likely mislabeled — clean datasets.

Debugging Failures

Most positively influential training examples reveal what pattern confused the model.

Model Security

Data poisoning exploits influential instances — inject high-influence corrupted points.

Counterfactual Data

'Which training example, if changed, would flip this prediction?'

Mathematical Derivation of Influence Functions

Classical Statistics Foundation

Up-weight training point z by ε : $\theta_\varepsilon = \operatorname{argmin}_\theta [\sum_i L(z_i, \theta) + \varepsilon \cdot L(z, \theta)] / n$

First-order Taylor expansion gives: $d\theta_\varepsilon/d\varepsilon|_{\varepsilon=0} = -H_\theta^{-1} \nabla_\theta L(z, \theta)$ where $H_\theta = (1/n) \sum_i \nabla^2_\theta L(z_i, \theta)$ is the Hessian.

Influence on test loss: $I_{\text{up}, \text{loss}}(z, z_{\text{test}}) = -\nabla_\theta L(z_{\text{test}})^T H_\theta^{-1} \nabla_\theta L(z)$ [computed without retraining!]

Computing $H^{-1}v$ Efficiently

Direct Hessian inversion is $O(p^3)$ — infeasible for large neural nets.

Practical approximations:

- LiSSA (Agarwal et al.): **stochastic estimation using recursive Neumann series** — $O(p)$ per step, requires assumptions of strong convexity
- Arnoldi iteration: Krylov subspace method for top eigenvectors (Schioppa et al., 2022)
- EK-FAC: Kronecker-factored Hessian approximation — state-of-art for transformers

Known Failure Modes

Basu et al. (2021): 'Influence Functions in the Wild' — influence functions fail badly for:

- Deep overparameterized networks (non-convex loss landscape)
- Non-convergent / early-stopped training
- Modern architectures with batch norm, skip connections

Koh et al. empirical observations often don't replicate at scale — requires careful validation.

Relatif (Barshan et al., 2020): 'Relatif — Identifying Explanatory Training Samples via Relative Influence' — computes relative influence against a reference class to find training samples responsible for systematic prediction errors across a test set, not just one prediction.

MMD-critic: Summarizing Data with Representative Examples

Maximum Mean Discrepancy (MMD)

$MMD^2 = E[k(x, x')] - 2E[k(x, z)] + E[k(z, z')]$ measures distance between distributions P (data) and Q (selected examples) in RKHS. Prototypes minimize MMD — most representative. Criticisms are points with high MMD residual — poorly explained by prototypes.

Algorithm

1. Select m prototypes greedily: each step adds the point minimizing MMD^2 between selected set and full dataset
2. Select k criticisms: points with largest MMD residual (maximize distance from prototype coverage)
3. Both selections use kernel function $k(x, x')$ — typically RBF kernel $k(x, x') = \exp(-||x - x'||^2 / 2\sigma^2)$
4. Hyperparameters: m (num prototypes), k (num criticisms), σ (kernel bandwidth)
5. Visualize prototypes as the 'canonical' examples of each class/cluster

Applications

Model Explanation

Show users representative training examples similar to their query — case-based reasoning, common in medical diagnosis.

Dataset Quality Assessment

Criticisms = outliers, annotation errors, distribution shift. MMD-critic provides unsupervised dataset audit.

Class Imbalance Visualization

Prototypes from underrepresented classes have high criticisms nearby — reveals data sparsity.

Debugging Predictions

Find training prototypes most similar to a misclassified test point — reveals what 'confused' the model.

05 INFLUENTIAL INSTANCES · Example-Based Explanations

Example-based explanations are psychologically intuitive: 'This prediction is based on patterns similar to these training examples.' They are especially powerful in domains where humans reason by analogy — medicine, law, finance.

k-Nearest Neighbors in Representation Space

Retrieve training examples with closest embedding distance to the test point (L2, cosine). Different from raw pixel distance — comparison is semantically meaningful. Used in retrieval-augmented explanation systems.

InfluenceMatch (Hanawa et al., 2021)

Extends influence functions to retrieve the top-k most influential training examples for a given test prediction — combining mathematical rigor of influence functions with the intuition of example retrieval.

This Looks Like That (Chen et al., 2019)

ProtoNet architecture: explicitly learns prototypical parts within training examples. Explanation format: 'This bird looks like THAT bird training example in THIS part of the image' — fully interpretable prototype-based model.

Counterfactual Data Augmentation

Find training examples where one feature differs but label stays same — validates that model uses the right features. Kaushik et al. (2020): human-written counterfactuals reduce spurious correlations in NLP models.

Dark Side: When Influential Instances Are Weaponized

Threat Model: Data Poisoning

An adversary with write access to training data can craft highly influential poisoned examples that degrade model accuracy, induce targeted misclassification, or cause model to discriminate against protected groups — without access to model weights.

Influence-Based Poisoning (Koh et al., 2022)

Use influence functions to craft training points that maximize an adversarial objective (e.g., misclassifying a target test point). Demonstrates that even a small fraction (<1%) of poisoned data can flip predictions for carefully chosen targets.

Subpopulation Attacks (Jagielski et al., 2021)

Target a demographic subgroup — inject influential training points that cause the model to fail specifically for that subgroup. Particularly harmful in loan approval, recidivism prediction, and medical screening systems.

Fairness via Influence Analysis

Conversely, use influence functions to identify training examples that contribute disproportionately to unfair predictions. Remove or down-weight these influential points to reduce disparity — data-centric debiasing approach.

Dataset Auditing with TracIn (Pruthi et al., 2020)

TracIn traces influence across training checkpoints: $\text{TracIn}(z, z_{\text{test}}) = \sum_t \alpha_t \nabla L(z_{\text{test}})^T \nabla L(z)$. More computationally feasible than full Hessian inversion, and empirically better at identifying mislabeled examples.

When Models Memorize Individual Training Examples

Individual Memorization (Feldman, 2020)

Some training examples are uniquely memorized — the model's prediction on them changes drastically if they are removed from training. Feldman shows this is not a bug: long-tail examples require memorization to achieve good accuracy on rare subpopulations. Memorization = influence = necessity for long-tail generalization.

Membership Inference (MIA)

Shokri et al. (2017): can an attacker determine if a specific example was in the training set? High influence = high memorization = high MIA vulnerability. Link between influence, privacy, and data protection (GDPR Article 17: right to erasure).

Machine Unlearning — The Emerging Frontier

Exact Unlearning:

Retrain from scratch without the target data point. Guarantees complete removal — computationally expensive. Feasible with SISA training (Bourtoule et al., 2021) using sharded, isolated, sliced, and aggregated models.

Scrubbing:

Fine-tune on remaining data with gradient ascent on the forgotten data — 'unlearning' through gradient steps. Balance between forgetting target data and retaining performance on rest of training data.

Approximate Unlearning:

Newton step (Guo et al., 2020): update model parameters using influence function direction to approximate removal without full retraining. Fast but approximate — useful for large models where retraining is infeasible.

Selective Synaptic Dampening:

Maini et al. (2023): selectively dampen model weights most critical for memorization of specific examples — targeted forgetting preserving general capabilities. Important for LLM copyright and privacy compliance.

Category	Key Method	Scope	Model Access	Pros	Cons
Learned Features	Activation Maximization, Network Dissection	Global	Gradient access	Reveals what neurons detect	Polysemanticity; regularization needed
Pixel Attribution	Integrated Gradients, GradCAM, SHAP	Local	Gradient or black-box	Pixel-level precision; axiom guarantees	Sanity check failures; baseline sensitivity
Concept Detection	TCAV, CBM, ACE, CRAFT	Both	Activation extraction	Human-interpretable semantics; interventionable	Needs concept labels or LLM; concept completeness
Adversarial Examples	FGSM, PGD, C&W, Universal Perturbations	Local	White/black/gray box	Exposes model fragility; probes features	Not directly a user-facing explanation
Influential Instances	Influence Functions, MMD-critic, TracIn	Local	Gradient + Hessian	Data-centric; catches mislabels; unlearning	Hessian approximation quality; non-convex failures

Methods can and should be combined: e.g., use GradCAM to localize → TCAV to verify concepts → influence functions to trace back to training data

XAI is Not One Tool — It's a Toolkit

Learned Features reveal the model's vocabulary

Activation maximization, network dissection, and circuit analysis show that hierarchical representations are real and partially interpretable — but polysemanticity remains a core challenge requiring mechanistic approaches.

Adversarial Examples expose fragility and features

They reveal the mismatch between human-robust and machine-learned features — simultaneously a security threat, an interpretability tool, and a gateway to robust model training.

Pixel Attribution localizes evidence

Integrated Gradients and GradCAM answer 'why this prediction?' — but evaluation (sanity checks, faithfulness metrics) is as important as the attribution method itself. Many popular methods fail basic reliability tests.

Influential Instances connect model behavior to data

Influence functions, Tracln, and MMD-critic enable data auditing, error diagnosis, fairness analysis, and machine unlearning — the foundation for responsible ML data governance.

Concept probing bridges human and machine understanding

TCAV and Concept Bottleneck Models let domain experts query model reasoning in their own vocabulary — LLMs now enable this at scale without manual concept labeling.

"Explainability is not a luxury — it is a prerequisite for responsible deployment of machine learning systems." — Arrieta et al. (2020)